COVID-19 Clinical Trials ; Exploratory Data Analysis

Author	Ibrahim A. Mikail
Internship	Unified Mentor ; Healthcare Data Analyst
Project	1 of 5
Tools	Python · Pandas · SQLite · Matplotlib · Seaborn
Dataset	ClinicalTrials.gov via Kaggle
Date	May 2026

Project Overview

ClinicalTrials.gov is a database of privately and publicly funded clinical studies maintained by the U.S. National Library of Medicine. This notebook performs a full exploratory data analysis on COVID-19 clinical trials registered on the platform, covering data cleaning, SQL-based querying, univariate and bivariate analysis, and time series trends.

Objectives

- Understand the landscape of COVID-19 clinical trials globally
- Identify patterns in trial status, phases, gender, and funding
- Analyze geographic distribution of trials
- Explore trends in trial registrations over time

Project Setup

Setting up the project environment, folder structure, and all required libraries.

In [1]:
```
# Project Setup & Folder Structure
import os
from pathlib import Path

# Defining project root relative to notebook location
ROOT    = Path().resolve()
DATA    = ROOT / "data"
OUTPUTS = ROOT / "outputs"
DB_PATH = ROOT / "covid_trials.db"

# Create folders if they don't exist
for folder in [DATA, OUTPUTS]:
```

```
    folder.mkdir(parents=True, exist_ok=True)
    print(f"✓ {folder}")

print(f"\nProject root : {ROOT}")
print(f"Database will be created at : {DB_PATH}")
```

```
✓ C:\Users\DELL\Documents\Data Science Projects\Unified Mentor Project\covid_clinical_trials_eda\data
✓ C:\Users\DELL\Documents\Data Science Projects\Unified Mentor Project\covid_clinical_trials_eda\outputs

Project root : C:\Users\DELL\Documents\Data Science Projects\Unified Mentor Project\covid_clinical_trials_eda
Database will be created at : C:\Users\DELL\Documents\Data Science Projects\Unified Mentor Project\covid_clinical_trials_eda\covid_trials.db
```

In [2]:
```
# Importing libraries & Global Settings
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")


# Pandas display settings
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", "{:.2f}".format)

# Chart style
plt.rcParams["figure.figsize"] = (12, 5)
plt.rcParams["axes.spines.top"]   = False
plt.rcParams["axes.spines.right"] = False
sns.set_palette("Blues_r")

print("All libraries loaded")
```

```
All libraries loaded
```

Loading Data & Ingesting to SQLite

Loading the raw CSV into a Pandas DataFrame, then pushing it into a SQLite database so that all subsequent analysis will query from the database first before using Pandas.

In [3]:
```
# Loading CSV & Pushing to SQLite

# Loading CSV
df_raw = pd.read_csv(DATA / "covid_clinical_trials.csv")

# Connecting to SQLite
conn = sqlite3.connect(DB_PATH)

# Writing dataframe to SQLite as a table called 'trials'
df_raw.to_sql("trials", conn, if_exists="replace", index=False)
```

```
print(f"CSV loaded        : {df_raw.shape[0]:,} rows, {df_raw.shape[1]} columns")
print(f"SQLite table created 'trials'")
```

```
CSV loaded        : 5,783 rows, 27 columns
SQLite table created 'trials'
```

Initial Data Exploration

Taking a first look at the structure, columns, and data types.

In [4]:
```
# Initial Inspection

# Querying the first 5 rows directly from SQLite
df = pd.read_sql("SELECT * FROM trials LIMIT 5", conn)
df
```

Out[4]:

	Rank	NCT Number	Title	Acronym	Status	Study Results	Conditions
0	1	NCT04785898	Diagnostic Performance of the ID Now™ COVID-19...	COVID-IDNow	Active, not recruiting	No Results Available	Covid19
1	2	NCT04595136	Study to Evaluate the Efficacy of COVID19-0001...	COVID-19	Not yet recruiting	No Results Available	SARS-CoV-2 Infection
2	3	NCT04395482	Lung CT Scan Analysis of SARS-CoV2 Induced Lun...	TAC-COVID19	Recruiting	No Results Available	covid19
3	4	NCT04416061	The Role of a Private Hospital in Hong Kong Am...	COVID-19	Active, not recruiting	No Results Available	COVID
4	5	NCT04395924	Maternal-foetal Transmission of SARS-Cov-2	TMF-COVID-19	Recruiting	No Results Available	Maternal Fetal Infection Transmission|COVID-19...

In [5]:
```
# Shape & Column Names
df_raw.shape
```

Out[5]:
```
(5783, 27)
```

In [6]:
```
# Checking Data Types
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5783 entries, 0 to 5782
Data columns (total 27 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   Rank                     5783 non-null   int64  
 1   NCT Number               5783 non-null   object 
 2   Title                    5783 non-null   object 
 3   Acronym                  2480 non-null   object 
 4   Status                   5783 non-null   object 
 5   Study Results            5783 non-null   object 
 6   Conditions               5783 non-null   object 
 7   Interventions            4897 non-null   object 
 8   Outcome Measures         5748 non-null   object 
 9   Sponsor/Collaborators    5783 non-null   object 
 10  Gender                   5773 non-null   object 
 11  Age                      5783 non-null   object 
 12  Phases                   3322 non-null   object 
 13  Enrollment               5749 non-null   float64
 14  Funded Bys               5783 non-null   object 
 15  Study Type               5783 non-null   object 
 16  Study Designs            5748 non-null   object 
 17  Other IDs                5782 non-null   object 
 18  Start Date               5749 non-null   object 
 19  Primary Completion Date  5747 non-null   object 
 20  Completion Date          5747 non-null   object 
 21  First Posted             5783 non-null   object 
 22  Results First Posted     36 non-null     object 
 23  Last Update Posted       5783 non-null   object 
 24  Locations                5198 non-null   object 
 25  Study Documents          182 non-null    object 
 26  URL                      5783 non-null   object 
dtypes: float64(1), int64(1), object(25)
memory usage: 1.2+ MB
```

Missing Data Analysis

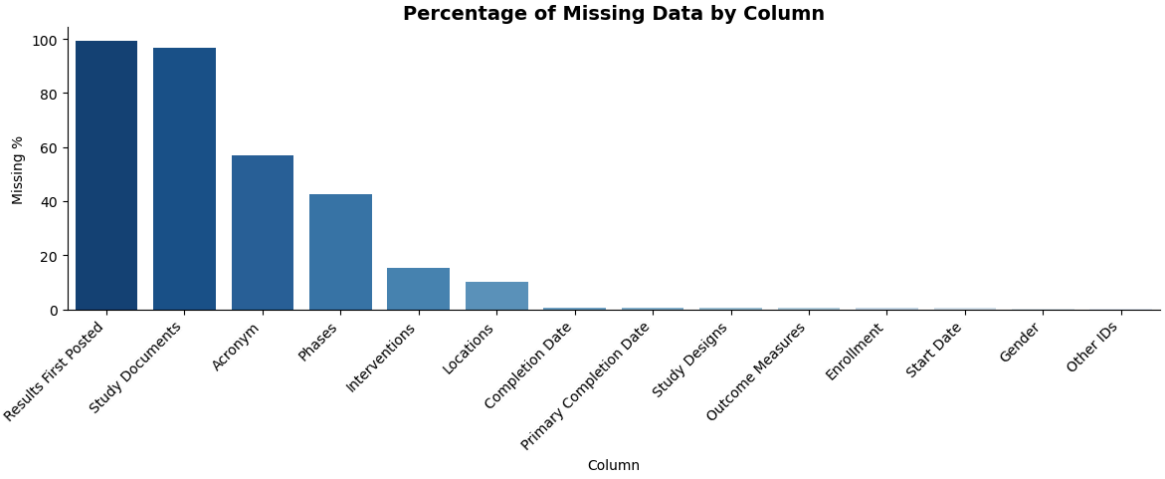
Before drawing any conclusions about missingness, i will quantify it first.

In [7]:
```
# Missing Data Percentage
missing = pd.DataFrame({
    "Missing Count" : df_raw.isnull().sum(),
    "Missing %" : (df_raw.isnull().mean() * 100).round(2)
}).sort_values("Missing %", ascending=False)

missing[missing["Missing %"] > 0]
```

Out[7]:

	Missing Count	Missing %
Results First Posted	5747	99.38
Study Documents	5601	96.85
Acronym	3303	57.12
Phases	2461	42.56
Interventions	886	15.32
Locations	585	10.12
Completion Date	36	0.62
Primary Completion Date	36	0.62
Study Designs	35	0.61
Outcome Measures	35	0.61
Enrollment	34	0.59
Start Date	34	0.59
Gender	10	0.17
Other IDs	1	0.02

In [8]:
```
# Visualizing Missing Data
missing_plot = missing[missing["Missing %"] > 0]

plt.figure(figsize=(12, 5))
sns.barplot(x=missing_plot.index, y=missing_plot["Missing %"], palette="Blues_r"
plt.title("Percentage of Missing Data by Column", fontsize=14, fontweight="bold"
plt.xlabel("Column")
plt.ylabel("Missing %")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "missing_data.png", dpi=150)
plt.show()
```



Initial Observations

- **Shape:** 5,783 trials and 27 columns

- **Columns to drop:** `Results First Posted` (99% missing) and `Study Documents` (97% missing) which are too sparse to impute meaningfully without destroying data integrity
- **Missing data classification:**
  - `Acronym` (57% missing) & likely **MAR**: smaller or international studies tend not to use acronyms, so missingness is related to other observed variables like country or sponsor
  - `Phases` (43% missing) & likely **MAR**: observational studies often don't have phases by design, so missingness is related to `Study Type`
  - `Interventions` (15% missing) & likely **MAR**: observational studies may not have formal interventions
  - `Locations` (10% missing) & likely **MAR**: missingness may relate to study size or sponsor type
  - `Results First Posted` (99% missing) & likely **MNAR**: trials without results simply haven't posted them yet, the missingness is related to the missing value itself
  - `Enrollment` (0.6% missing) & likely **MCAR**: small random missingness with no clear pattern
- **Only numeric column:** `Enrollment` will be imputed with median due to high skewness expected in trial sizes
- **Date columns** are stored as strings and will be converted to datetime in cleaning

---

Data Cleaning

Dropping unusable columns, handling missing values, converting data types, and engineering new features.

Dropping High-Missingness Columns & Duplicates

In [9]:
```
# Dropping Columns & Duplicates
df_clean = df_raw.copy()

# Dropping columns with >95% missing
df_clean.drop(columns=["Results First Posted", "Study Documents"], inplace=True)

# Dropping duplicates
before = df_clean.shape[0]
df_clean.drop_duplicates(inplace=True)
after = df_clean.shape[0]

print(f"✓ Duplicate rows removed : {before - after}")
print(f"✓ Shape after cleaning   : {df_clean.shape}")
```

```
✓ Duplicate rows removed : 0
✓ Shape after cleaning   : (5783, 25)
```

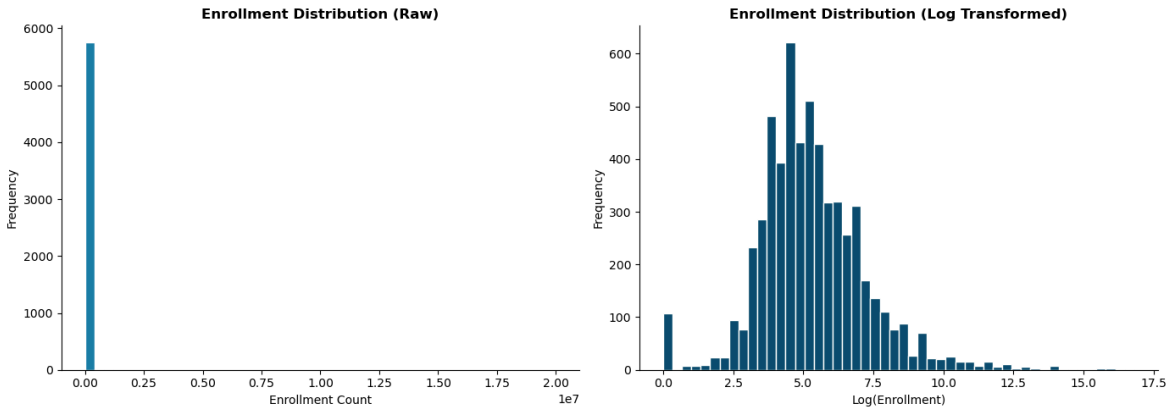
In [10]:
```
# Handling Missing Categorical Columns

# Imputing Columns classified as MAR with a missing indicator
cat_cols = ["Acronym", "Interventions", "Outcome Measures",
            "Gender", "Phases", "Study Designs", "Locations"]

for col in cat_cols:
    before = df_clean[col].isnull().sum()
    df_clean[col] = df_clean[col].fillna(f"Not Specified")
    print(f"✓ {col:<25} {before} nulls filled")
```

```
✓ Acronym                   3303 nulls filled
✓ Interventions             886 nulls filled
✓ Outcome Measures          35 nulls filled
✓ Gender                    10 nulls filled
✓ Phases                    2461 nulls filled
✓ Study Designs             35 nulls filled
✓ Locations                 585 nulls filled
```

In [11]:
```
# Handling Missing Numeric Column
print(f"Enrollment skewness : {df_clean['Enrollment'].skew():.2f}")

# Using median due to high skewness
median_enrollment = df_clean["Enrollment"].median()
df_clean["Enrollment"] = df_clean["Enrollment"].fillna(median_enrollment)

print(f"Median used for imputation : {median_enrollment}")
print(f"Remaining nulls : {df_clean['Enrollment'].isnull().sum()}")
```

```
Enrollment skewness : 34.07
Median used for imputation : 170.0
Remaining nulls : 0
```

In [12]:
```
# Enrollment Distribution
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Before log transform
axes[0].hist(df_clean["Enrollment"].dropna(), bins=50, color="#1a7ea8", edgecolo
axes[0].set_title("Enrollment Distribution (Raw)", fontweight="bold")
axes[0].set_xlabel("Enrollment Count")
axes[0].set_ylabel("Frequency")

# Log transform to handle skewness
axes[1].hist(np.log1p(df_clean["Enrollment"].dropna()), bins=50, color="#0a4f6e"
axes[1].set_title("Enrollment Distribution (Log Transformed)", fontweight="bold"
axes[1].set_xlabel("Log(Enrollment)")
axes[1].set_ylabel("Frequency")

plt.tight_layout()
plt.savefig(OUTPUTS / "enrollment_distribution.png", dpi=150)
plt.show()
```



Observations

- No duplicate rows found in the dataset
- All categorical missing values are filled with "Not Specified" as a missing indicator preserving the information that data was absent
- `Enrollment` has a skewness of 34.07 which is an extreme right skew caused by a small number of very large trials
- Median (170) was used for imputation instead of mean as it is robust to outliers

In [13]:
```
# Feature Engineering

# Extracting Country from Locations
df_clean["Country"] = df_clean["Locations"].apply(
    lambda x: x.strip().split(",")[-1].strip()
    if x != "Not Specified" else "Not Specified"
)

# Converting date columns to datetime
date_cols = ["Start Date", "Primary Completion Date",
             "Completion Date", "First Posted", "Last Update Posted"]

for col in date_cols:
    df_clean[col] = pd.to_datetime(df_clean[col], errors="coerce")

# Extracting start year and month
df_clean["Start Year"]  = df_clean["Start Date"].dt.year
df_clean["Start Month"] = df_clean["Start Date"].dt.month_name()


print(f"✓ Final shape : {df_clean.shape}")
```

```
✓ Final shape : (5783, 28)
```

In [14]:
```
# Saving Cleaned Data

# Pushing cleaned dataframe back to SQLite as a new table
df_clean.to_sql("trials_clean", conn, if_exists="replace", index=False)

# Saving cleaned CSV to my outputs folder
df_clean.to_csv(OUTPUTS / "covid_trials_cleaned.csv", index=False)

print(f"✓ Final dataset : {df_clean.shape[0]:,} rows, {df_clean.shape[1]} columr
```

```
✓ Final dataset : 5,783 rows, 28 columns
```

Exploratory Data Analysis

Trial Status Distribution

What is the current status of COVID-19 clinical trials globally?

In [15]:
```
# Trial Status Distribution
# SQL query
status_df = pd.read_sql("""
    SELECT Status, COUNT(*) as Count
    FROM trials_clean
    GROUP BY Status
    ORDER BY Count DESC
""", conn)

status_df
```

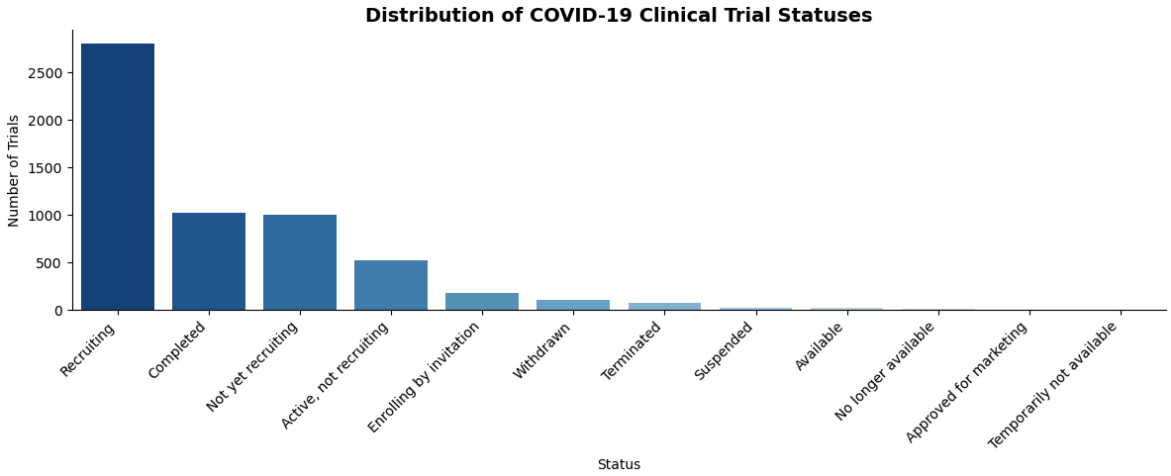
Out[15]:

	Status	Count
0	Recruiting	2805
1	Completed	1025
2	Not yet recruiting	1004
3	Active, not recruiting	526
4	Enrolling by invitation	181
5	Withdrawn	107
6	Terminated	74
7	Suspended	27
8	Available	19
9	No longer available	12
10	Approved for marketing	2
11	Temporarily not available	1

Observations

- `Recruiting` is the dominant status (2,805 trials — 48%), suggesting that this dataset was captured during the peak pandemic period when trials were launching faster than completing
- Only 1,025 trials (18%) are `Completed`, reflecting the early stage nature of COVID-19 research at the time of data collection
- 181 trials were `Withdrawn` or `Terminated`, which may indicate safety concerns, lack of efficacy, or loss of funding
- 1,004 trials are `Not yet recruiting`, meaning the research pipeline was still growing as more trials are being registered than concluded.

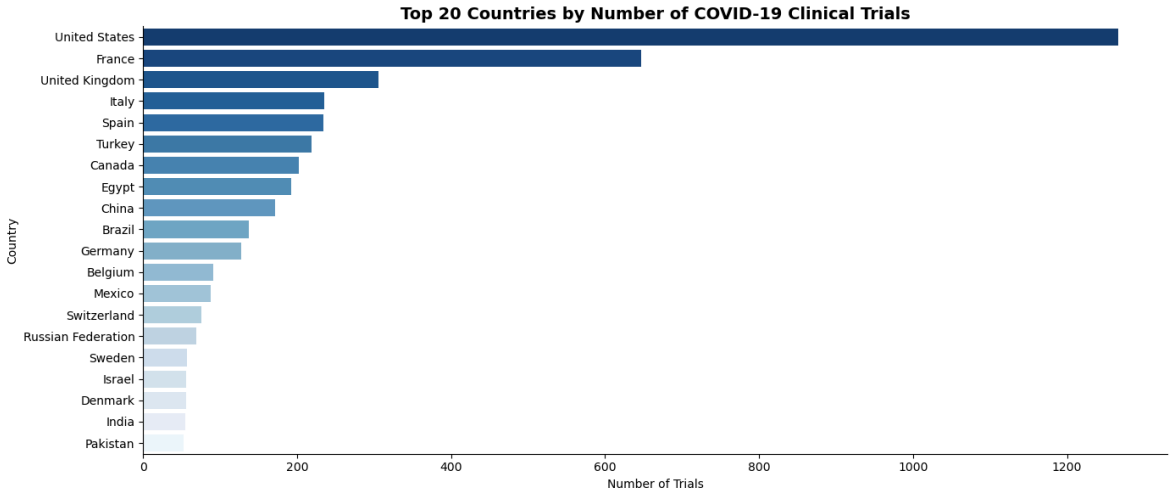
In [16]:
```
# Trial Status Chart
plt.figure(figsize=(12, 5))
sns.barplot(data=status_df, x="Status", y="Count", palette="Blues_r")
plt.title("Distribution of COVID-19 Clinical Trial Statuses",
          fontsize=14, fontweight="bold")
plt.xlabel("Status")
plt.ylabel("Number of Trials")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "trial_status.png", dpi=150)
plt.show()
```



Geographic Distribution

Which countries are leading COVID-19 clinical trial research?

In [17]:
```
# Top 20 Countries
country_df = pd.read_sql("""
    SELECT Country, COUNT(*) as Trial_Count
    FROM trials_clean
    WHERE Country != 'Not Specified'
    GROUP BY Country
    ORDER BY Trial_Count DESC
    LIMIT 20
""", conn)

plt.figure(figsize=(14, 6))
sns.barplot(data=country_df, x="Trial_Count", y="Country", palette="Blues_r")
plt.title("Top 20 Countries by Number of COVID-19 Clinical Trials",
          fontsize=14, fontweight="bold")
plt.xlabel("Number of Trials")
plt.ylabel("Country")
plt.tight_layout()
plt.savefig(OUTPUTS / "top_countries.png", dpi=150)
plt.show()
```



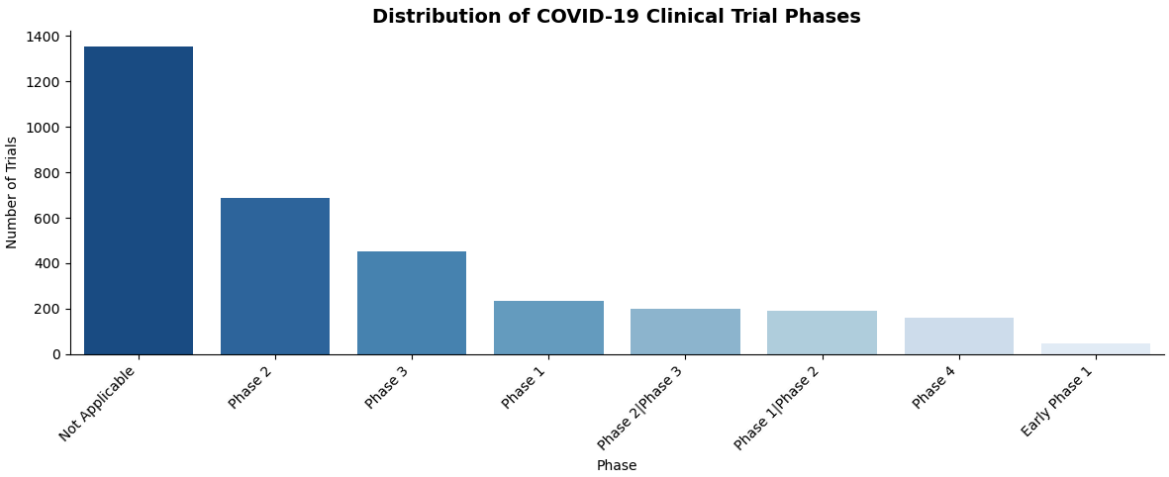
Observations

- The United States leads significantly, which is partly expected given that ClinicalTrials.gov is a US government platform (NIH), and the US has unmatched research funding infrastructure through bodies like NIH, FDA, and private industry
- European countries (France, UK, Italy, Spain) feature prominently, reflecting strong publicly funded research systems
- Egypt's appearance in the top 20 is notable. As Africa's most populous country with established medical institutions, it represents growing research capacity in the MENA region responding to local COVID burden
- The distribution is heavily skewed toward high-income countries, raising questions about equity in global clinical research representation

4.3 Trial Phase Distribution

What phases are most COVID-19 trials operating in?

In [18]:
```
# Trial Phase Distribution
phase_df = pd.read_sql("""
    SELECT Phases, COUNT(*) as Count
    FROM trials_clean
    WHERE Phases != 'Not Specified'
    GROUP BY Phases
    ORDER BY Count DESC
""", conn)

plt.figure(figsize=(12, 5))
sns.barplot(data=phase_df, x="Phases", y="Count", palette="Blues_r")
plt.title("Distribution of COVID-19 Clinical Trial Phases",
          fontsize=14, fontweight="bold")
plt.xlabel("Phase")
plt.ylabel("Number of Trials")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "trial_phases.png", dpi=150)
plt.show()
```



Observations

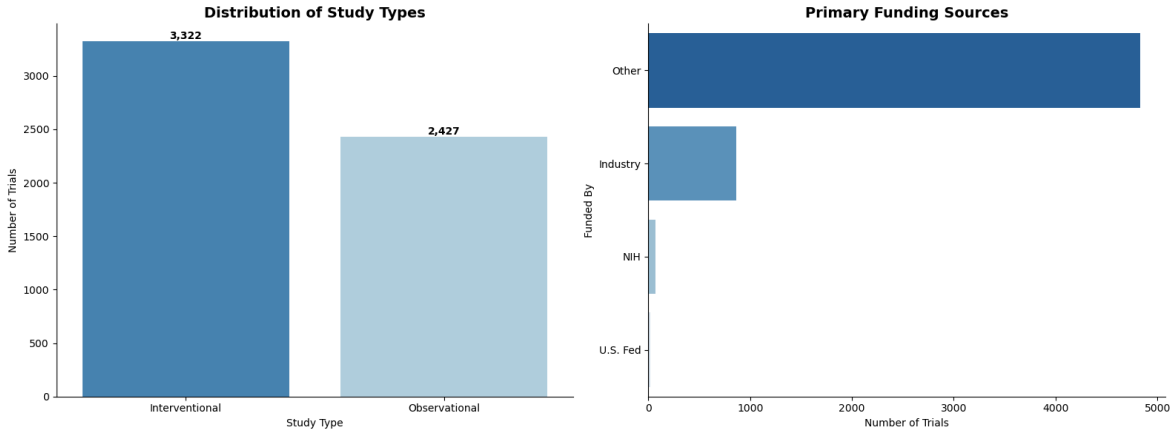
- `Not Applicable` dominates because a large proportion of trials are observational studies that don't follow traditional drug development phase structure
- `Phase 2` and `Phase 3` lead among interventional trials which is reflecting the pandemic urgency to fast-track efficacy and safety testing, sometimes running both phases simultaneously through adaptive trial designs
- Fewer `Phase 1` trials suggest basic safety testing was either compressed or already established for repurposed drugs
- `Phase 4` (post-market surveillance) trials are minimal, This is consistent with the early stage of the dataset as few interventions had reached approval yet

Study Type & Funding Distribution

Are most trials interventional or observational, and who is funding them?

In [19]:
```
# Study Type & Funding Distribution

# Filtering to main study types only
study_type_df = pd.read_sql("""
    SELECT "Study Type", COUNT(*) as Count
    FROM trials_clean
    WHERE "Study Type" IN ('Interventional', 'Observational')
    GROUP BY "Study Type"
    ORDER BY Count DESC
""", conn)

# taking only the first funding source where multiple exist
funded_df = pd.read_sql("""
    SELECT
        TRIM(SUBSTR("Funded Bys", 1,
            CASE WHEN INSTR("Funded Bys", '|') = 0
                 THEN LENGTH("Funded Bys")
                 ELSE INSTR("Funded Bys", '|') - 1
            END)) as Primary_Funder,
        COUNT(*) as Count
    FROM trials_clean
    GROUP BY Primary_Funder
    ORDER BY Count DESC
""", conn)
```

```
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Study Type
sns.barplot(data=study_type_df, x="Study Type", y="Count",
            palette="Blues_r", ax=axes[0])
axes[0].set_title("Distribution of Study Types",
                  fontsize=14, fontweight="bold")
axes[0].set_xlabel("Study Type")
axes[0].set_ylabel("Number of Trials")

# Adding value labels on bars
for p in axes[0].patches:
    axes[0].annotate(f"{int(p.get_height()):,}",
                     (p.get_x() + p.get_width() / 2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")

# Funding
sns.barplot(data=funded_df, x="Count", y="Primary_Funder",
            palette="Blues_r", ax=axes[1])
axes[1].set_title("Primary Funding Sources",
                  fontsize=14, fontweight="bold")
axes[1].set_xlabel("Number of Trials")
axes[1].set_ylabel("Funded By")

plt.tight_layout()
plt.savefig(OUTPUTS / "study_type_funding.png", dpi=150)
plt.show()
```



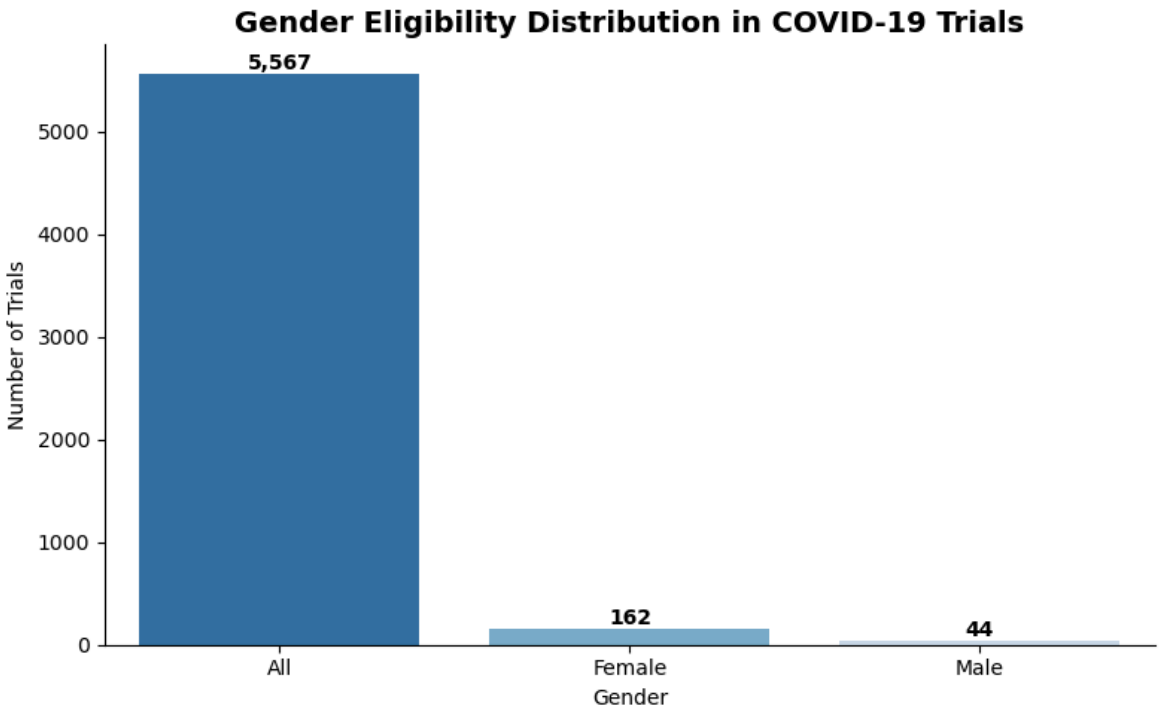
Observations

- Interventional trials (3,322) slightly outnumber Observational (2,427), reflecting thepandemic urgency to actively test treatments and vaccines rather than just document outcomes
- "Other" dominates funding (nearly 5,000 trials) which is representing universities, hospitals, and non-profit research institutions, confirming that public health institutions were the primary drivers of COVID research
- Industry funding is a distant second (~900 trials), suggesting pharmaceutical companies were not the primary force behind COVID trial activity with public health urgency outweighing commercial interest as the main motivation
- NIH and U.S. Federal funding appear separately but are minimal individually, though their combined influence through grants to "Other" institutions is likely much larger

4.5 Gender Distribution

How are COVID-19 clinical trials distributed across gender eligibility?

In [20]:
```
# Gender Distribution
gender_df = pd.read_sql("""
    SELECT Gender, COUNT(*) as Count
    FROM trials_clean
    WHERE Gender != 'Not Specified'
    GROUP BY Gender
    ORDER BY Count DESC
""", conn)

plt.figure(figsize=(8, 5))
sns.barplot(data=gender_df, x="Gender", y="Count", palette="Blues_r")
plt.title("Gender Eligibility Distribution in COVID-19 Trials",
          fontsize=14, fontweight="bold")
plt.xlabel("Gender")
plt.ylabel("Number of Trials")
for p in plt.gca().patches:
    plt.gca().annotate(f"{int(p.get_height()):,}",
                       (p.get_x() + p.get_width() / 2, p.get_height()),
                       ha="center", va="bottom", fontweight="bold")
plt.tight_layout()
plt.savefig(OUTPUTS / "gender_distribution.png", dpi=150)
plt.show()
```



Observations

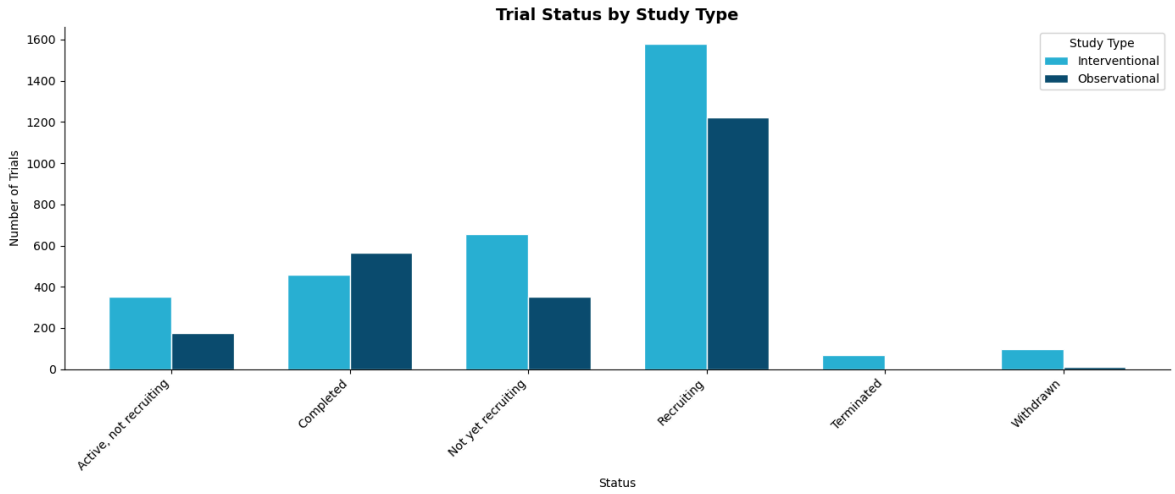
- The vast majority of trials (5,567) are open to all genders, reflecting that COVID-19 affects the entire population without gender-specific biological differences in most cases
- 162 Female-only trials likely represent studies focused on pregnancy outcomes, maternal-foetal transmission, and breastfeeding safety. with these conditions

uniquely relevant to women during the pandemic
- Only 44 Male-only trials exist, suggesting fewer COVID-specific conditions requiring male-only populations
- The dominance of "All" gender eligibility is a positive indicator of inclusive trial design in COVID research

Bivariate Analysis : Study Type vs Trial Status

How does trial status vary across different study types?

In [21]:
```
# Study Type vs Status
bivar_df = pd.read_sql("""
    SELECT "Study Type", Status, COUNT(*) as Count
    FROM trials_clean
    WHERE "Study Type" IN ('Interventional', 'Observational')
    AND Status IN ('Recruiting', 'Completed', 'Not yet recruiting',
                   'Active, not recruiting', 'Terminated', 'Withdrawn')
    GROUP BY "Study Type", Status
    ORDER BY "Study Type", Count DESC
""", conn)

pivot_df = bivar_df.pivot(index="Status", columns="Study Type", values="Count").

pivot_df.plot(kind="bar", figsize=(14, 6),
              color=["#2ab0d4", "#0a4f6e"],
              edgecolor="white", width=0.7)
plt.title("Trial Status by Study Type", fontsize=14, fontweight="bold")
plt.xlabel("Status")
plt.ylabel("Number of Trials")
plt.xticks(rotation=45, ha="right")
plt.legend(title="Study Type")
plt.tight_layout()
plt.savefig(OUTPUTS / "status_by_study_type.png", dpi=150)
plt.show()
```



Observations

- Interventional trials dominate the Recruiting status, reflecting the pandemic push to actively test treatments and vaccines. These take longer to complete due to regulatory requirements and safety monitoring

- Observational studies have a higher Completed rate relative to their total count. They move faster because they involve no active intervention, fewer ethical hurdles, and simply collect existing patient data
- Terminated and Withdrawn trials are almost entirely Interventional, which makes clinical sense because observational studies rarely get terminated as they pose minimal risk to participants
- This pattern confirms that while the world urgently needed interventional results, observational research was delivering conclusions faster during the pandemic

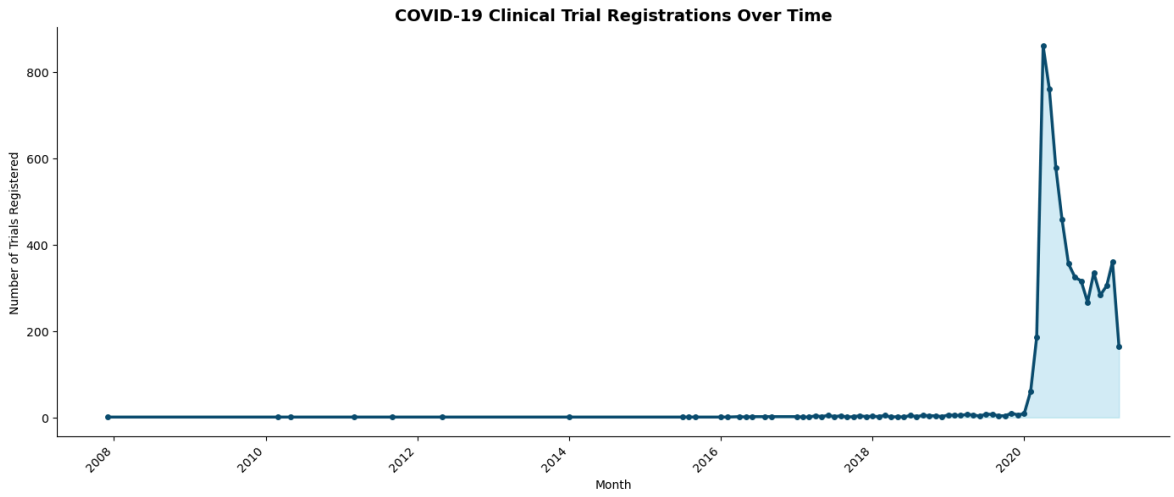
---

Time Series Analysis

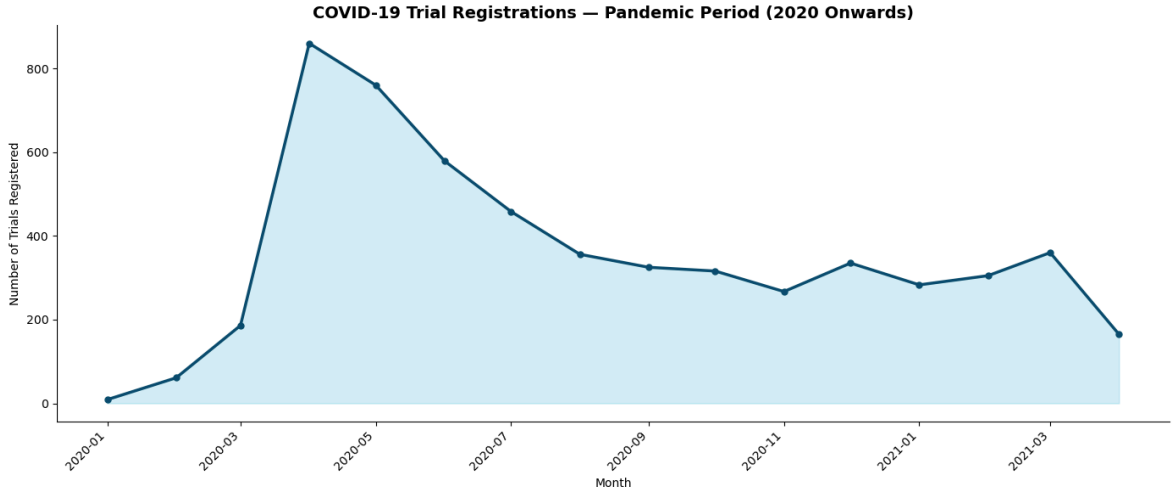
Analysing how COVID-19 clinical trial registrations evolved over time.

Trial Registrations Over Time

In [22]:
```
# Trials Registered Over Time
time_df = pd.read_sql("""
    SELECT
        strftime('%Y-%m', "First Posted") as Month,
        COUNT(*) as Trial_Count
    FROM trials_clean
    WHERE "First Posted" IS NOT NULL
    GROUP BY Month
    ORDER BY Month
""", conn)

time_df["Month"] = pd.to_datetime(time_df["Month"])

plt.figure(figsize=(14, 6))
plt.plot(time_df["Month"], time_df["Trial_Count"],
         color="#0a4f6e", linewidth=2.5, marker="o", markersize=4)
plt.fill_between(time_df["Month"], time_df["Trial_Count"],
                 alpha=0.2, color="#2ab0d4")
plt.title("COVID-19 Clinical Trial Registrations Over Time",
          fontsize=14, fontweight="bold")
plt.xlabel("Month")
plt.ylabel("Number of Trials Registered")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "trials_over_time.png", dpi=150)
plt.show()
```



In [23]:
```
# Zooming in to 2020 Onwards
time_zoom = time_df[time_df["Month"] >= "2020-01-01"]

plt.figure(figsize=(14, 6))
plt.plot(time_zoom["Month"], time_zoom["Trial_Count"],
         color="#0a4f6e", linewidth=2.5, marker="o", markersize=5)
plt.fill_between(time_zoom["Month"], time_zoom["Trial_Count"],
                 alpha=0.2, color="#2ab0d4")
plt.title("COVID-19 Trial Registrations — Pandemic Period (2020 Onwards)",
          fontsize=14, fontweight="bold")
plt.xlabel("Month")
plt.ylabel("Number of Trials Registered")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "trials_pandemic_period.png", dpi=150)
plt.show()
```



Observations

- Trial registrations were virtually zero before 2020, confirming this dataset is specifically COVID-19 related
- A dramatic spike occurred in **April 2020**, directly corresponding to the WHO pandemic declaration (March 11, 2020) and the first global lockdowns. The research community responded within weeks
- Registrations declined sharply after the initial peak but stabilised at ~300 trials/month through late 2020, suggesting sustained but more measured research

activity

- A small secondary peak appears in **March 2021**, coinciding with the Delta variant wave and ongoing vaccine rollout debates
- The declining tail in early 2021 likely reflects the dataset capture date rather than a true drop in research activity

---

Summary & Conclusions

Key findings from the exploratory data analysis of COVID-19 clinical trials.

In [24]:
```
# Summary Statistics
summary = pd.read_sql("""
    SELECT
        COUNT(*) as Total_Trials,
        COUNT(DISTINCT Country) as Countries_Represented,
        ROUND(AVG(Enrollment), 0) as Avg_Enrollment,
        SUM(CASE WHEN Status = 'Completed' THEN 1 ELSE 0 END) as Completed_Trial
        SUM(CASE WHEN Status = 'Recruiting' THEN 1 ELSE 0 END) as Recruiting_Tri
        SUM(CASE WHEN Status IN ('Withdrawn', 'Terminated') THEN 1 ELSE 0 END) a
        SUM(CASE WHEN "Study Type" = 'Interventional' THEN 1 ELSE 0 END) as Inte
        SUM(CASE WHEN "Study Type" = 'Observational' THEN 1 ELSE 0 END) as Obser
    FROM trials_clean
""", conn)

summary.T.rename(columns={0: "Value"})
```

Out[24]:

	Value
Total_Trials	5783.00
Countries_Represented	120.00
Avg_Enrollment	18213.00
Completed_Trials	1025.00
Recruiting_Trials	2805.00
Discontinued_Trials	181.00
Interventional	3322.00
Observational	2427.00

Conclusions

Key Findings

Metric	Value
Total Trials	5,783
Countries Represented	120

Metric	Value
Average Enrollment	18,213
Completed Trials	1,025 (18%)
Recruiting Trials	2,805 (48%)
Discontinued Trials	181 (3%)
Interventional	3,322 (57%)
Observational	2,427 (43%)

Summary of Findings

1. **Global Reach:** COVID-19 trials spanned 120 countries, though research activity was heavily concentrated in high-income countries particularly the United States and France which is raising questions about equity in global clinical research

2. **Research Stage:** With 48% of trials still recruiting and only 18% completed, this dataset captures the pandemic at its most active research phase as more trials were being launched than concluded

3. **Pandemic Response Speed:** The explosive spike in trial registrations in April 2020 within weeks of the WHO pandemic declaration, demonstrates an unprecedented mobilisation of the global research community

4. **Study Design:** Interventional trials slightly outnumber observational ones, but observational studies complete faster due to fewer regulatory and ethical requirements. This is an important consideration for future pandemic preparedness

5. **Funding:** Public health institutions, universities and hospitals drove the majority of COVID research and not pharmaceutical companies which is reflecting that public health urgency was the primary motivation

6. **Gender Inclusivity:** Most trials were open to all genders, with female-only trials focused on pregnancy and maternal outcomes. This is a clinically appropriate design decision

7. **Data Quality:** Two columns (Results First Posted, Study Documents) were too sparse to be useful; missing data in key fields like Phases and Acronym was classified as MAR and handled with missing indicators rather than imputation

Limitations

- The Dataset represents a snapshot in time. Trial statuses will have changed significantly since collection
- Country extraction from Locations field was approximate as multi-site trials were assigned only to their last listed location
- Average enrollment is inflated by extreme outliers with the median (170) being more representative

In [25]:
```
# Closing Connection
conn.close()
```

Exploratory Data Analysis Of Drugs, Side Effects and Medical Conditions

Author	Ibrahim A. Mikail
Internship	Unified Mentor : Healthcare Data Analyst
Project	2 of 5
Tools	Python · Pandas · SQLite · Matplotlib · Seaborn
Dataset	Drugs.com via Kaggle
Date	May 2026

Project Overview

This dataset contains details of various drugs used for conditions like Acne, Cancer, Heart Disease and more, including their side effects, drug classifications, pregnancy safety categories, controlled substance schedules and user effectiveness ratings sourced from Drugs.com.

Objectives

- Explore the distribution of drugs across medical conditions and drug classes
- Analyse pregnancy safety categories and controlled substance schedules
- Investigate user ratings across drug types and conditions
- Identify patterns between drug activity, side effects and medical use

In [1]:
```python
# Project Setup
import os
from pathlib import Path

ROOT    = Path().resolve()
DATA    = ROOT / "data"
OUTPUTS = ROOT / "outputs"
DB_PATH = ROOT / "drugs_side_effects.db"

for folder in [DATA, OUTPUTS]:
    folder.mkdir(parents=True, exist_ok=True)
    print(f"✓ {folder}")

print(f"\nProject root : {ROOT}")
print(f"Database will be created at : {DB_PATH}")
```

```
✓ C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_2_drugs_side_effects\data
✓ C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_2_drugs_side_effects\outputs

Project root : C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_2_drugs_side_effects
Database will be created at : C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_2_drugs_side_effects\drugs_side_effects.db
```

In [2]:
```python
#Importing libraries & Global Settings

import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
from collections import Counter

# Pandas display settings
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", "{:.2f}".format)

# Chart style
plt.rcParams["figure.figsize"] = (12, 5)
plt.rcParams["axes.spines.top"]   = False
plt.rcParams["axes.spines.right"] = False
sns.set_palette("Blues_r")
```

Loading Data & Ingesting to SQLite

Loading the raw CSV into Pandas, pushing to SQLite, and taking a first look at the structure.

In [3]:
```python
# Loading CSV & Pushing to SQLite
df_raw = pd.read_csv(DATA / "drugs_side_effects_drugs_com.csv")

# Connecting to SQLite
conn = sqlite3.connect(DB_PATH)

# Pushing to SQLite
df_raw.to_sql("drugs", conn, if_exists="replace", index=False)

print(f"CSV loaded           : {df_raw.shape[0]:,} rows, {df_raw.shape[1]} colum
print(f"SQLite table created : 'drugs'")
```

```
CSV loaded           : 2,931 rows, 17 columns
SQLite table created : 'drugs'
```

In [4]:
```python
# Initial inspection
df = pd.read_sql("SELECT * FROM drugs LIMIT 5", conn)
df
```

Out[4]:

	drug_name	medical_condition	side_effects	generic_name	drug_classes	brand_
0	doxycycline	Acne	(hives, difficult breathing, swelling in your ...	doxycycline	Miscellaneous antimalarials, Tetracyclines	Ac Adc Ado Adc
1	spironolactone	Acne	hives ; difficulty breathing; swelling of your...	spironolactone	Aldosterone receptor antagonists, Potassium-sp...	Alda Ca
2	minocycline	Acne	skin rash, fever, swollen glands, flu-like sym...	minocycline	Tetracyclines	D M M Sc Ximi
3	Accutane	Acne	problems with your vision or hearing; muscle o...	isotretinoin (oral)	Miscellaneous antineoplastics, Miscellaneous u...	
4	clindamycin	Acne	hives ; difficult breathing; swelling of your ...	clindamycin topical	Topical acne agents, Vaginal anti-infectives	Cle Clindac Clind Cli

In [5]:
```
# Shape & Column Names
print(f"Shape: {df_raw.shape}")
print(f"\nColumns:\n{df_raw.columns.tolist()}")
```

```
Shape: (2931, 17)

Columns:
['drug_name', 'medical_condition', 'side_effects', 'generic_name', 'drug_classe
s', 'brand_names', 'activity', 'rx_otc', 'pregnancy_category', 'csa', 'alcohol',
'related_drugs', 'medical_condition_description', 'rating', 'no_of_reviews', 'dru
g_link', 'medical_condition_url']
```

In [6]:
```
# Data Types & Non-Null Counts
df_raw.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2931 entries, 0 to 2930
Data columns (total 17 columns):
 #   Column                          Non-Null Count  Dtype  
---  ------                          --------------  -----  
 0   drug_name                       2931 non-null   object 
 1   medical_condition               2931 non-null   object 
 2   side_effects                    2807 non-null   object 
 3   generic_name                    2888 non-null   object 
 4   drug_classes                    2849 non-null   object 
 5   brand_names                     1718 non-null   object 
 6   activity                        2931 non-null   object 
 7   rx_otc                          2930 non-null   object 
 8   pregnancy_category              2702 non-null   object 
 9   csa                             2931 non-null   object 
 10  alcohol                         1377 non-null   object 
 11  related_drugs                   1462 non-null   object 
 12  medical_condition_description   2931 non-null   object 
 13  rating                          1586 non-null   float64
 14  no_of_reviews                   1586 non-null   float64
 15  drug_link                       2931 non-null   object 
 16  medical_condition_url           2931 non-null   object 
dtypes: float64(2), object(15)
memory usage: 389.4+ KB
```

Missing Data Analysis

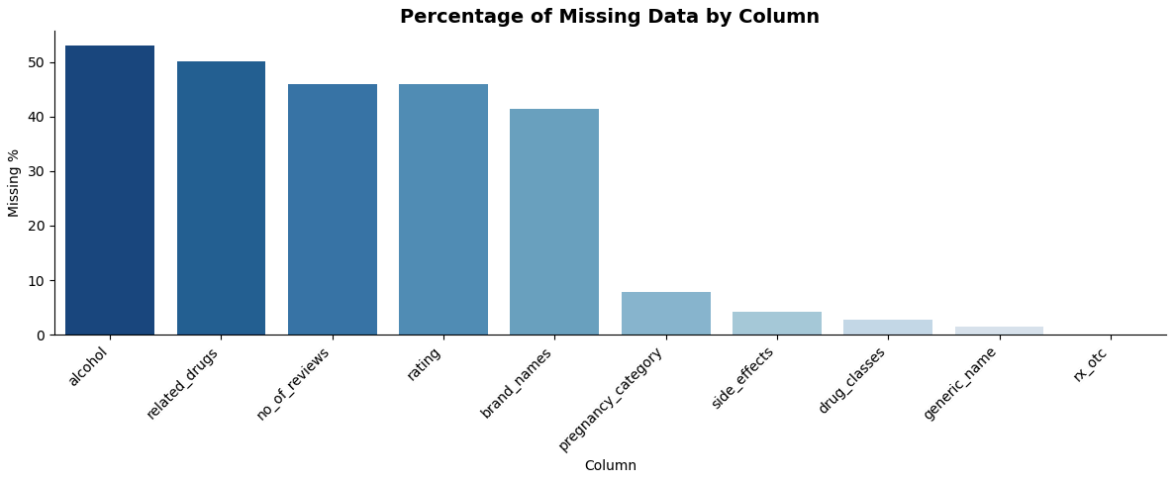
Quantifying missingness before making any decisions about columns.

In [7]:
```python
# Missing Data Percentage
missing = pd.DataFrame({
    "Missing Count" : df_raw.isnull().sum(),
    "Missing %"     : (df_raw.isnull().mean() * 100).round(2)
}).sort_values("Missing %", ascending=False)

missing[missing["Missing %"] > 0]
```

Out[7]:

	Missing Count	Missing %
alcohol	1554	53.02
related_drugs	1469	50.12
no_of_reviews	1345	45.89
rating	1345	45.89
brand_names	1213	41.39
pregnancy_category	229	7.81
side_effects	124	4.23
drug_classes	82	2.80
generic_name	43	1.47
rx_otc	1	0.03

In [8]:
```
# Visualising Missing Data

missing_plot = missing[missing["Missing %"] > 0]
plt.figure(figsize=(12, 5))
sns.barplot(x=missing_plot.index, y=missing_plot["Missing %"], palette="Blues_r"
plt.title("Percentage of Missing Data by Column", fontsize=14, fontweight="bold"
plt.xlabel("Column")
plt.ylabel("Missing %")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "missing_data.png", dpi=150)
plt.show()
```



3.1 Missing Data Observations

- `alcohol` (53%) is **MNAR**: missing values here indicate no known alcohol interaction, not lost data. I Will be engineering into a binary column
- `related_drugs` (50%) is **MNAR**: unique drugs genuinely have no related drugs. I Will be filling with "No Related Drugs"
- `rating` and `no_of_reviews` (46%) is **MNAR**: these are unreviewed drugs on Drugs.com and likely newer or less commonly prescribed. Rows will be retained for

non-rating analysis

- `brand_names` (41%) is **MNAR**: this is most likely some drugs that may only exist as generics with no brand name. Will be filled with "No Brand Name"
- `pregnancy_category` (7.81%) is **MAR**: missingness likely related to drug type or age. I will be filling with "N" (FDA not classified)
- `side_effects` , `drug_classes` , `generic_name` have low missingness (<5%) and will be filled with "Not Specified"
- Key observation: `side_effects` , `drug_classes` and `brand_names` store multiple values as delimited strings and will require special parsing during analysis

---

Data Cleaning

Dropping irrelevant columns, handling missing values, engineering new features and cleaning data types.

Dropping Irrelevant Columns

In [9]:
```
# Dropping Irrelevant Columns
df_clean = df_raw.copy()

cols_to_drop = ["drug_link", "medical_condition_url", "medical_condition_descrip
df_clean.drop(columns=cols_to_drop, inplace=True)

print(f" Remaining columns  : {df_clean.shape[1]}")
print(f" Shape              : {df_clean.shape}")
```

```
Remaining columns  : 14
Shape              : (2931, 14)
```

In [10]:
```
# Handling Missing Values

# Binary encoding of alcohol interaction
df_clean["alcohol_interaction"] = df_clean["alcohol"].apply(
    lambda x: 1 if x == "X" else 0
)
df_clean.drop(columns=["alcohol"], inplace=True)

# Filling categorical columns
fill_map = {
    "related_drugs"      : "No Related Drugs",
    "brand_names"        : "No Brand Name",
    "pregnancy_category" : "N",
    "side_effects"       : "Not Specified",
    "drug_classes"       : "Not Specified",
    "generic_name"       : "Not Specified",
    "rx_otc"             : "Not Specified",
}

for col, fill_value in fill_map.items():
    before = df_clean[col].isnull().sum()
    df_clean[col] = df_clean[col].fillna(fill_value)
    print(f" {col:<25} {before} nulls → filled with '{fill_value}'")
```

```
# Handling rating and no_of_reviews separately
df_clean["no_of_reviews"] = df_clean["no_of_reviews"].fillna(0)
print(f" {'no_of_reviews':<25} filled with 0")
# Keep rating NaN which i will exclude them in rating-specific analysis
print(f" {'rating':<25} kept as NaN and excluded in rating analysis")
```

```
related_drugs             1469 nulls → filled with 'No Related Drugs'
brand_names               1213 nulls → filled with 'No Brand Name'
pregnancy_category        229 nulls → filled with 'N'
side_effects              124 nulls → filled with 'Not Specified'
drug_classes              82 nulls → filled with 'Not Specified'
generic_name              43 nulls → filled with 'Not Specified'
rx_otc                    1 nulls → filled with 'Not Specified'
no_of_reviews             filled with 0
rating                    kept as NaN and excluded in rating analysis
```

In [11]:
```
# Cleaning Data Types

# Activity is stored as "87%",  converting to float
df_clean["activity"] = df_clean["activity"].str.replace("%", "").astype(float)

# Converting no_of_reviews to integer
df_clean["no_of_reviews"] = df_clean["no_of_reviews"].astype(int)

print(f"\nData types after cleaning:")
print(df_clean.dtypes)
```

```
Data types after cleaning:
drug_name               object
medical_condition       object
side_effects            object
generic_name            object
drug_classes            object
brand_names             object
activity               float64
rx_otc                  object
pregnancy_category      object
csa                     object
related_drugs           object
rating                 float64
no_of_reviews            int64
alcohol_interaction      int64
dtype: object
```

In [12]:
```
# Saving Cleaned Data
df_clean.to_sql("drugs_clean", conn, if_exists="replace", index=False)
df_clean.to_csv(OUTPUTS / "drugs_side_effects_cleaned.csv", index=False)


print(f"Final shape : {df_clean.shape[0]:,} rows, {df_clean.shape[1]} columns")
```

```
Final shape : 2,931 rows, 14 columns
```

Cleaning Observations

- Dropped 3 irrelevant columns: URLs and condition description as they are web scraping artifacts
- `alcohol` engineered into binary `alcohol_interaction` (1 = interacts, 0 = no interaction) which is more analytically useful than the raw "X" flag

- Categorical missing values filled with meaningful indicators rather than arbitrary values
- `rating` kept as NaN intentionally and will be excluded in rating-specific analysis to avoid distorting results with imputed values
- `activity` successfully converted from percentage string to float for numerical analysis
- Final dataset: 2,931 rows, 14 columns and ready for analysis

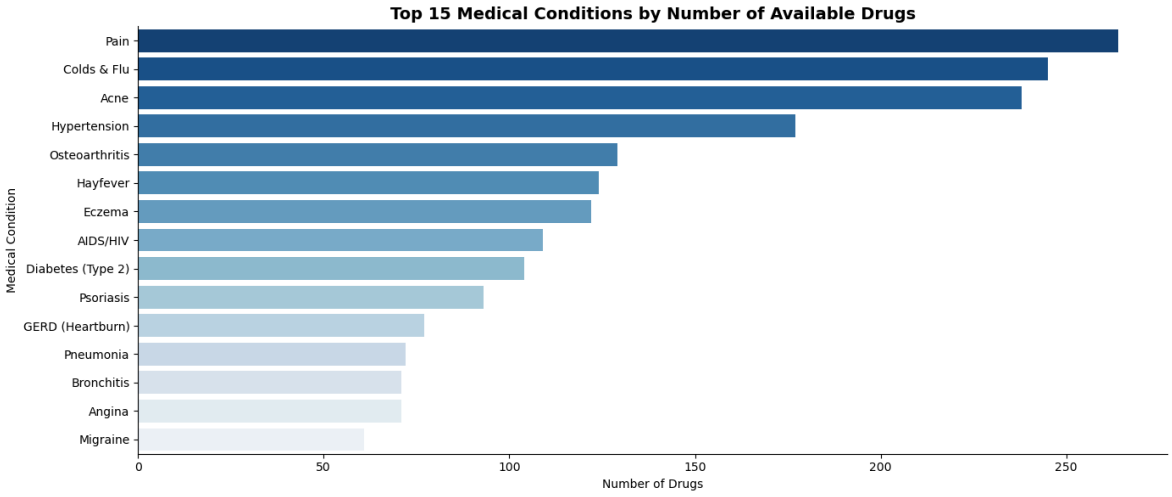
---

Exploratory Data Analysis

Top Medical Conditions

Which medical conditions have the most drugs available?

In [13]:
```
# Top Medical Conditions
conditions_df = pd.read_sql("""
    SELECT medical_condition, COUNT(*) as Drug_Count
    FROM drugs_clean
    GROUP BY medical_condition
    ORDER BY Drug_Count DESC
    LIMIT 15
""", conn)

plt.figure(figsize=(14, 6))
sns.barplot(data=conditions_df, x="Drug_Count", y="medical_condition",
            palette="Blues_r")
plt.title("Top 15 Medical Conditions by Number of Available Drugs",
          fontsize=14, fontweight="bold")
plt.xlabel("Number of Drugs")
plt.ylabel("Medical Condition")
plt.tight_layout()
plt.savefig(OUTPUTS / "top_conditions.png", dpi=150)
plt.show()
```



Observations

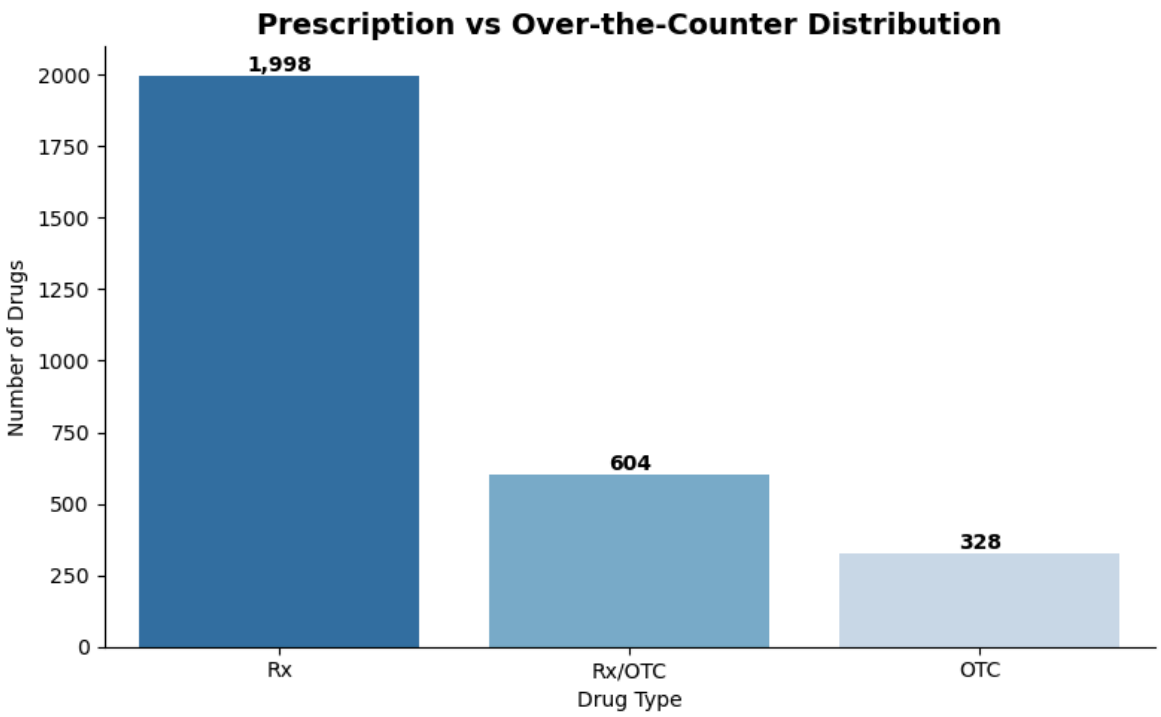
- Pain, Cold & Flu, and Acne top the list of conditions with the most available drugs

- This reflects market dynamics as highly prevalent conditions that affect large populations drive more drug development and more OTC competition
- These conditions are largely self-managed, meaning pharmaceutical companies have strong commercial incentive to develop multiple competing products
- Serious conditions like Pneumonia and Angina appear lower on the list despite their severity, reflecting the higher complexity and cost of developing drugs for those conditions

Prescription vs Over-the-Counter Distribution

How are drugs distributed across prescription requirements?

In [14]:
```
# Rx vs OTC Distribution
rx_df = pd.read_sql("""
    SELECT rx_otc, COUNT(*) as Count
    FROM drugs_clean
    WHERE rx_otc != 'Not Specified'
    GROUP BY rx_otc
    ORDER BY Count DESC
""", conn)

plt.figure(figsize=(8, 5))
sns.barplot(data=rx_df, x="rx_otc", y="Count", palette="Blues_r")
plt.title("Prescription vs Over-the-Counter Distribution",
          fontsize=14, fontweight="bold")
plt.xlabel("Drug Type")
plt.ylabel("Number of Drugs")
for p in plt.gca().patches:
    plt.gca().annotate(f"{int(p.get_height()):,}",
                       (p.get_x() + p.get_width() / 2, p.get_height()),
                       ha="center", va="bottom", fontweight="bold")
plt.tight_layout()
plt.savefig(OUTPUTS / "rx_otc.png", dpi=150)
plt.show()
```



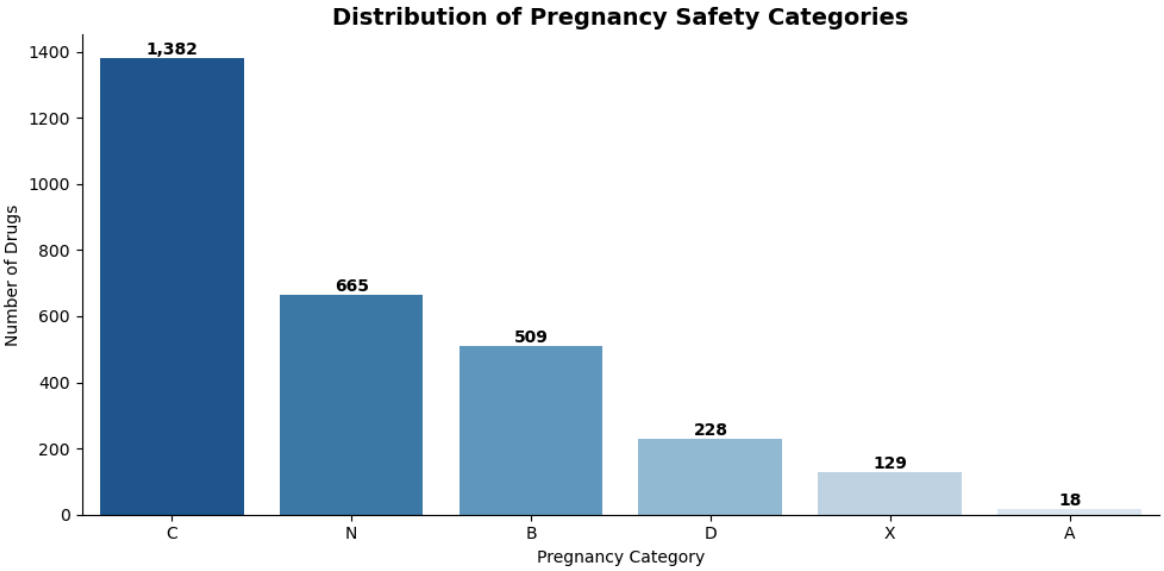
Observations

- Prescription (Rx) drugs dominate at 1,998 (68%) reflecting heavy regulatory control in the pharmaceutical industry
- Only 328 drugs (11%) are pure OTC, confirming that most medications require professional oversight for safe use.
- 604 drugs (21%) are Rx/OTC meaning they exist in both forms depending on dosage, reflecting the Rx-to-OTC switch process where proven safe drugs are gradually made more accessible
- The dominance of Rx drugs highlights the importance of combating self-medication. Drugs used without proper diagnosis risk masking serious conditions, incorrect dosing, and dangerous drug interactions

Pregnancy Safety Category Distribution

How safe are the available drugs for use during pregnancy?

In [15]:
```
# ── Pregnancy Category Distribution ──────────────────────────────────
preg_df = pd.read_sql("""
    SELECT pregnancy_category, COUNT(*) as Count
    FROM drugs_clean
    GROUP BY pregnancy_category
    ORDER BY Count DESC
""", conn)

plt.figure(figsize=(10, 5))
sns.barplot(data=preg_df, x="pregnancy_category", y="Count", palette="Blues_r")
plt.title("Distribution of Pregnancy Safety Categories",
          fontsize=14, fontweight="bold")
plt.xlabel("Pregnancy Category")
plt.ylabel("Number of Drugs")
for p in plt.gca().patches:
    plt.gca().annotate(f"{int(p.get_height()):,}",
                       (p.get_x() + p.get_width() / 2, p.get_height()),
                       ha="center", va="bottom", fontweight="bold")
plt.tight_layout()
plt.savefig(OUTPUTS / "pregnancy_categories.png", dpi=150)
plt.show()
```



Observations

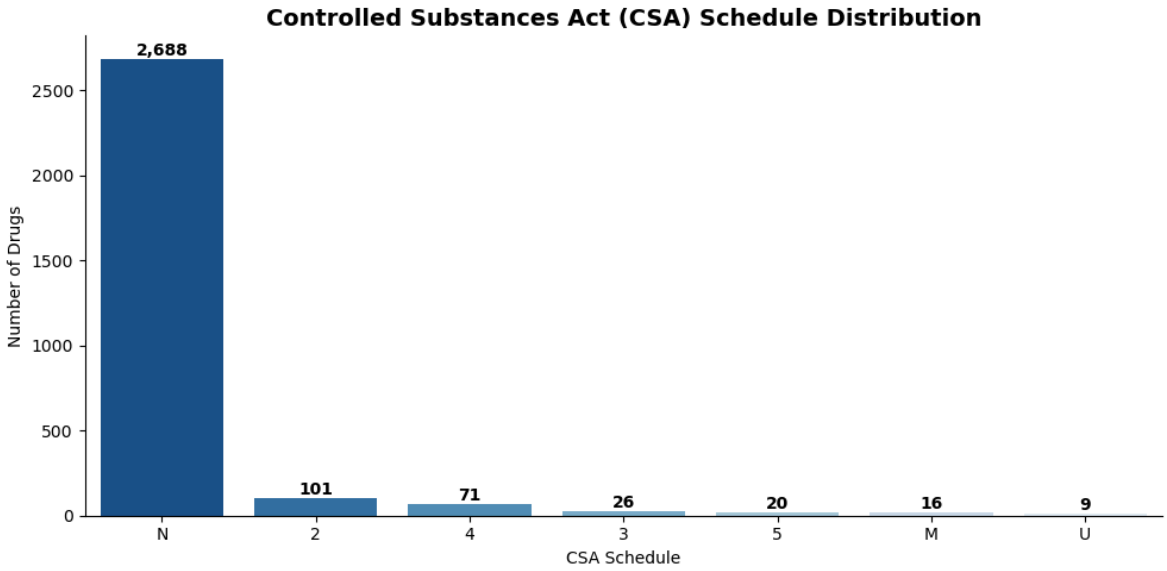
- Category C dominates at 1,382 drugs (47%) meaning the majority of available drugs have shown adverse fetal effects in animal studies, requiring careful risk-benefit assessment before use in pregnancy
- Only 18 drugs (0.6%) fall under Category which means truly safe medications for pregnant women are extremely rare, highlighting the vulnerability of this population
- 129 drugs are Category X and are absolutely contraindicated in pregnancy, including well-known drugs like Accutane (isotretinoin)
- 665 drugs are Category N FDA unclassified, meaning pregnant patients face additional uncertainty with these medications
- Key clinical implication: pregnant patients must seek professional consultation before taking virtually any medication as self-medication during pregnancy carries serious fetal risk

Controlled Substances Act (CSA) Schedule Distribution

How are drugs distributed across controlled substance schedules?

In [16]:
```
# ── CSA Schedule Distribution ──────────────────────────────────
csa_df = pd.read_sql("""
    SELECT csa, COUNT(*) as Count
    FROM drugs_clean
    GROUP BY csa
    ORDER BY Count DESC
""", conn)

plt.figure(figsize=(10, 5))
sns.barplot(data=csa_df, x="csa", y="Count", palette="Blues_r")
plt.title("Controlled Substances Act (CSA) Schedule Distribution",
          fontsize=14, fontweight="bold")
plt.xlabel("CSA Schedule")
plt.ylabel("Number of Drugs")
for p in plt.gca().patches:
    plt.gca().annotate(f"{int(p.get_height()):,}",
                       (p.get_x() + p.get_width() / 2, p.get_height()),
```

```
                        ha="center", va="bottom", fontweight="bold")
plt.tight_layout()
plt.savefig(OUTPUTS / "csa_schedule.png", dpi=150)
plt.show()
```



Observations

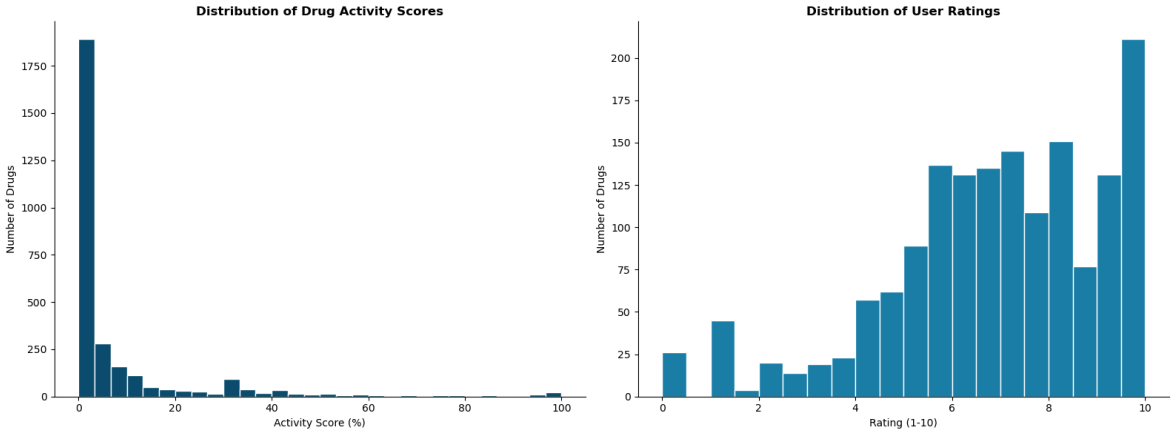
- The vast majority of drugs (2,688 — 92%) fall under "N" and not subject to the Controlled Substances Act, confirming most medications in this dataset are standard non-controlled drugs
- Schedule 2 leads among controlled substances (101 drugs). These include opioids (oxycodone, morphine, fentanyl) and stimulants (Adderall), reflecting high medical utility but also high abuse potential
- Schedule 1 drugs are absent entirely as expected, since Schedule 1 substances have no accepted medical use and would not appear in a clinical drug database
- The small number of Schedule 4 (71) and Schedule 3 (26) drugs reflects the relatively limited set of medications with moderate abuse potential in clinical use
- The presence of controlled substances in this dataset underscores the importance of prescription regulation as these are drugs where misuse carries serious public health consequences

Drug Activity & User Ratings Analysis

Exploring the distribution of drug activity scores and user effectiveness ratings.

In [17]:
```
# ── Activity & Rating Distributions ──────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Activity distribution
axes[0].hist(df_clean["activity"], bins=30, color="#0a4f6e", edgecolor="white")
axes[0].set_title("Distribution of Drug Activity Scores", fontweight="bold")
axes[0].set_xlabel("Activity Score (%)")
axes[0].set_ylabel("Number of Drugs")

# Rating distribution
rating_data = df_clean["rating"].dropna()
```

```
axes[1].hist(rating_data, bins=20, color="#1a7ea8", edgecolor="white")
axes[1].set_title("Distribution of User Ratings", fontweight="bold")
axes[1].set_xlabel("Rating (1-10)")
axes[1].set_ylabel("Number of Drugs")

plt.tight_layout()
plt.savefig(OUTPUTS / "activity_rating_dist.png", dpi=150)
plt.show()

print(f"Activity — Mean: {df_clean['activity'].mean():.1f}%  "
      f"Median: {df_clean['activity'].median():.1f}%")
print(f"Rating   — Mean: {rating_data.mean():.2f}  "
      f"Median: {rating_data.median():.2f}")
```




```
Activity — Mean: 8.5%  Median: 2.0%
Rating   — Mean: 6.81  Median: 7.00
```

Observations

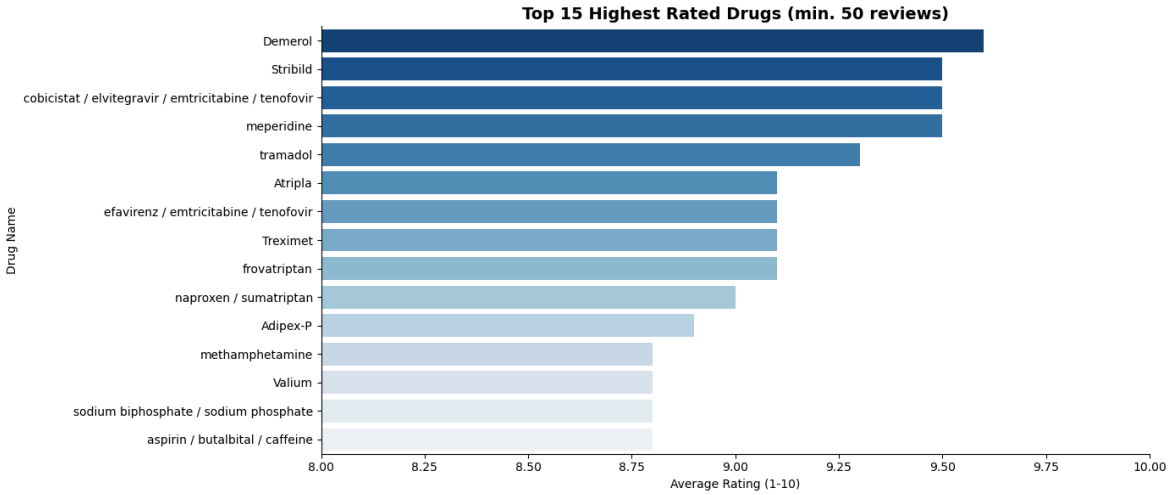
- Activity scores are heavily right skewed. The mean (8.5%) is far above median (2%), indicating most drugs have very low site activity while a small number of highly popular drugs drive the average up
- User ratings skew positively towards 10, with a mean of 6.81 and median of 7.00 suggesting users generally find their medications effective
- However, the rating distribution reflects **survivorship bias** as users who had neutral experiences rarely review, meaning ratings overrepresent strong opinions (very satisfied or very dissatisfied patients)
- The spike at rating 10 is particularly notable as patients who find a drug life-changing are highly motivated to share that experience
- Only 1,586 of 2,931 drugs have ratings. Therefore, analysis of ratings should be interpreted with caution given this significant subset

Top Rated Drugs per Medical Condition

Which drugs have the highest user ratings and for which conditions?

In [18]:
```
# ── Top Rated Drugs ──────────────────────────────────────────────
top_rated = pd.read_sql("""
    SELECT drug_name, medical_condition, rating, no_of_reviews
    FROM drugs_clean
    WHERE rating IS NOT NULL
```

```
    AND no_of_reviews >= 50
    ORDER BY rating DESC
    LIMIT 15
""", conn)

plt.figure(figsize=(14, 6))
sns.barplot(data=top_rated, x="rating", y="drug_name",
            palette="Blues_r", orient="h")
plt.title("Top 15 Highest Rated Drugs (min. 50 reviews)",
          fontsize=14, fontweight="bold")
plt.xlabel("Average Rating (1-10)")
plt.ylabel("Drug Name")
plt.xlim(8, 10)
plt.tight_layout()
plt.savefig(OUTPUTS / "top_rated_drugs.png", dpi=150)
plt.show()
```



Observations

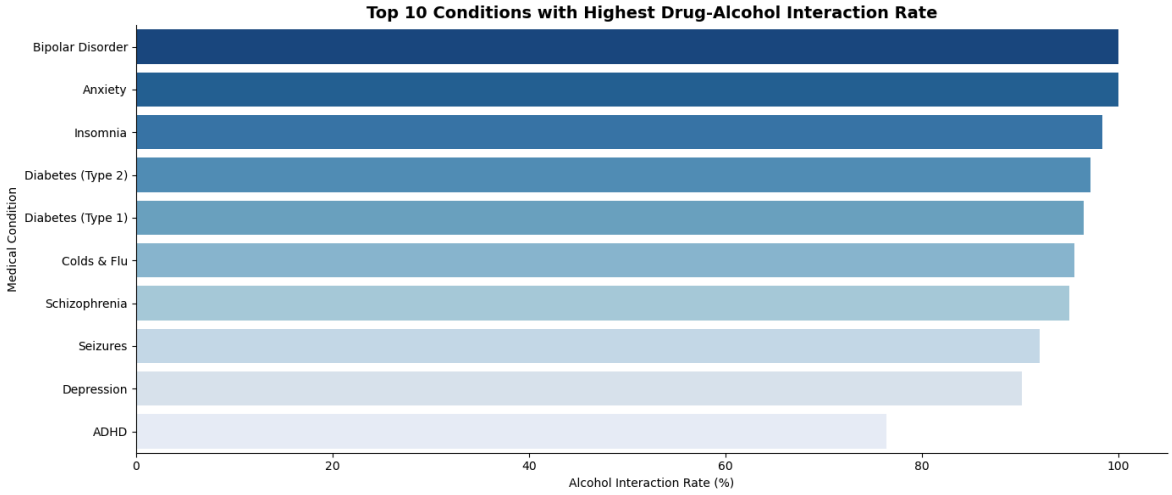
- Demerol (meperidine) and tramadol which are both opioid pain medications top the ratings chart, reflecting their high clinical effectiveness but also their psychological satisfaction effect which contributes to dependency and addiction risk
- The presence of highly rated controlled substances (Valium, tramadol) in this list is a public health concern as high patient satisfaction ratings do not account for abuse potential and long-term dependency
- HIV antiretroviral combinations (Stribild, cobicistat/elvitegravir) rate highly, reflecting genuine life-improving impact for patients managing a chronic condition
- The minimum of 50 reviews filter ensures statistical reliability ratings based on small samples would be misleading
- This analysis illustrates why patient satisfaction alone cannot drive drug policy. Effectiveness must be balanced against safety, dependency risk and long term outcomes

Alcohol Interaction Analysis

What proportion of drugs have known alcohol interactions and how does this vary by condition?

In [19]:
```
# ──Alcohol Interaction ──────────────────────────────────────────────
alcohol_df = pd.read_sql("""
    SELECT
        SUM(alcohol_interaction) as Interacts,
        COUNT(*) - SUM(alcohol_interaction) as No_Interaction,
        ROUND(SUM(alcohol_interaction) * 100.0 / COUNT(*), 2) as Pct_Interacts
    FROM drugs_clean
""", conn)

print(alcohol_df)

# top 10 conditions with highest alcohol interaction rate
alcohol_condition = pd.read_sql("""
    SELECT medical_condition,
           COUNT(*) as Total,
           SUM(alcohol_interaction) as Interacts,
           ROUND(SUM(alcohol_interaction) * 100.0 / COUNT(*), 2) as Interaction_
    FROM drugs_clean
    GROUP BY medical_condition
    HAVING Total >= 10
    ORDER BY Interaction_Rate DESC
    LIMIT 10
""", conn)

plt.figure(figsize=(14, 6))
sns.barplot(data=alcohol_condition, x="Interaction_Rate",
            y="medical_condition", palette="Blues_r")
plt.title("Top 10 Conditions with Highest Drug-Alcohol Interaction Rate",
          fontsize=14, fontweight="bold")
plt.xlabel("Alcohol Interaction Rate (%)")
plt.ylabel("Medical Condition")
plt.tight_layout()
plt.savefig(OUTPUTS / "alcohol_interaction.png", dpi=150)
plt.show()
```

```
   Interacts  No_Interaction  Pct_Interacts
0       1377            1554          46.98
```



Observations

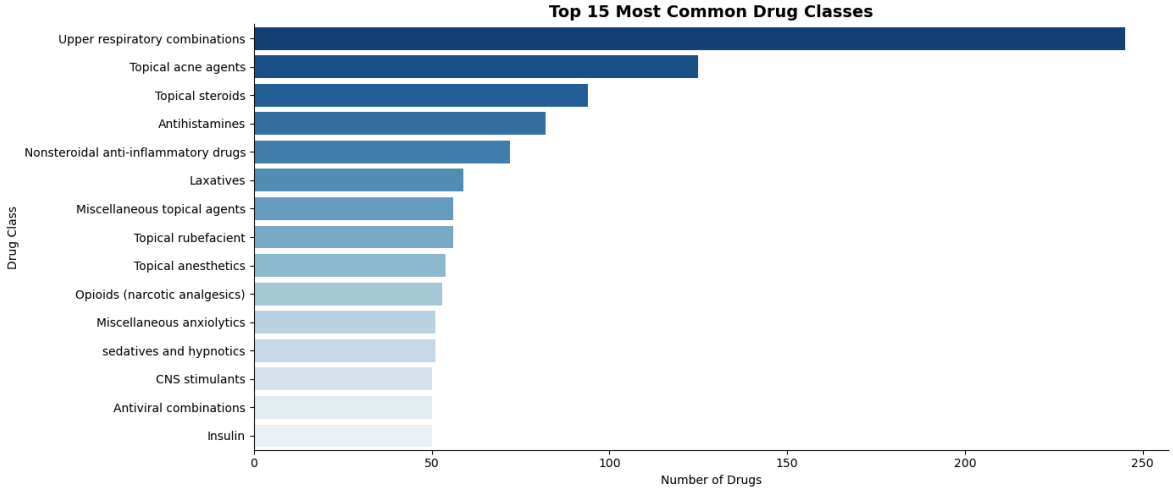
- 47% of all drugs (1,377) have known alcohol interactions. This is a significant public health finding given how commonly alcohol is consumed alongside medications

- Bipolar Disorder and Anxiety show 100% alcohol interaction rates. Every single drug for these conditions interacts with alcohol
- Psychiatric and neurological conditions dominate the top 10 because their medications act on the Central Nervous System (CNS) which is the same pathway alcohol affects. Combining these drugs with alcohol amplifies sedation, impairs cognitive function and can be fatal
- Both types of Diabetes appear due to alcohol's direct interference with blood sugar regulation, compounding the risk for diabetic patients
- This analysis has direct clinical implications as healthcare providers must consistently counsel patients on alcohol avoidance when prescribing CNS-acting medications

Most Common Drug Classes

Parsing multi-value drug class strings to identify the most represented drug classes.

In [20]:
```
# ──Drug Classes Analysis ──────────────────────────────────────────
# Split delimited drug classes and count individual occurrences
all_classes = []
for entry in df_clean["drug_classes"]:
    if entry != "Not Specified":
        classes = [c.strip() for c in entry.split(",")]
        all_classes.extend(classes)

class_counts = pd.DataFrame(
    Counter(all_classes).most_common(15),
    columns=["Drug_Class", "Count"]
)

plt.figure(figsize=(14, 6))
sns.barplot(data=class_counts, x="Count", y="Drug_Class", palette="Blues_r")
plt.title("Top 15 Most Common Drug Classes",
          fontsize=14, fontweight="bold")
plt.xlabel("Number of Drugs")
plt.ylabel("Drug Class")
plt.tight_layout()
plt.savefig(OUTPUTS / "drug_classes.png", dpi=150)
plt.show()
```



Observations

- Upper Respiratory Combinations dominate at ~245 drugs which is directly consistent with Cold & Flu being one of the top conditions by drug count. These are multi-ingredient products combining decongestants, antihistamines and analgesics
- Topical Acne Agents (130) and Topical Steroids (95) reflect the high volume of skin condition treatments in this dataset
- Antihistamines and NSAIDs (Non-Steroidal Anti-Inflammatory Drugs) feature prominently. Both are widely used across multiple conditions including allergies, pain and inflammation
- Opioids (narcotic analgesics) appearing in the top 15 is consistent with Pain being the top condition and opioids dominating user ratings. Confirming their widespread clinical use despite high abuse potential
- The presence of Sedatives, Anxiolytics and CNS Stimulants reflects the significant burden of psychiatric conditions in this dataset

---

Summary Statistics & Conclusions

In [21]:
```
# ── Summary Statistics ──────────────────────────────────────────
summary = pd.read_sql("""
    SELECT
        COUNT(*) as Total_Drugs,
        COUNT(DISTINCT medical_condition) as Unique_Conditions,
        ROUND(AVG(rating), 2) as Avg_Rating,
        ROUND(AVG(activity), 2) as Avg_Activity_Score,
        SUM(CASE WHEN rx_otc = 'Rx' THEN 1 ELSE 0 END) as Prescription_Only,
        SUM(CASE WHEN rx_otc = 'OTC' THEN 1 ELSE 0 END) as OTC_Only,
        SUM(CASE WHEN pregnancy_category = 'X' THEN 1 ELSE 0 END) as Contraindic
        SUM(CASE WHEN pregnancy_category = 'A' THEN 1 ELSE 0 END) as Safe_Pregna
        SUM(alcohol_interaction) as Alcohol_Interactions,
        SUM(CASE WHEN csa NOT IN ('N', 'U') THEN 1 ELSE 0 END) as Controlled_Sub
    FROM drugs_clean
""", conn)

summary.T.rename(columns={0: "Value"})
```

Out[21]:

	Value
Total_Drugs	2931.00
Unique_Conditions	47.00
Avg_Rating	6.81
Avg_Activity_Score	8.45
Prescription_Only	1998.00
OTC_Only	328.00
Contraindicated_Pregnancy	129.00
Safe_Pregnancy	18.00
Alcohol_Interactions	1377.00
Controlled_Substances	234.00

Conclusions

Key Metrics

Metric	Value
Total Drugs	2,931
Unique Medical Conditions	47
Average User Rating	6.81 / 10
Average Activity Score	8.45%
Prescription Only	1,998 (68%)
OTC Only	328 (11%)
Contraindicated in Pregnancy	129
Safe in Pregnancy (Category A)	18
Alcohol Interactions	1,377 (47%)
Controlled Substances	234

Summary of Findings

1. **Market Concentration:** Pain, Cold & Flu and Acne have the highest number of available drugs. High prevalence conditions drive pharmaceutical market competition

2. **Heavy Regulation:** 68% of drugs are prescription-only, confirming that most medications require professional oversight. Self-medication poses serious risks including misdiagnosis, incorrect dosing and dangerous interactions

3. **Pregnancy Risk:** Only 18 drugs are classified as completely safe in pregnancy (Category A) while 129 are absolutely contraindicated (Category X). Pregnant patients face significant medication risk and must always seek professional guidance

4. **Alcohol Interactions:** 47% of drugs interact with alcohol. Psychiatric and neurological medications show the highest interaction rates due to shared CNS pathways. This has direct implications for patient counselling

5. **Opioid Paradox:** Opioids rank among the highest rated drugs reflecting genuine pain relief effectiveness, but their high satisfaction ratings also reflect psychological dependency mechanisms — a critical public health concern

6. **Drug Classes:** Upper respiratory combinations dominate drug classes, consistent with Cold & Flu being a top condition. CNS-acting drug classes (anxiolytics, sedatives, stimulants) reflect significant psychiatric disease burden

Limitations

- Rating data covers only 54% of drugs — findings may not represent the full dataset
- Activity scores reflect Drugs.com site traffic, not clinical effectiveness
- Alcohol interaction data is binary. Severity of interactions varies significantly and is not captured here
- Dataset represents a snapshot and may not reflect current drug approvals or withdrawals

In [22]:
```
# ── Close Connection ──────────────────────────────────────────────
conn.close()
```

Life Expectancy Analysis EDA & Machine Learning

Author	Ibrahim A. Mikail
Internship	Unified Mentor — Healthcare Data Analyst
Project	3 of 5
Tools	Python · Pandas · SQLite · Matplotlib · Seaborn · Scikit-learn
Dataset	WHO & United Nations via Kaggle
Date	May 2026

Project Overview

This dataset contains health, economic and social indicators for 193 countries spanning 2000-2015, collected from the WHO Global Health Observatory and the United Nations. The project performs exploratory data analysis and builds machine learning regression models to identify the key factors that predict life expectancy and quantify their impact.

Key Questions

1. Which factors most significantly affect life expectancy?
2. Should countries with low life expectancy increase healthcare expenditure?
3. How do infant and adult mortality rates affect life expectancy?
4. What is the impact of schooling, immunization and economic factors on lifespan?
5. Can we build a reliable model to predict life expectancy from these factors?

In [1]:
```
# ── Project Setup ──────────────────────────────────────────────
import os
from pathlib import Path

ROOT    = Path().resolve()
DATA    = ROOT / "data"
OUTPUTS = ROOT / "outputs"
DB_PATH = ROOT / "life_expectancy.db"

for folder in [DATA, OUTPUTS]:
    folder.mkdir(parents=True, exist_ok=True)
    print(f" {folder}")

print(f"\nProject root : {ROOT}")
print(f"Database will be created at : {DB_PATH}")
```

```
 C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_3_
life_expectancy\data
 C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_3_
life_expectancy\outputs

Project root : C:\Users\DELL\Documents\Data Science Projects\unified_mentor_proje
cts\project_3_life_expectancy
Database will be created at : C:\Users\DELL\Documents\Data Science Projects\unifi
ed_mentor_projects\project_3_life_expectancy\life_expectancy.db
```

In [2]:
```python
# ── Imports & Global Settings ──────────────────────────────────────
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error
from sklearn.preprocessing import LabelEncoder

# Display settings
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", "{:.2f}".format)

# Chart style
plt.rcParams["figure.figsize"] = (12, 5)
plt.rcParams["axes.spines.top"]   = False
plt.rcParams["axes.spines.right"] = False
sns.set_palette("Blues_r")

import warnings
warnings.filterwarnings("ignore")
```

Loading Data & Ingesting to SQLite

In [3]:
```python
# ── Loading CSV & Pushing to SQLite ────────────────────────────────
df_raw = pd.read_csv(DATA / "Life Expectancy Data.csv")

# Connecting to SQLite
conn = sqlite3.connect(DB_PATH)

# Pushing to SQLite
df_raw.to_sql("life_expectancy", conn, if_exists="replace", index=False)

print(f" CSV loaded           : {df_raw.shape[0]:,} rows, {df_raw.shape[1]} colu
print(f" SQLite table created : 'life_expectancy'")
print(f" Database saved at    : {DB_PATH}")
```

```
 CSV loaded           : 2,938 rows, 22 columns
 SQLite table created : 'life_expectancy'
 Database saved at    : C:\Users\DELL\Documents\Data Science Projects\unified_men
tor_projects\project_3_life_expectancy\life_expectancy.db
```

In [4]:
```
# ── Initial inspection ──────────────────────────────────────────────
df = pd.read_sql("SELECT * FROM life_expectancy LIMIT 5", conn)
df
```

Out[4]:

	Country	Year	Status	Life expectancy	Adult Mortality	infant deaths	Alcohol	percentage expenditure
0	Afghanistan	2015	Developing	65.00	263.00	62	0.01	71.28
1	Afghanistan	2014	Developing	59.90	271.00	64	0.01	73.52
2	Afghanistan	2013	Developing	59.90	268.00	66	0.01	73.22
3	Afghanistan	2012	Developing	59.50	272.00	69	0.01	78.18
4	Afghanistan	2011	Developing	59.20	275.00	71	0.01	7.10

In [5]:
```
# ── Shape, Columns & Data Types ─────────────────────────────────────
print(f"Shape: {df_raw.shape}")
print(f"\nColumns:\n{df_raw.columns.tolist()}")
print(f"\nData Types:\n{df_raw.dtypes}")
```

```
Shape: (2938, 22)

Columns:
['Country', 'Year', 'Status', 'Life expectancy ', 'Adult Mortality', 'infant deat
hs', 'Alcohol', 'percentage expenditure', 'Hepatitis B', 'Measles ', ' BMI ', 'un
der-five deaths ', 'Polio', 'Total expenditure', 'Diphtheria ', ' HIV/AIDS', 'GD
P', 'Population', ' thinness  1-19 years', ' thinness 5-9 years', 'Income composi
tion of resources', 'Schooling']

Data Types:
Country                           object
Year                               int64
Status                            object
Life expectancy                  float64
Adult Mortality                  float64
infant deaths                      int64
Alcohol                          float64
percentage expenditure           float64
Hepatitis B                      float64
Measles                            int64
 BMI                             float64
under-five deaths                  int64
Polio                            float64
Total expenditure                float64
Diphtheria                       float64
 HIV/AIDS                        float64
GDP                              float64
Population                       float64
 thinness  1-19 years            float64
 thinness 5-9 years              float64
Income composition of resources  float64
Schooling                        float64
dtype: object
```

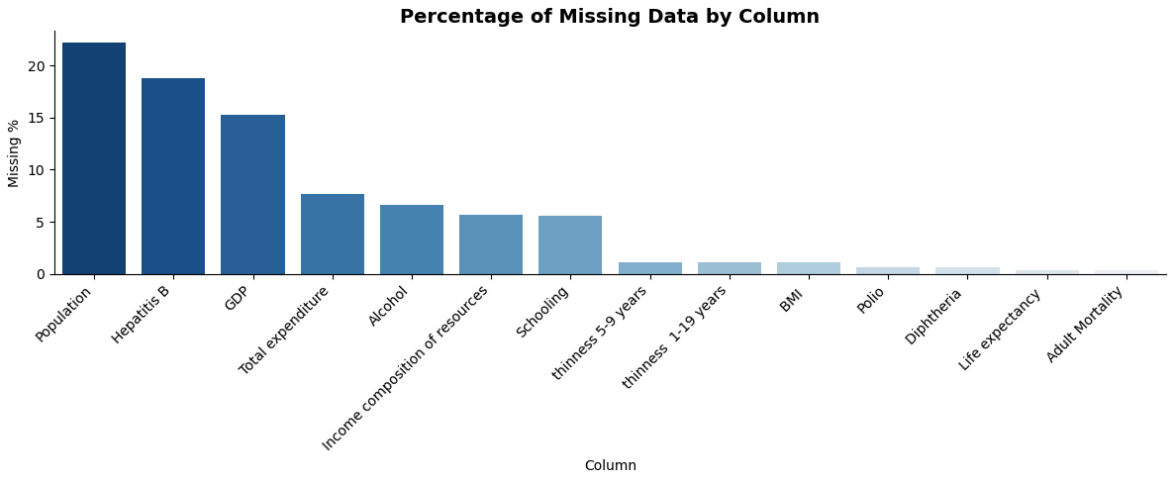
Missing Data Analysis

In [6]:
```
# ── Missing Data Percentage ──────────────────────────────────────
missing = pd.DataFrame({
    "Missing Count" : df_raw.isnull().sum(),
    "Missing %"     : (df_raw.isnull().mean() * 100).round(2)
}).sort_values("Missing %", ascending=False)

missing[missing["Missing %"] > 0]
```

Out[6]:

	Missing Count	Missing %
Population	652	22.19
Hepatitis B	553	18.82
GDP	448	15.25
Total expenditure	226	7.69
Alcohol	194	6.60
Income composition of resources	167	5.68
Schooling	163	5.55
thinness 5-9 years	34	1.16
thinness 1-19 years	34	1.16
BMI	34	1.16
Polio	19	0.65
Diphtheria	19	0.65
Life expectancy	10	0.34
Adult Mortality	10	0.34

In [7]:
```
# ── Visualising Missing Data ─────────────────────────────────────
missing_plot = missing[missing["Missing %"] > 0]

plt.figure(figsize=(12, 5))
sns.barplot(x=missing_plot.index, y=missing_plot["Missing %"], palette="Blues_r"
plt.title("Percentage of Missing Data by Column", fontsize=14, fontweight="bold"
plt.xlabel("Column")
plt.ylabel("Missing %")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig(OUTPUTS / "missing_data.png", dpi=150)
plt.show()
```



Missing Data Observations

- **Population (22%)** MAR: missingness concentrated in less developed/smaller nations with limited data infrastructure. Will impute with country-level median
- **Hepatitis B (19%)** MAR: confirmed by dataset brief as missing from less known countries. Will impute with country-level median
- **GDP (15%)** MAR: same as above, economic data unavailable for smaller nations. Will impute with country-level median
- **Schooling, Alcohol, Total expenditure, Income composition (5-8%)** MAR: country-level panel data. Will impute using grouped median by Country to maintain the time-series structure of the data
- **BMI, thinness, Polio, Diphtheria (<2%)** MCAR: low random missingness, will impute with country-level median
- **Life expectancy & Adult Mortality (0.34%)** is the target variable. Rows with missing values will be dropped entirely as we cannot impute what we are trying to predict

---

Data Cleaning & Feature Engineering

Cleaning Column Names

In [8]:
```
# ——Cleaning Column Names ————————————————————————————————————
# Strip leading/trailing whitespace from all column names
df_clean = df_raw.copy()
df_clean.columns = df_clean.columns.str.strip()

# Also cleaning up double spaces inside column names
df_clean.columns = df_clean.columns.str.replace(r'\s+', ' ', regex=True)

print("Cleaned column names:")
print(df_clean.columns.tolist())
```

```
Cleaned column names:
['Country', 'Year', 'Status', 'Life expectancy', 'Adult Mortality', 'infant death
s', 'Alcohol', 'percentage expenditure', 'Hepatitis B', 'Measles', 'BMI', 'under-
five deaths', 'Polio', 'Total expenditure', 'Diphtheria', 'HIV/AIDS', 'GDP', 'Pop
ulation', 'thinness 1-19 years', 'thinness 5-9 years', 'Income composition of res
ources', 'Schooling']
```

In [9]:
```python
# ── Handle Missing Values ──────────────────────────────────────────
# Drop rows where target variable is missing
before = df_clean.shape[0]
df_clean.dropna(subset=["Life expectancy", "Adult Mortality"], inplace=True)
after = df_clean.shape[0]
print(f" Dropped {before - after} rows with missing target variable")

# Country-grouped median imputation for panel data columns
group_cols = ["Schooling", "Alcohol", "Total expenditure",
              "Income composition of resources", "Hepatitis B",
              "GDP", "Population", "BMI", "thinness 1-19 years",
              "thinness 5-9 years", "Polio", "Diphtheria"]

for col in group_cols:
    before_nulls = df_clean[col].isnull().sum()
    df_clean[col] = df_clean.groupby("Country")[col].transform(
        lambda x: x.fillna(x.median())
    )
    # If still null (country had all nulls), fill with global median
    remaining = df_clean[col].isnull().sum()
    if remaining > 0:
        df_clean[col].fillna(df_clean[col].median(), inplace=True)
    print(f" {col:<35} {before_nulls} nulls imputed")

print(f"\n Remaining nulls : {df_clean.isnull().sum().sum()}")
print(f" Final shape     : {df_clean.shape}")
```

```
Dropped 10 rows with missing target variable
Schooling                           160 nulls imputed
Alcohol                             193 nulls imputed
Total expenditure                   226 nulls imputed
Income composition of resources     160 nulls imputed
Hepatitis B                         553 nulls imputed
GDP                                 443 nulls imputed
Population                          644 nulls imputed
BMI                                 32 nulls imputed
thinness 1-19 years                 32 nulls imputed
thinness 5-9 years                  32 nulls imputed
Polio                               19 nulls imputed
Diphtheria                          19 nulls imputed

Remaining nulls : 0
Final shape     : (2928, 22)
```

In [10]:
```python
# ── Feature Engineering ────────────────────────────────────────────
# Encode Status (Developing/Developed) as binary
df_clean["Status_encoded"] = (df_clean["Status"] == "Developed").astype(int)

# Mortality rate ratio (combined mortality pressure)
df_clean["mortality_ratio"] = df_clean["Adult Mortality"] / (df_clean["infant de

# Immunization coverage average across 3 vaccines
df_clean["immunization_avg"] = df_clean[["Hepatitis B", "Polio", "Diphtheria"]].
```

```
print(" Status encoded as binary (1=Developed, 0=Developing)")
print(" Mortality ratio engineered")
print(" Immunization average engineered")
print(f" Final shape : {df_clean.shape}")
```

```
Status encoded as binary (1=Developed, 0=Developing)
Mortality ratio engineered
Immunization average engineered
Final shape : (2928, 25)
```

In [11]:
```
# ── Saving Cleaned Data ──────────────────────────────────────────
df_clean.to_sql("life_expectancy_clean", conn, if_exists="replace", index=False)
df_clean.to_csv(OUTPUTS / "life_expectancy_cleaned.csv", index=False)

print(f" Final dataset : {df_clean.shape[0]:,} rows, {df_clean.shape[1]} columns
```

```
Final dataset : 2,928 rows, 25 columns
```

4.1 Cleaning Observations

- Column names stripped of leading/trailing whitespace and double spaces to prevent KeyError issues during analysis
- 10 rows dropped where target variable (Life expectancy) was missing as we cannot impute what we are predicting
- Country-grouped median imputation applied to all numeric columns. It respects the panel data structure by using each country's own historical data as the imputation reference rather than a global median
- Global median used as fallback for countries with all nulls in a column
- 3 new features engineered:
  - `Status_encoded` : binary encoding of Developed/Developing status
  - `mortality_ratio` : combined adult and infant mortality pressure
  - `immunization_avg` : average coverage across Hepatitis B, Polio and Diphtheria
- Final dataset: 2,928 rows, 25 columns ready for analysis

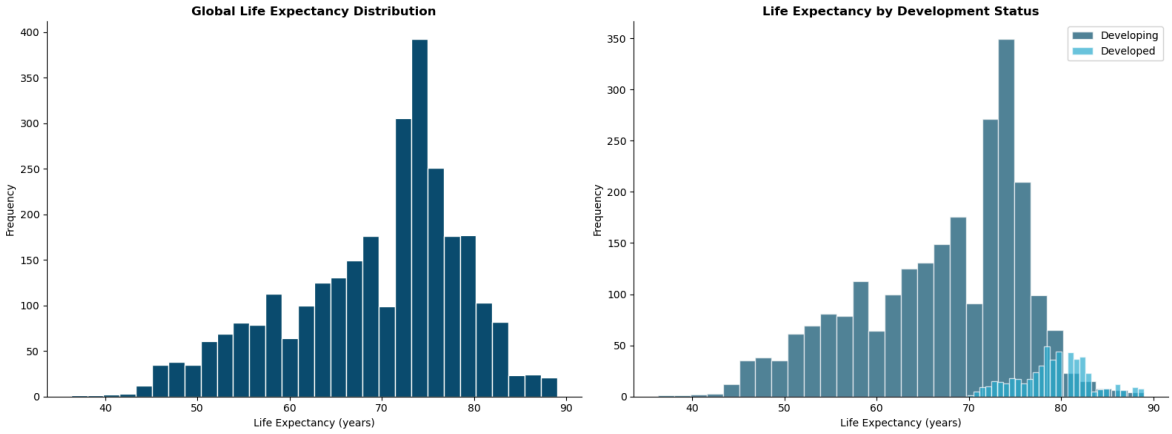
Exploratory Data Analysis

Life Expectancy Distribution

How is life expectancy distributed globally?

In [12]:
```
# ── Life Expectancy Distribution ─────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Overall distribution
axes[0].hist(df_clean["Life expectancy"], bins=30,
             color="#0a4f6e", edgecolor="white")
axes[0].set_title("Global Life Expectancy Distribution", fontweight="bold")
axes[0].set_xlabel("Life Expectancy (years)")
axes[0].set_ylabel("Frequency")

# By development status
```

```
developed   = df_clean[df_clean["Status"] == "Developed"]["Life expectancy"]
developing  = df_clean[df_clean["Status"] == "Developing"]["Life expectancy"]

axes[1].hist(developing, bins=30, alpha=0.7, color="#0a4f6e",
             label="Developing", edgecolor="white")
axes[1].hist(developed, bins=30, alpha=0.7, color="#2ab0d4",
             label="Developed", edgecolor="white")
axes[1].set_title("Life Expectancy by Development Status", fontweight="bold")
axes[1].set_xlabel("Life Expectancy (years)")
axes[1].set_ylabel("Frequency")
axes[1].legend()

plt.tight_layout()
plt.savefig(OUTPUTS / "life_expectancy_dist.png", dpi=150)
plt.show()

print(f"Global Mean    : {df_clean['Life expectancy'].mean():.1f} years")
print(f"Developed Mean : {developed.mean():.1f} years")
print(f"Developing Mean: {developing.mean():.1f} years")
print(f"Gap            : {developed.mean() - developing.mean():.1f} years")
```




```
Global Mean    : 69.2 years
Developed Mean : 79.2 years
Developing Mean: 67.1 years
Gap            : 12.1 years
```

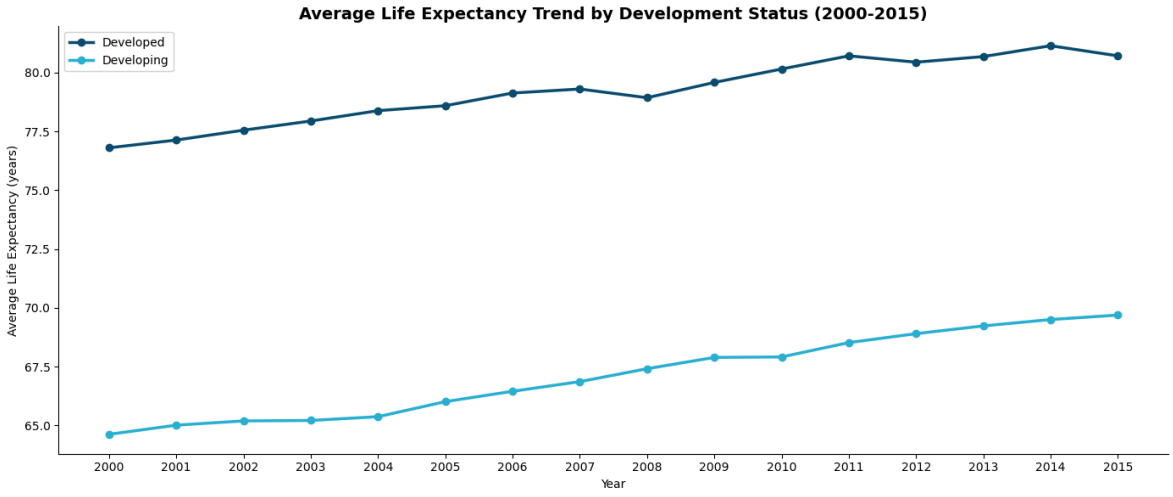
Observations

- Global mean life expectancy is 69.2 years across 193 countries from 2000-2015
- A striking 12.1 year gap exists between developed (79.2 years) and developing countries (67.1 years) a child's birthplace alone accounts for over a decade difference in expected lifespan
- The global distribution is bimodal. One peak around 65-70 years (developing nations) and another around 75-80 years (developed nations), confirming the two-speed nature of global health outcomes
- Developed country distribution is tightly clustered at the high end, while developing countries show much wider spread. This reflects greater inequality within the developing world

Life Expectancy Trends Over Time (2000-2015)

How has life expectancy evolved globally over the 15-year period?

In [13]:
```
# ——Trends Over Time ————————————————————————————————————
trend_df = pd.read_sql("""
    SELECT Year, Status,
           ROUND(AVG("Life expectancy"), 2) as Avg_Life_Expectancy
    FROM life_expectancy_clean
    GROUP BY Year, Status
    ORDER BY Year
""", conn)

plt.figure(figsize=(14, 6))
for status, group in trend_df.groupby("Status"):
    plt.plot(group["Year"], group["Avg_Life_Expectancy"],
             marker="o", linewidth=2.5,
             label=status,
             color="#0a4f6e" if status == "Developed" else "#2ab0d4")

plt.title("Average Life Expectancy Trend by Development Status (2000-2015)",
          fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Average Life Expectancy (years)")
plt.legend()
plt.xticks(range(2000, 2016))
plt.tight_layout()
plt.savefig(OUTPUTS / "life_expectancy_trend.png", dpi=150)
plt.show()
```



Observations

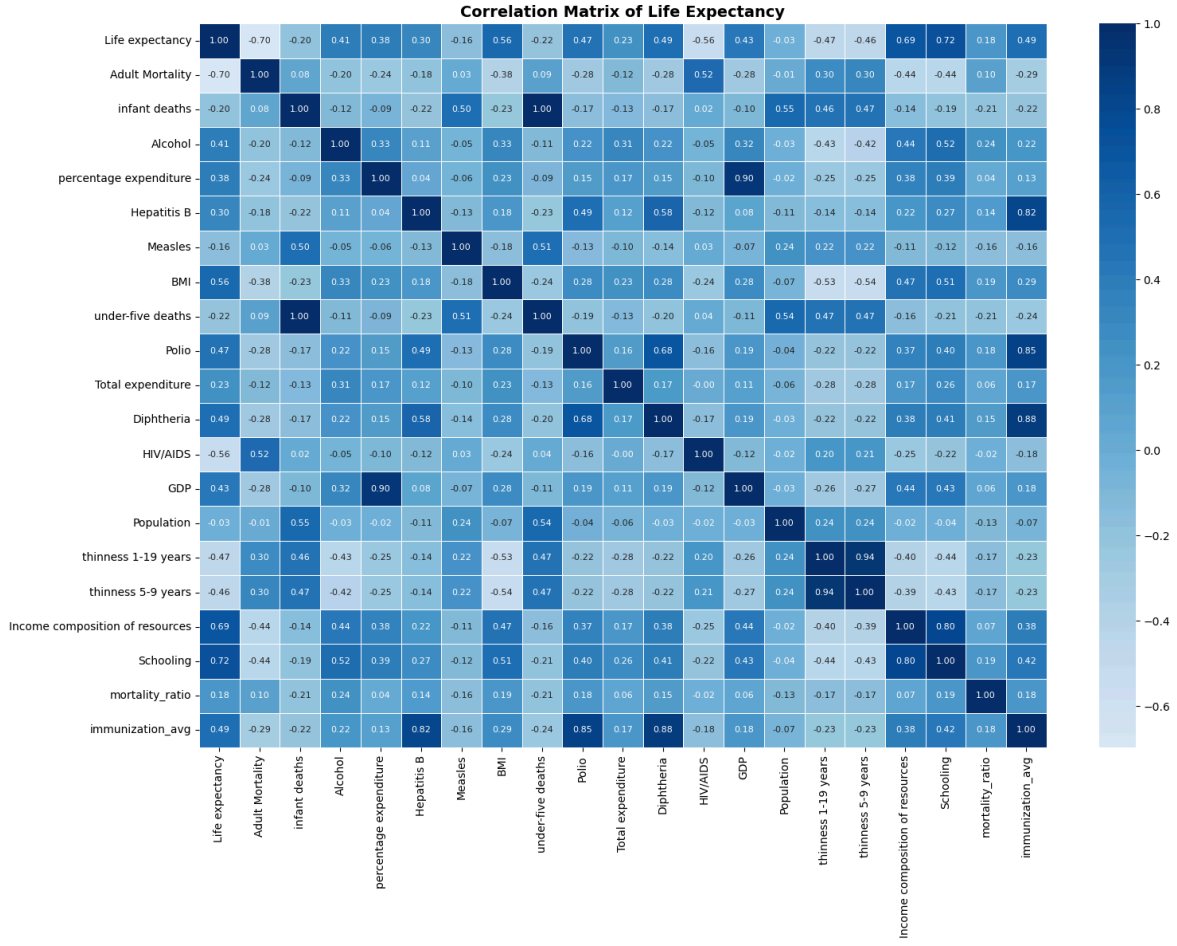
- Both developed and developing countries show consistent improvement in life expectancy from 2000-2015, reflecting global progress in healthcare, immunization and disease control
- Developing countries improved from 64.7 to 69.8 years. This is a gain of ~5 years over 15 years, suggesting accelerating healthcare development in lower-income nations
- Developed countries improved from 77.0 to 80.8 years which is a smaller absolute gain reflecting already high baseline levels
- A notable dip appears for developed countries around 2008 that is consistent with the global financial crisis, which reduced healthcare spending, increased unemployment and negatively impacted population health outcomes

- Despite both groups improving, the gap remains persistently wide. Convergence is occurring but slowly

Correlation Analysis

Which factors are most strongly correlated with life expectancy?

In [14]:
```
# ──Correlation Matrix ──────────────────────────────────────────────────
# Select numeric columns only
numeric_cols = df_clean.select_dtypes(include=np.number).columns.tolist()
numeric_cols = [c for c in numeric_cols if c not in ["Year", "Status_encoded"]]

corr_matrix = df_clean[numeric_cols].corr()

plt.figure(figsize=(16, 12))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="Blues",
            center=0, linewidths=0.5, annot_kws={"size": 8})
plt.title("Correlation Matrix of Life Expectancy",
          fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig(OUTPUTS / "correlation_matrix.png", dpi=150)
plt.show()
```

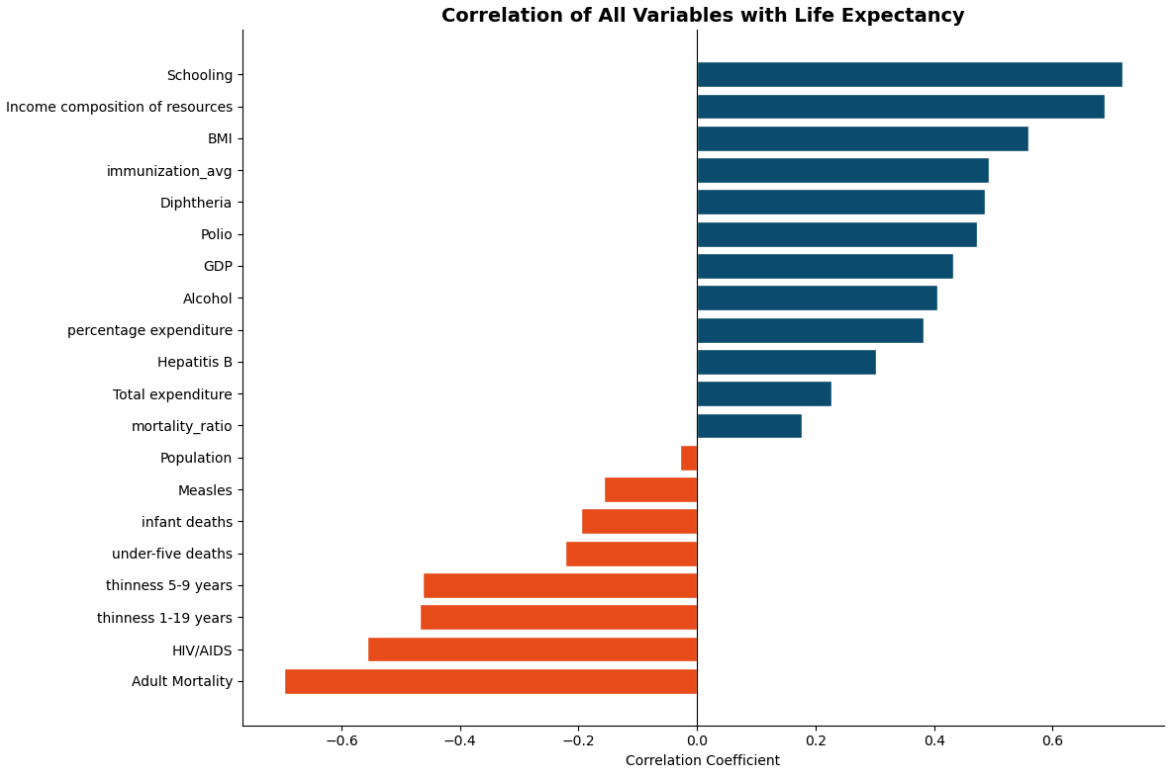


Observations

- **Schooling (0.72)** is the strongest positive predictor of life expectancy surpassing even GDP and healthcare expenditure. Education drives health literacy, earlier disease detection, better nutrition decisions and higher vaccination rates

- **Income composition of resources (0.69)** reflects how efficiently a country converts income into human development. This is a strong proxy for institutional quality
- **BMI (0.56)** is counterintuitively positive in this global context in developing nations low BMI signals malnutrition and poverty, so higher BMI reflects better nutrition and food security
- **Adult Mortality (-0.70)** is the strongest negative predictor. As expected, higher death rates among adults directly compress life expectancy
- **HIV/AIDS (-0.56)** has a strong negative impact particularly relevant for Sub-Saharan African countries where the epidemic significantly reduced life expectancy
- **Thinness (-0.47)** confirms that malnutrition is a major life expectancy driver in developing nations
- These correlations will directly inform our feature selection for the ML models

In [15]:
```
# ──Top Correlations with Life Expectancy ──────────────────────────
le_corr = corr_matrix["Life expectancy"].drop("Life expectancy").sort_values()

plt.figure(figsize=(12, 8))
colors = ["#e84f1c" if x < 0 else "#0a4f6e" for x in le_corr.values]
plt.barh(le_corr.index, le_corr.values, color=colors, edgecolor="white")
plt.title("Correlation of All Variables with Life Expectancy",
          fontsize=14, fontweight="bold")
plt.xlabel("Correlation Coefficient")
plt.axvline(x=0, color="black", linewidth=0.8)
plt.tight_layout()
plt.savefig(OUTPUTS / "correlations_life_expectancy.png", dpi=150)
plt.show()
```



Predicting Life Expectancy (Machine Learning)

Feature Selection & Data Preparation

Selecting features based on correlation analysis and preparing data for modelling.

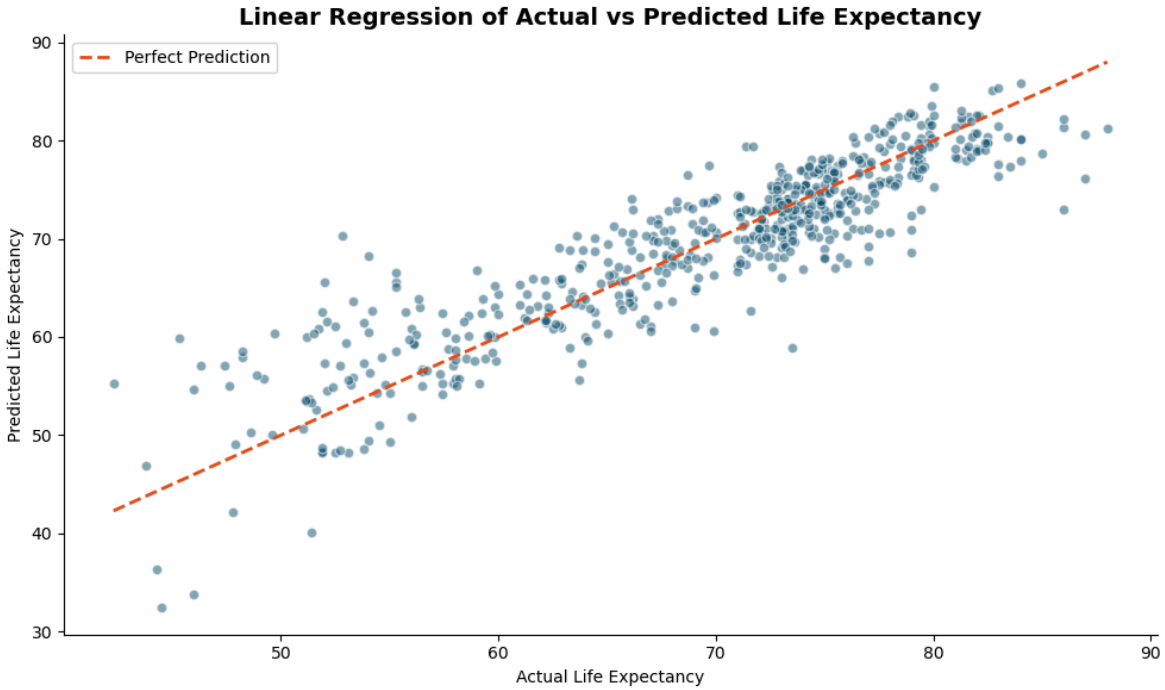
In [16]:
```
# ── Feature Selection & Train/Test Split ──────────────────────────
# Features selected after removing multicollinear variables:
# - immunization_avg replaces Hepatitis B, Polio, Diphtheria
# - infant deaths replaces under-five deaths (correlation = 1.00)
# - thinness 1-19 years replaces thinness 5-9 years (correlation = 0.94)

features = [
    "Adult Mortality",
    "Schooling",
    "Income composition of resources",
    "BMI",
    "HIV/AIDS",
    "GDP",
    "Alcohol",
    "percentage expenditure",
    "Total expenditure",
    "thinness 1-19 years",
    "immunization_avg",
    "Status_encoded",
    "infant deaths",
    "Population"
]

target = "Life expectancy"

X = df_clean[features]
y = df_clean[target]

# Train/test split — 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f" Features selected  : {len(features)}")
print(f" Multicollinear features removed : Hepatitis B, Polio, Diphtheria,")
print(f"  under-five deaths, thinness 5-9 years")
print(f" Features selected  : {len(features)}")
print(f" Training samples   : {X_train.shape[0]:,}")
print(f" Testing samples    : {X_test.shape[0]:,}")
```

```
Features selected  : 14
Multicollinear features removed : Hepatitis B, Polio, Diphtheria,
 under-five deaths, thinness 5-9 years
Features selected  : 14
Training samples   : 2,342
Testing samples    : 586
```

In [17]:
```
# ──Linear Regression Model ──────────────────────────────────────────
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

# Predictions
lr_pred = lr_model.predict(X_test)

# Evaluation
lr_r2   = r2_score(y_test, lr_pred)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
```

```
print("── Linear Regression Results ──────────────────────")
print(f"R² Score : {lr_r2:.4f}")
print(f"RMSE     : {lr_rmse:.4f} years")

# Visualise actual vs predicted
plt.figure(figsize=(10, 6))
plt.scatter(y_test, lr_pred, alpha=0.5, color="#0a4f6e", edgecolor="white")
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color="#e84f1c", linewidth=2, linestyle="--", label="Perfect Prediction
plt.title("Linear Regression of Actual vs Predicted Life Expectancy",
          fontsize=14, fontweight="bold")
plt.xlabel("Actual Life Expectancy")
plt.ylabel("Predicted Life Expectancy")
plt.legend()
plt.tight_layout()
plt.savefig(OUTPUTS / "lr_actual_vs_predicted.png", dpi=150)
plt.show()
```

```
── Linear Regression Results ──────────────────────
R² Score : 0.8078
RMSE     : 4.0773 years
```

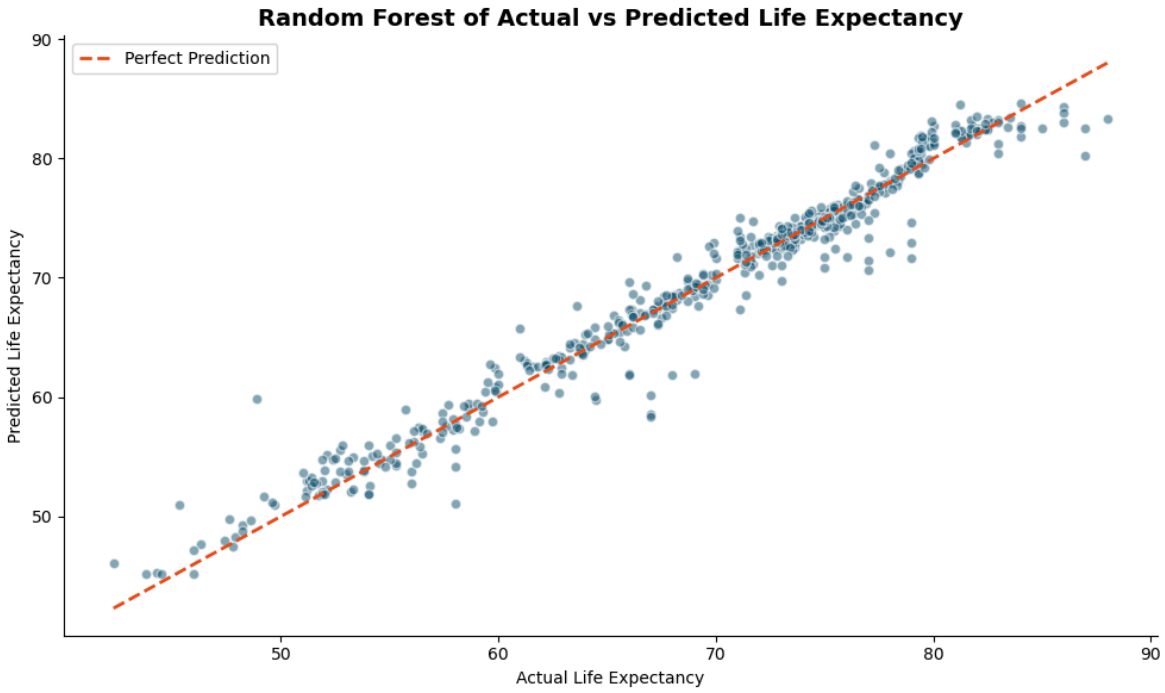


In [18]:

```
# ──Random Forest Model ──────────────────────────────────────────
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Predictions
rf_pred = rf_model.predict(X_test)

# Evaluation
rf_r2   = r2_score(y_test, rf_pred)
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_pred))

print("── Random Forest Results ──────────────────────────")
print(f"R² Score : {rf_r2:.4f}")
print(f"RMSE     : {rf_rmse:.4f} years")
```

```
# Visualise actual vs predicted
plt.figure(figsize=(10, 6))
plt.scatter(y_test, rf_pred, alpha=0.5, color="#0a4f6e", edgecolor="white")
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         color="#e84f1c", linewidth=2, linestyle="--", label="Perfect Prediction
plt.title("Random Forest of Actual vs Predicted Life Expectancy",
          fontsize=14, fontweight="bold")
plt.xlabel("Actual Life Expectancy")
plt.ylabel("Predicted Life Expectancy")
plt.legend()
plt.tight_layout()
plt.savefig(OUTPUTS / "rf_actual_vs_predicted.png", dpi=150)
plt.show()
```

```
── Random Forest Results ──────────────────────────
R² Score : 0.9660
RMSE     : 1.7148 years
```

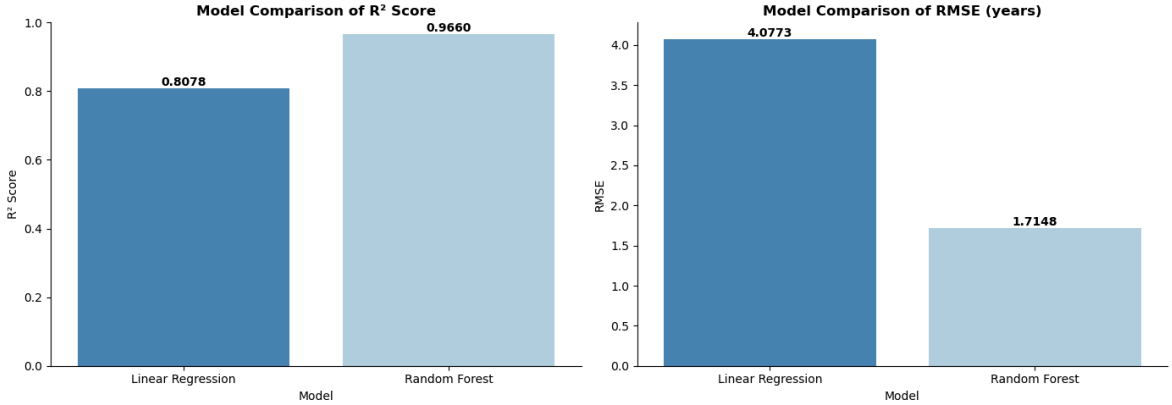


In [19]:

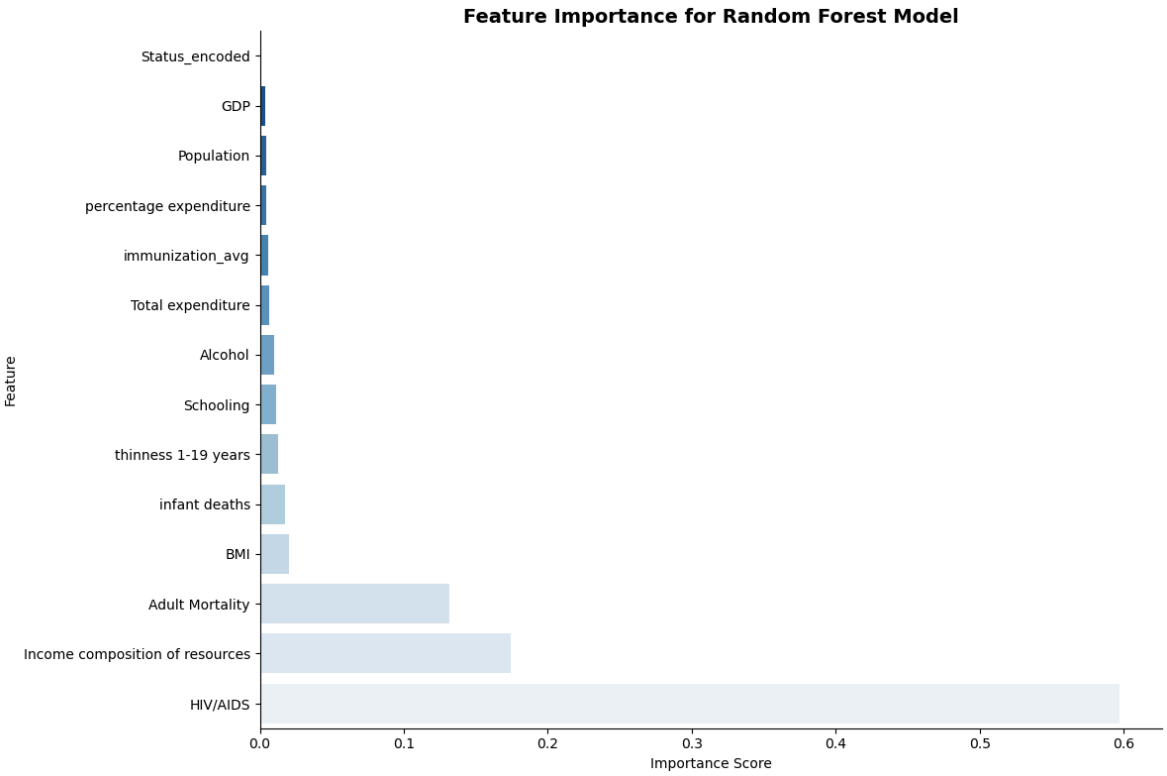
```
# ──Model Comparison ──────────────────────────────────────────────────
comparison = pd.DataFrame({
    "Model"   : ["Linear Regression", "Random Forest"],
    "R² Score": [lr_r2, rf_r2],
    "RMSE"    : [lr_rmse, rf_rmse]
})

print(comparison.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# R² comparison
sns.barplot(data=comparison, x="Model", y="R² Score",
            palette="Blues_r", ax=axes[0])
axes[0].set_title("Model Comparison of R² Score", fontweight="bold")
axes[0].set_ylim(0, 1)
for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height():.4f}",
                     (p.get_x() + p.get_width() / 2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")
```

```
# RMSE comparison
sns.barplot(data=comparison, x="Model", y="RMSE",
            palette="Blues_r", ax=axes[1])
axes[1].set_title("Model Comparison of RMSE (years)", fontweight="bold")
for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.4f}",
                     (p.get_x() + p.get_width() / 2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")

plt.tight_layout()
plt.savefig(OUTPUTS / "model_comparison.png", dpi=150)
plt.show()
```

Model	R^2 Score	RMSE
Linear Regression	0.81	4.08
Random Forest	0.97	1.71



In [20]:
```
# ——Feature Importance (Random Forest) ————————————————————————
importance_df = pd.DataFrame({
    "Feature"   : features,
    "Importance": rf_model.feature_importances_
}).sort_values("Importance", ascending=True)

plt.figure(figsize=(12, 8))
sns.barplot(data=importance_df, x="Importance", y="Feature", palette="Blues_r")
plt.title("Feature Importance for Random Forest Model",
          fontsize=14, fontweight="bold")
plt.xlabel("Importance Score")
plt.ylabel("Feature")
plt.tight_layout()
plt.savefig(OUTPUTS / "feature_importance.png", dpi=150)
plt.show()
```



Model Results & Feature Importance Observations

Model	R^2 Score	RMSE
Linear Regression	0.8078	4.08 years
Random Forest	0.9660	1.71 years

- **Random Forest significantly outperforms Linear Regression** . R^2 improves from 0.81 to 0.97 and RMSE drops from 4.08 to 1.71 years, reflecting Random Forest's ability to capture non-linear relationships and feature interactions
- **Linear Regression** still performs respectably at 0.81 R^2 confirming that many relationships in this dataset have a meaningful linear component
- **HIV/AIDS is the most important feature (0.60)** in the Random Forest model despite ranking second in correlation analysis. This reflects its non-linear impact: most countries have low rates but Sub-Saharan African countries with very high HIV/AIDS rates experience dramatic life expectancy drops that Random Forest captures through decision tree splits
- **Income composition of resources (0.17)** ranks second, confirming that efficient conversion of national income into human development is a critical life expectancy driver
- **Adult Mortality (0.13)** ranks third with direct mortality pressure as expected
- **Schooling**, despite being the top correlator, has relatively low feature importance in Random Forest likely because its effect is partially captured through Income composition of resources which is highly correlated with it
- **GDP, Status and Population have near-zero importance** suggesting that raw economic size matters less than how resources are distributed and utilised

---

Summary & Conclusions

In [21]:
```
# ── Summary Statistics ──────────────────────────────────────
summary = pd.read_sql("""
    SELECT
        COUNT(DISTINCT Country) as Total_Countries,
        COUNT(DISTINCT Year) as Years_Covered,
        ROUND(AVG("Life expectancy"), 1) as Global_Avg_Life_Expectancy,
        ROUND(MIN("Life expectancy"), 1) as Min_Life_Expectancy,
        ROUND(MAX("Life expectancy"), 1) as Max_Life_Expectancy,
        ROUND(AVG(CASE WHEN Status='Developed'
                  THEN "Life expectancy" END), 1) as Developed_Avg,
        ROUND(AVG(CASE WHEN Status='Developing'
                  THEN "Life expectancy" END), 1) as Developing_Avg,
        ROUND(AVG(Schooling), 1) as Avg_Schooling_Years,
        ROUND(AVG("Adult Mortality"), 1) as Avg_Adult_Mortality
    FROM life_expectancy_clean
""", conn)

summary.T.rename(columns={0: "Value"})
```

Out[21]:

	Value
Total_Countries	183.00
Years_Covered	16.00
Global_Avg_Life_Expectancy	69.20
Min_Life_Expectancy	36.30
Max_Life_Expectancy	89.00
Developed_Avg	79.20
Developing_Avg	67.10
Avg_Schooling_Years	12.00
Avg_Adult_Mortality	164.80

Key Metrics

Metric	Value
Total Countries	183
Years Covered	2000-2015
Global Average Life Expectancy	69.2 years
Minimum Life Expectancy	36.3 years
Maximum Life Expectancy	89.0 years
Developed Countries Average	79.2 years
Developing Countries Average	67.1 years
Development Gap	12.1 years

Metric	Value
Average Schooling Years	12.0 years
Average Adult Mortality	164.8 per 1,000

Answers to Key Project Questions

1. **Which factors most significantly affect life expectancy?** HIV/AIDS, Income composition of resources and Adult Mortality are the strongest predictors per Random Forest feature importance. Schooling and BMI show the strongest linear correlations.

2. **Should countries with low life expectancy increase healthcare expenditure?** Healthcare expenditure alone has low predictive power. The data suggests investing in education, income distribution and HIV/AIDS control yields greater life expectancy improvements than raw spending increases.

3. **How do infant and adult mortality affect life expectancy?** Adult Mortality has a strong negative correlation (-0.70) and is the third most important ML feature. Infant mortality compounds this effect. Both directly compress life expectancy and signal broader healthcare system failures.

4. **What is the impact of schooling on lifespan?** Schooling is the strongest linear predictor (0.72 correlation) as more years of education directly improves health literacy, disease prevention and healthcare seeking behaviour.

5. **Can we reliably predict life expectancy from these factors?** Yes. The Random Forest model achieves R^2=0.966 and RMSE=1.71 years, meaning predictions are accurate to within less than 2 years using 14 health, economic and social indicators.

Model Summary

Model	R^2 Score	RMSE	Verdict
Linear Regression	0.8078	4.08 years	Good baseline
Random Forest	0.9660	1.71 years	Strong and recommended

Limitations

- Data spans 2000-2015 and does not reflect post-2015 developments including COVID-19 pandemic impact on global life expectancy
- Missing data was concentrated in smaller/less developed nations. Imputation may introduce bias for those countries
- Feature importance from Random Forest can be influenced by correlated features so results should be interpreted alongside domain knowledge
- The 36.3 year minimum life expectancy reflects extreme conflict or epidemic situations that may be outliers rather than representative of sustained trends

EDA & Analysis Of OCD Patient Dataset: Demographics & Clinical Data

Author	Ibrahim A. Mikail
Internship	Unified Mentor — Healthcare Data Analyst
Project	4 of 5
Tools	Python · Pandas · SQLite · Matplotlib · Seaborn
Dataset	OCD Patient Dataset via Kaggle
Date	May 2026

Project Overview

This dataset contains demographic and clinical information for 1,500 individuals diagnosed with Obsessive-Compulsive Disorder (OCD). It covers patient demographics, symptom profiles, Y-BOCS severity scores, co-occurring mental health conditions, medication patterns and family history of OCD.

Objectives

- Understand the demographic profile of OCD patients
- Analyse OCD symptom types, severity scores and their distributions
- Investigate co-occurring conditions (depression, anxiety) across patient groups
- Explore medication patterns and treatment approaches
- Examine the relationship between demographics and OCD severity

In [1]:
```
# ── Project Setup ──────────────────────────────────────────────
import os
from pathlib import Path

ROOT    = Path().resolve()
DATA    = ROOT / "data"
OUTPUTS = ROOT / "outputs"
DB_PATH = ROOT / "ocd_patients.db"

for folder in [DATA, OUTPUTS]:
    folder.mkdir(parents=True, exist_ok=True)
    print(f" {folder}")

print(f"\nProject root : {ROOT}")
print(f"Database will be created at : {DB_PATH}")
```

```
 C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_4_
ocd_patients\data
 C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_4_
ocd_patients\outputs

Project root : C:\Users\DELL\Documents\Data Science Projects\unified_mentor_proje
cts\project_4_ocd_patients
Database will be created at : C:\Users\DELL\Documents\Data Science Projects\unifi
ed_mentor_projects\project_4_ocd_patients\ocd_patients.db
```

In [2]:
```python
# ── Imports & Global Settings ──────────────────────────────────────
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns

# Display settings
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", "{:.2f}".format)

# Chart style
plt.rcParams["figure.figsize"] = (12, 5)
plt.rcParams["axes.spines.top"]   = False
plt.rcParams["axes.spines.right"] = False
sns.set_palette("Blues_r")

import warnings
warnings.filterwarnings("ignore")
```

Loading Data & Ingesting to SQLite

In [3]:
```python
# ── Load CSV & Push to SQLite ──────────────────────────────────────
df_raw = pd.read_csv(DATA / "OCD Patient Dataset_ Demographics & Clinical Data.c

# Connect to SQLite
conn = sqlite3.connect(DB_PATH)

# Push to SQLite
df_raw.to_sql("ocd_patients", conn, if_exists="replace", index=False)

print(f" CSV loaded           : {df_raw.shape[0]:,} rows, {df_raw.shape[1]} colu
print(f" SQLite table created : 'ocd_patients'")
print(f" Database saved at    : {DB_PATH}")
```

```
 CSV loaded           : 1,500 rows, 17 columns
 SQLite table created : 'ocd_patients'
 Database saved at    : C:\Users\DELL\Documents\Data Science Projects\unified_men
tor_projects\project_4_ocd_patients\ocd_patients.db
```

In [4]:
```python
# ── Initial Inspection ─────────────────────────────────────────────
df = pd.read_sql("SELECT * FROM ocd_patients LIMIT 5", conn)
df
```

Out[4]:

	Patient ID	Age	Gender	Ethnicity	Marital Status	Education Level	OCD Diagnosis Date	Duration of Symptoms (months)	Previc Diagno
0	1018	32	Female	African	Single	Some College	2016-07-15	203	M
1	2406	69	Male	African	Divorced	Some College	2017-04-28	180	Nc
2	1188	57	Male	Hispanic	Divorced	College Degree	2018-02-02	173	M
3	6200	27	Female	Hispanic	Married	College Degree	2014-08-25	126	PT
4	5824	56	Female	Hispanic	Married	High School	2022-02-20	168	PT

In [5]:
```
# ── Shape, Columns & Data Types ──────────────────────────────────────
print(f"Shape: {df_raw.shape}")
print(f"\nColumns:\n{df_raw.columns.tolist()}")
print(f"\nData Types:\n{df_raw.dtypes}")
```

```
Shape: (1500, 17)

Columns:
['Patient ID', 'Age', 'Gender', 'Ethnicity', 'Marital Status', 'Education Level',
'OCD Diagnosis Date', 'Duration of Symptoms (months)', 'Previous Diagnoses', 'Fam
ily History of OCD', 'Obsession Type', 'Compulsion Type', 'Y-BOCS Score (Obsessio
ns)', 'Y-BOCS Score (Compulsions)', 'Depression Diagnosis', 'Anxiety Diagnosis',
'Medications']

Data Types:
Patient ID                          int64
Age                                 int64
Gender                             object
Ethnicity                          object
Marital Status                     object
Education Level                    object
OCD Diagnosis Date                 object
Duration of Symptoms (months)       int64
Previous Diagnoses                 object
Family History of OCD              object
Obsession Type                     object
Compulsion Type                    object
Y-BOCS Score (Obsessions)           int64
Y-BOCS Score (Compulsions)          int64
Depression Diagnosis               object
Anxiety Diagnosis                  object
Medications                        object
dtype: object
```

Missing Data Analysis

In [6]:
```
# ──Missing Data Percentage ──────────────────────────────────────────
missing = pd.DataFrame({
    "Missing Count" : df_raw.isnull().sum(),
    "Missing %"     : (df_raw.isnull().mean() * 100).round(2)
}).sort_values("Missing %", ascending=False)

missing
```

Out[6]:

	Missing Count	Missing %
Medications	386	25.73
Previous Diagnoses	248	16.53
Gender	0	0.00
Ethnicity	0	0.00
Marital Status	0	0.00
Education Level	0	0.00
OCD Diagnosis Date	0	0.00
Duration of Symptoms (months)	0	0.00
Age	0	0.00
Family History of OCD	0	0.00
Obsession Type	0	0.00
Compulsion Type	0	0.00
Y-BOCS Score (Obsessions)	0	0.00
Y-BOCS Score (Compulsions)	0	0.00
Depression Diagnosis	0	0.00
Anxiety Diagnosis	0	0.00
Patient ID	0	0.00

Missing Data Observations

- This is a remarkably clean dataset as only 2 columns have missing values
- **Medications (25.73%)** MNAR: missing values indicate the patient is not currently on any medication for OCD. Will be filled with "None"
- **Previous Diagnoses (16.53%)** MNAR: missing values indicate no previous psychiatric diagnosis on record. Will be filled with "None"
- All clinical measurement columns (Y-BOCS scores, symptom types, diagnosis flags) are fully complete.

Data Cleaning & Feature Engineering

Clean & Prepare Data

In [7]:
```
# ——Data Cleaning ————————————————————————————————————————
df_clean = df_raw.copy()

# Dropping Patient ID since its unique identifier, no analytical value
df_clean.drop(columns=["Patient ID"], inplace=True)

# Filling missing values
df_clean["Medications"]         = df_clean["Medications"].fillna("None")
df_clean["Previous Diagnoses"]  = df_clean["Previous Diagnoses"].fillna("None")

# Converting OCD Diagnosis Date to datetime
df_clean["OCD Diagnosis Date"]  = pd.to_datetime(df_clean["OCD Diagnosis Date"])

# Extracting diagnosis year and month
df_clean["Diagnosis Year"]  = df_clean["OCD Diagnosis Date"].dt.year
df_clean["Diagnosis Month"] = df_clean["OCD Diagnosis Date"].dt.month_name()

# Engineering total Y-BOCS score
df_clean["Y-BOCS Total"] = (df_clean["Y-BOCS Score (Obsessions)"] +
                            df_clean["Y-BOCS Score (Compulsions)"])

# Engineering severity category based on total Y-BOCS score
def classify_severity(score):
    if score <= 14:
        return "Subclinical/Mild"
    elif score <= 30:
        return "Moderate"
    elif score <= 46:
        return "Severe"
    else:
        return "Extreme"

df_clean["OCD Severity"] = df_clean["Y-BOCS Total"].apply(classify_severity)

print(f" Patient ID dropped")
print(f" Missing values filled")
print(f" OCD Diagnosis Date converted to datetime")
print(f" Diagnosis Year and Month extracted")
print(f" Y-BOCS Total score engineered")
print(f" OCD Severity category engineered")
print(f" Final shape : {df_clean.shape}")
```

```
Patient ID dropped
Missing values filled
OCD Diagnosis Date converted to datetime
Diagnosis Year and Month extracted
Y-BOCS Total score engineered
OCD Severity category engineered
Final shape : (1500, 20)
```

In [8]:
```
# —— Cell 8: Save Cleaned Data ——————————————————————————————
df_clean.to_sql("ocd_clean", conn, if_exists="replace", index=False)
df_clean.to_csv(OUTPUTS / "ocd_patients_cleaned.csv", index=False)
```

```
print(f" Final dataset : {df_clean.shape[0]:,} rows, {df_clean.shape[1]} columns
```

```
Final dataset : 1,500 rows, 20 columns
```

Cleaning Observations

- `Patient ID` dropped as its a unique identifier with no analytical value
- Missing values in `Medications` and `Previous Diagnoses` filled with "None", reflecting their MNAR classification where absence means genuinely none recorded
- `OCD Diagnosis Date` converted from string to datetime, enabling time series analysis
- 4 new features engineered:
  - `Diagnosis Year` and `Diagnosis Month` extracted from diagnosis date
  - `Y-BOCS Total` is sum of obsession and compulsion scores (range 0-80)
  - `OCD Severity` is the clinical severity classification based on total Y-BOCS score (Subclinical/Mild ≤14, Moderate ≤30, Severe ≤46, Extreme >46)
- Final dataset: 1,500 rows, 20 columns and ready for analysis

---

Exploratory Data Analysis

Demographic Overview

Understanding the demographic profile of OCD patients in this dataset.

In [9]:
```
# ──Demographic Overview ──────────────────────────────────────────────
demo_df = pd.read_sql("""
    SELECT
        COUNT(*) as Total_Patients,
        ROUND(AVG(Age), 1) as Avg_Age,
        MIN(Age) as Min_Age,
        MAX(Age) as Max_Age,
        SUM(CASE WHEN Gender = 'Male' THEN 1 ELSE 0 END) as Male,
        SUM(CASE WHEN Gender = 'Female' THEN 1 ELSE 0 END) as Female,
        SUM(CASE WHEN "Family History of OCD" = 'Yes' THEN 1 ELSE 0 END) as Fami
        SUM(CASE WHEN "Depression Diagnosis" = 'Yes' THEN 1 ELSE 0 END) as Has_D
        SUM(CASE WHEN "Anxiety Diagnosis" = 'Yes' THEN 1 ELSE 0 END) as Has_Anxi
    FROM ocd_clean
""", conn)

demo_df.T.rename(columns={0: "Value"})
```

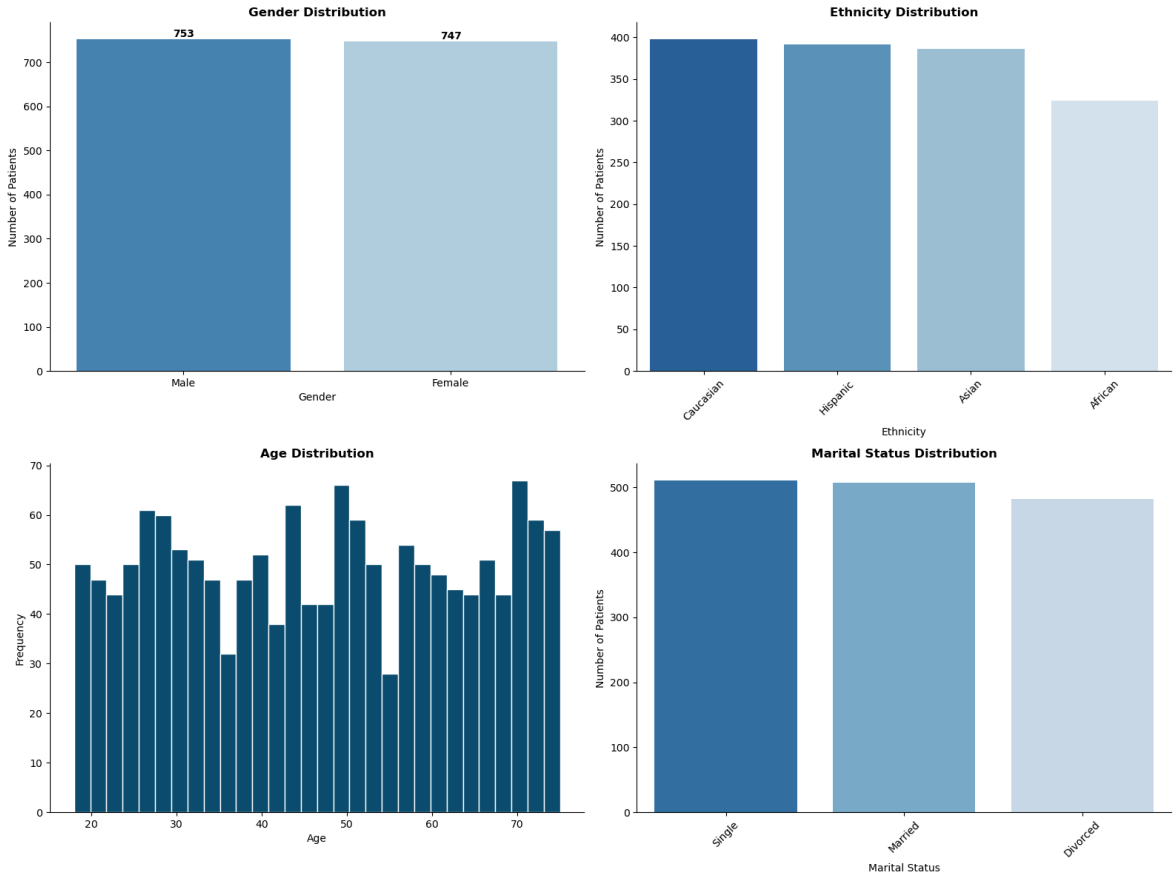
Out[9]:

	Value
Total_Patients	1500.00
Avg_Age	46.80
Min_Age	18.00
Max_Age	75.00
Male	753.00
Female	747.00
Family_History	760.00
Has_Depression	772.00
Has_Anxiety	751.00

In [10]:

```
# ——Demographic Visualisations ————————————————————————————————
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Gender distribution
gender_df = df_clean["Gender"].value_counts().reset_index()
sns.barplot(data=gender_df, x="Gender", y="count",
            palette="Blues_r", ax=axes[0,0])
axes[0,0].set_title("Gender Distribution", fontweight="bold")
axes[0,0].set_xlabel("Gender")
axes[0,0].set_ylabel("Number of Patients")
for p in axes[0,0].patches:
    axes[0,0].annotate(f"{int(p.get_height()):,}",
                       (p.get_x() + p.get_width()/2, p.get_height()),
                       ha="center", va="bottom", fontweight="bold")

# Ethnicity distribution
eth_df = df_clean["Ethnicity"].value_counts().reset_index()
sns.barplot(data=eth_df, x="Ethnicity", y="count",
            palette="Blues_r", ax=axes[0,1])
axes[0,1].set_title("Ethnicity Distribution", fontweight="bold")
axes[0,1].set_xlabel("Ethnicity")
axes[0,1].set_ylabel("Number of Patients")
axes[0,1].tick_params(axis='x', rotation=45)

# Age distribution
axes[1,0].hist(df_clean["Age"], bins=30, color="#0a4f6e", edgecolor="white")
axes[1,0].set_title("Age Distribution", fontweight="bold")
axes[1,0].set_xlabel("Age")
axes[1,0].set_ylabel("Frequency")

# Marital status
marital_df = df_clean["Marital Status"].value_counts().reset_index()
sns.barplot(data=marital_df, x="Marital Status", y="count",
            palette="Blues_r", ax=axes[1,1])
axes[1,1].set_title("Marital Status Distribution", fontweight="bold")
axes[1,1].set_xlabel("Marital Status")
axes[1,1].set_ylabel("Number of Patients")
axes[1,1].tick_params(axis='x', rotation=45)

plt.tight_layout()
```

```
plt.savefig(OUTPUTS / "demographics.png", dpi=150)
plt.show()
```



Observations

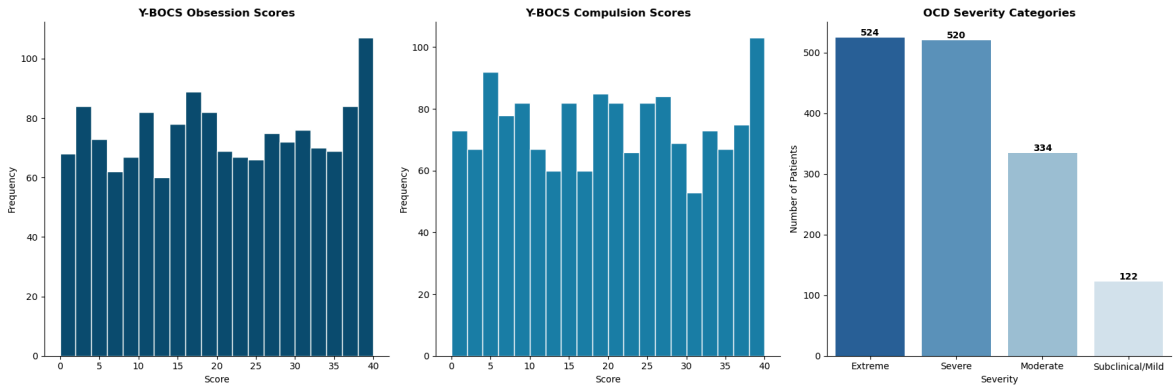
- Gender is nearly perfectly balanced with 753 Male (50.2%) and 747 Female (49.8%), suggesting OCD affects both genders equally in this population
- Ethnicity distribution is fairly even across Caucasian, Hispanic and Asian groups with African patients slightly underrepresented. This is worth noting for research diversity
- Age distribution is relatively uniform from 18-75 with average age of 46.8 years as OCD affects patients across all adult age groups
- Marital status is evenly split across Single, Married and Divorced. No strong association with relationship status apparent at first glance
- **51% of patients have a family history of OCD** strongly suggesting genetic or environmental hereditary factors in OCD development
- **51% have comorbid Depression and 50% have comorbid Anxiety**. Half of all OCD patients carry at least one additional mental health diagnosis, highlighting the complex, interconnected nature of mental health conditions

OCD Severity Distribution

How severe is OCD among patients based on Y-BOCS scores?

In [11]:
```
# ── OCD Severity Distribution ──────────────────────────────────
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Y-BOCS Obsessions distribution
axes[0].hist(df_clean["Y-BOCS Score (Obsessions)"], bins=20,
```

```
            color="#0a4f6e", edgecolor="white")
axes[0].set_title("Y-BOCS Obsession Scores", fontweight="bold")
axes[0].set_xlabel("Score")
axes[0].set_ylabel("Frequency")

# Y-BOCS Compulsions distribution
axes[1].hist(df_clean["Y-BOCS Score (Compulsions)"], bins=20,
            color="#1a7ea8", edgecolor="white")
axes[1].set_title("Y-BOCS Compulsion Scores", fontweight="bold")
axes[1].set_xlabel("Score")
axes[1].set_ylabel("Frequency")

# Severity category
severity_df = df_clean["OCD Severity"].value_counts().reset_index()
sns.barplot(data=severity_df, x="OCD Severity", y="count",
            palette="Blues_r", ax=axes[2])
axes[2].set_title("OCD Severity Categories", fontweight="bold")
axes[2].set_xlabel("Severity")
axes[2].set_ylabel("Number of Patients")
for p in axes[2].patches:
    axes[2].annotate(f"{int(p.get_height()):,}",
                     (p.get_x() + p.get_width()/2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")

plt.tight_layout()
plt.savefig(OUTPUTS / "severity_distribution.png", dpi=150)
plt.show()

print(f"Avg Y-BOCS Obsessions  : {df_clean['Y-BOCS Score (Obsessions)'].mean():.
print(f"Avg Y-BOCS Compulsions : {df_clean['Y-BOCS Score (Compulsions)'].mean():
print(f"Avg Y-BOCS Total       : {df_clean['Y-BOCS Total'].mean():.1f}")
```




```
Avg Y-BOCS Obsessions  : 20.0
Avg Y-BOCS Compulsions : 19.6
Avg Y-BOCS Total       : 39.7
```

Observations

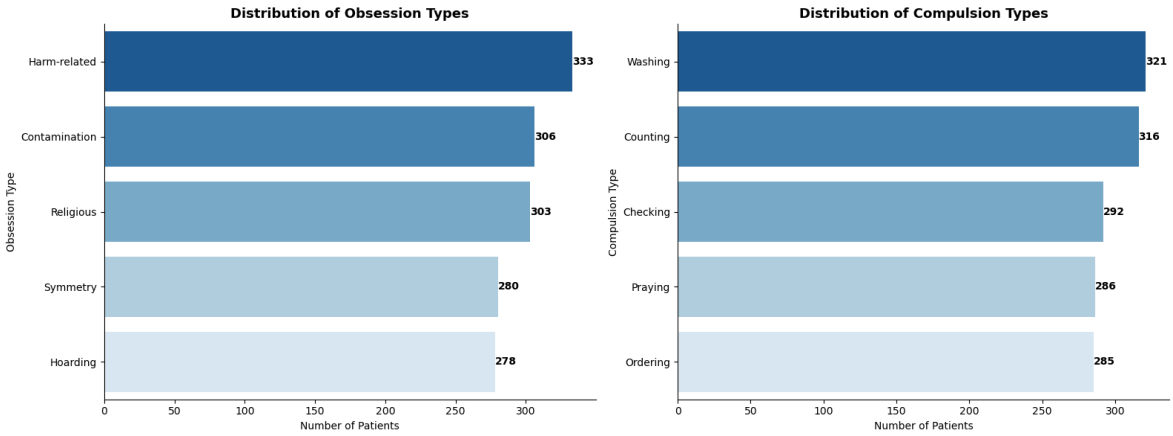
- Both Y-BOCS Obsession and Compulsion score distributions are relatively uniform across the full range as patients are spread across all severity levels with a slight skew toward higher scores
- Average Y-BOCS scores: Obsessions 20.0, Compulsions 19.6, Total 39.7. Placing the average patient at the Severe/Extreme boundary
- **70% of patients (1,044) fall in Severe or Extreme categories**. This is a highly affected clinical population, not a mild OCD sample

- Only 122 patients (8%) are Subclinical/Mild. Suggesting this dataset likely captures patients who sought clinical help, introducing selection bias toward more severe cases
- Obsession and Compulsion scores are nearly equal on average (20.0 vs 19.6). Indicating OCD manifests with roughly balanced obsessive thoughts and compulsive behaviours in this population

Obsession & Compulsion Type Distribution

What are the most common OCD symptom types?

In [12]:
```
# ── Obsession & Compulsion Types ──────────────────────────────────
obs_df = pd.read_sql("""
    SELECT "Obsession Type", COUNT(*) as Count
    FROM ocd_clean
    GROUP BY "Obsession Type"
    ORDER BY Count DESC
""", conn)

comp_df = pd.read_sql("""
    SELECT "Compulsion Type", COUNT(*) as Count
    FROM ocd_clean
    GROUP BY "Compulsion Type"
    ORDER BY Count DESC
""", conn)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

sns.barplot(data=obs_df, x="Count", y="Obsession Type",
            palette="Blues_r", ax=axes[0])
axes[0].set_title("Distribution of Obsession Types",
                  fontsize=13, fontweight="bold")
axes[0].set_xlabel("Number of Patients")
axes[0].set_ylabel("Obsession Type")
for p in axes[0].patches:
    axes[0].annotate(f"{int(p.get_width()):,}",
                     (p.get_width(), p.get_y() + p.get_height()/2),
                     ha="left", va="center", fontweight="bold")

sns.barplot(data=comp_df, x="Count", y="Compulsion Type",
            palette="Blues_r", ax=axes[1])
axes[1].set_title("Distribution of Compulsion Types",
                  fontsize=13, fontweight="bold")
axes[1].set_xlabel("Number of Patients")
axes[1].set_ylabel("Compulsion Type")
for p in axes[1].patches:
    axes[1].annotate(f"{int(p.get_width()):,}",
                     (p.get_width(), p.get_y() + p.get_height()/2),
                     ha="left", va="center", fontweight="bold")

plt.tight_layout()
plt.savefig(OUTPUTS / "symptom_types.png", dpi=150)
plt.show()
```



Observations

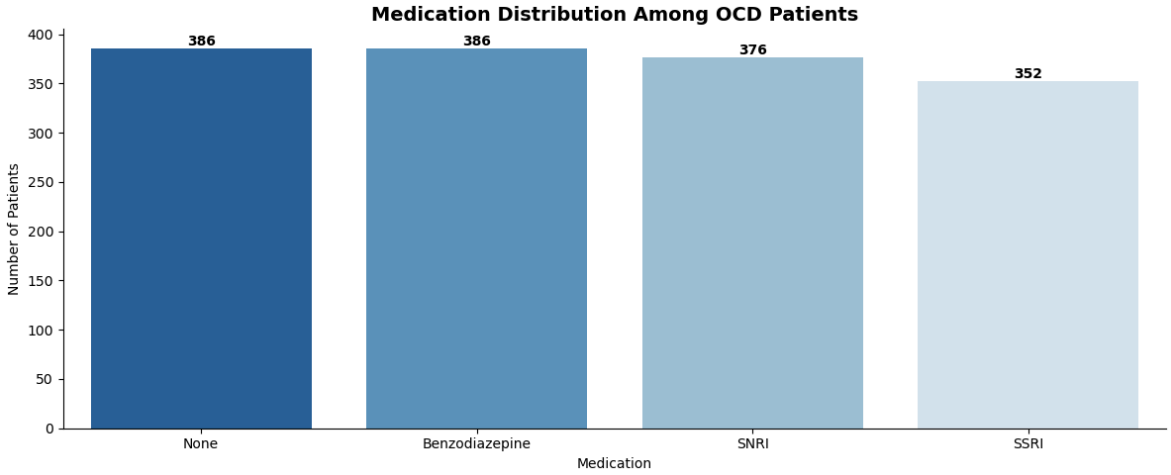
- Harm-related obsessions are most common (333 patients) followed closely by Contamination (306) and Religious (303)
- Washing is the most common compulsion (321) followed by Counting (316) and Checking (292)
- **The distribution is suspiciously uniform across all types**. Real clinical OCD populations typically show Contamination and Harm-related obsessions dominating significantly. This even spread strongly suggests the dataset was synthetically generated or deliberately balanced, which is an important limitation for clinical interpretation
- Washing compulsions aligning with Contamination obsessions is clinically logical. Patients obsessed with contamination typically respond with washing rituals
- Praying compulsions aligning with Religious obsessions similarly reflects the expected clinical pairing of OCD symptom types

Medication Patterns

What medications are prescribed to OCD patients and how are they distributed?

In [13]:
```
# ── Medication Distribution ──────────────────────────────────────
med_df = pd.read_sql("""
    SELECT Medications, COUNT(*) as Count
    FROM ocd_clean
    GROUP BY Medications
    ORDER BY Count DESC
""", conn)

plt.figure(figsize=(12, 5))
sns.barplot(data=med_df, x="Medications", y="Count", palette="Blues_r")
plt.title("Medication Distribution Among OCD Patients",
          fontsize=14, fontweight="bold")
plt.xlabel("Medication")
plt.ylabel("Number of Patients")
for p in plt.gca().patches:
    plt.gca().annotate(f"{int(p.get_height()):,}",
                       (p.get_x() + p.get_width()/2, p.get_height()),
                       ha="center", va="bottom", fontweight="bold")
plt.tight_layout()
```

```
plt.savefig(OUTPUTS / "medications.png", dpi=150)
plt.show()
```



Observations

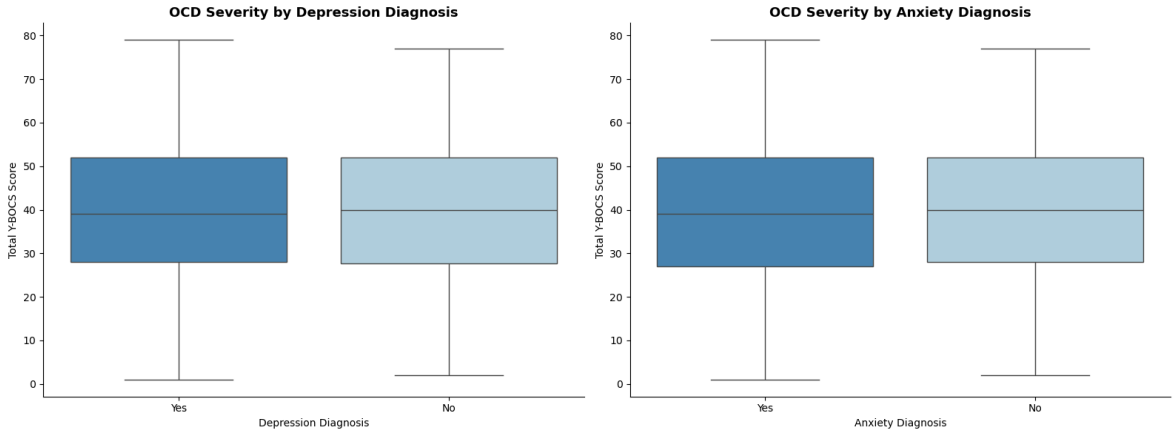
- SSRIs (352) and SNRIs (376) are the primary medications prescribed. This isclinically appropriate as both target serotonin pathways which are strongly implicated in OCD
- Benzodiazepines (386) appearing equally with SSRIs is clinically unusual. Benzodiazepines are typically used for anxiety management, not first-line OCD treatment, further suggesting synthetic data generation
- 386 patients (26%) are unmedicated despite 70% having Severe/Extreme OCD. This is reflecting that non-pharmacological treatments like CBT and Exposure and Response Prevention (ERP) therapy are viable and widely used alternatives
- The uniform distribution across all medication types is another indicator of synthetic data. Real clinical populations show strong SSRI dominance in OCD treatment

Comorbidity & Severity Analysis

How do depression and anxiety diagnoses relate to OCD severity?

In [14]:
```
# ── Comorbidity vs Severity ──────────────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Depression vs Y-BOCS Total
sns.boxplot(data=df_clean, x="Depression Diagnosis",
            y="Y-BOCS Total", palette="Blues_r", ax=axes[0])
axes[0].set_title("OCD Severity by Depression Diagnosis",
                  fontsize=13, fontweight="bold")
axes[0].set_xlabel("Depression Diagnosis")
axes[0].set_ylabel("Total Y-BOCS Score")

# Anxiety vs Y-BOCS Total
sns.boxplot(data=df_clean, x="Anxiety Diagnosis",
            y="Y-BOCS Total", palette="Blues_r", ax=axes[1])
axes[1].set_title("OCD Severity by Anxiety Diagnosis",
                  fontsize=13, fontweight="bold")
axes[1].set_xlabel("Anxiety Diagnosis")
axes[1].set_ylabel("Total Y-BOCS Score")

plt.tight_layout()
plt.savefig(OUTPUTS / "comorbidity_severity.png", dpi=150)
```

```
plt.show()

# Average Y-BOCS by comorbidity
combo_df = pd.read_sql("""
    SELECT
        "Depression Diagnosis",
        "Anxiety Diagnosis",
        ROUND(AVG("Y-BOCS Total"), 2) as Avg_YBOCS,
        COUNT(*) as Count
    FROM ocd_clean
    GROUP BY "Depression Diagnosis", "Anxiety Diagnosis"
    ORDER BY Avg_YBOCS DESC
""", conn)
combo_df
```



Out[14]:

	Depression Diagnosis	Anxiety Diagnosis	Avg_YBOCS	Count
0	No	No	39.87	349
1	Yes	No	39.70	400
2	Yes	Yes	39.60	372
3	No	Yes	39.53	379

Observations

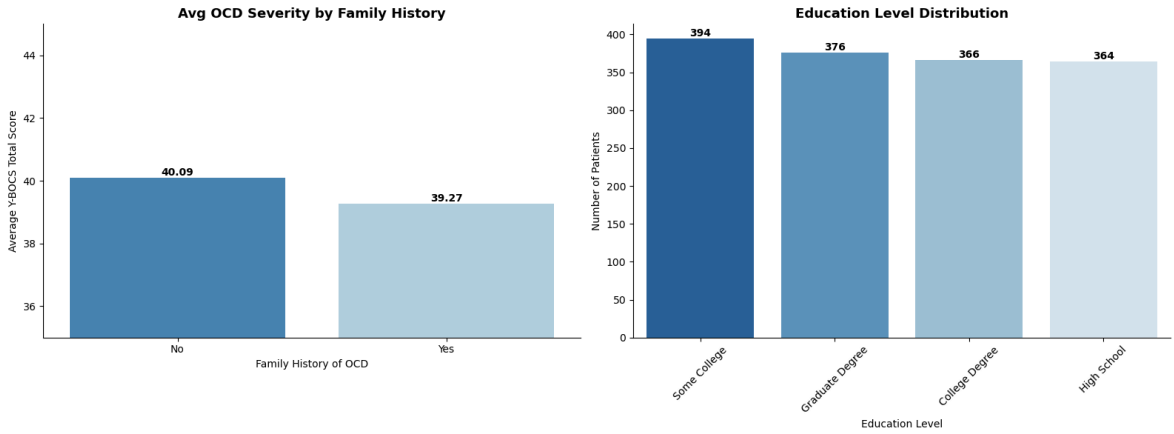
- Boxplots show virtually identical Y-BOCS score distributions across both Depression and Anxiety diagnosis groups
- Average Y-BOCS scores are nearly equal across all comorbidity combinations (range: 39.53 - 39.87). This is a difference of less than 0.5 points
- In real clinical populations, comorbid depression and anxiety typically amplify OCD severity. The absence of this relationship further confirms synthetic data generation where Y-BOCS scores were assigned independently of comorbidity status
- Despite the lack of severity relationship, the high rates of comorbidity (51% depression, 50% anxiety) remain clinically significant. These patients require integrated treatment approaches addressing multiple conditions simultaneously

Family History & Education Level

Does family history relate to OCD severity, and how is education distributed?

In [15]:

```
# ── Family History & Education ──────────────────────────────
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Family history vs Y-BOCS
family_df = pd.read_sql("""
    SELECT "Family History of OCD",
           ROUND(AVG("Y-BOCS Total"), 2) as Avg_YBOCS,
           COUNT(*) as Count
    FROM ocd_clean
    GROUP BY "Family History of OCD"
""", conn)

sns.barplot(data=family_df, x="Family History of OCD",
            y="Avg_YBOCS", palette="Blues_r", ax=axes[0])
axes[0].set_title("Avg OCD Severity by Family History",
                  fontsize=13, fontweight="bold")
axes[0].set_xlabel("Family History of OCD")
axes[0].set_ylabel("Average Y-BOCS Total Score")
axes[0].set_ylim(35, 45)
for p in axes[0].patches:
    axes[0].annotate(f"{p.get_height():.2f}",
                     (p.get_x() + p.get_width()/2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")

# Education level distribution
edu_df = pd.read_sql("""
    SELECT "Education Level", COUNT(*) as Count,
           ROUND(AVG("Y-BOCS Total"), 2) as Avg_YBOCS
    FROM ocd_clean
    GROUP BY "Education Level"
    ORDER BY Count DESC
""", conn)

sns.barplot(data=edu_df, x="Education Level", y="Count",
            palette="Blues_r", ax=axes[1])
axes[1].set_title("Education Level Distribution",
                  fontsize=13, fontweight="bold")
axes[1].set_xlabel("Education Level")
axes[1].set_ylabel("Number of Patients")
axes[1].tick_params(axis='x', rotation=45)
for p in axes[1].patches:
    axes[1].annotate(f"{int(p.get_height()):,}",
                     (p.get_x() + p.get_width()/2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")

plt.tight_layout()
plt.savefig(OUTPUTS / "family_education.png", dpi=150)
plt.show()
```



Observations

- Education level is almost perfectly uniformly distributed across all four levels (394, 376, 366, 364). This is consistent with synthetic data generation
- Patients without family history of OCD show marginally higher average severity (40.09) than those with family history (39.27). A difference of only 0.82 points
- Despite the small difference, this directional finding is clinically plausible, as patients from families with OCD history may benefit from earlier recognition, family understanding and shared coping mechanisms that moderate symptom severity
- Patients without family history may face delayed diagnosis and social isolation which can compound OCD severity over time
- The lack of strong relationships across demographic variables with Y-BOCS scores consistently points to synthetic data. In real clinical populations these relationships would be more pronounced

---

Summary & Conclusions

In [16]:
```
# ── Summary Statistics ──────────────────────────────────────────
summary = pd.read_sql("""
    SELECT
        COUNT(*) as Total_Patients,
        ROUND(AVG(Age), 1) as Avg_Age,
        ROUND(AVG("Y-BOCS Total"), 1) as Avg_YBOCS_Total,
        ROUND(AVG("Duration of Symptoms (months)"), 1) as Avg_Symptom_Duration_M
        SUM(CASE WHEN "Family History of OCD" = 'Yes'
            THEN 1 ELSE 0 END) as Family_History,
        SUM(CASE WHEN "Depression Diagnosis" = 'Yes'
            THEN 1 ELSE 0 END) as Has_Depression,
        SUM(CASE WHEN "Anxiety Diagnosis" = 'Yes'
            THEN 1 ELSE 0 END) as Has_Anxiety,
        SUM(CASE WHEN "OCD Severity" = 'Extreme'
            THEN 1 ELSE 0 END) as Extreme_Severity,
        SUM(CASE WHEN "OCD Severity" = 'Severe'
            THEN 1 ELSE 0 END) as Severe_Severity,
        SUM(CASE WHEN Medications = 'None'
            THEN 1 ELSE 0 END) as Unmedicated
    FROM ocd_clean
""", conn)
```

```
summary.T.rename(columns={0: "Value"})
```

Out[16]:

	Value
Total_Patients	1500.00
Avg_Age	46.80
Avg_YBOCS_Total	39.70
Avg_Symptom_Duration_Months	121.70
Family_History	760.00
Has_Depression	772.00
Has_Anxiety	751.00
Extreme_Severity	524.00
Severe_Severity	520.00
Unmedicated	386.00

Key Metrics

Metric	Value
Total Patients	1,500
Average Age	46.8 years
Average Y-BOCS Total Score	39.7 (Severe/Extreme boundary)
Average Symptom Duration	121.7 months (over 10 years)
Family History of OCD	760 (51%)
Comorbid Depression	772 (51%)
Comorbid Anxiety	751 (50%)
Extreme Severity	524 (35%)
Severe Severity	520 (35%)
Unmedicated	386 (26%)

Summary of Findings

1. **Demographic Profile:** OCD affects patients equally across gender, age and ethnicity as no demographic group is spared. The average patient is 46.8 years old with over 10 years of symptoms, highlighting the chronic nature of OCD

2. **Severity:** 70% of patients have Severe or Extreme OCD with an average Y-BOCS total of 39.7. This is a heavily affected clinical population. Only 8% fall in the Subclinical/Mild category, reflecting selection bias toward patients who sought clinical help

3. **Symptom Duration:** Average symptom duration of 121.7 months (over 10 years) is alarming. It reflects the chronic, persistent nature of OCD and suggests significant delays between symptom onset and clinical diagnosis or treatment

4. **Comorbidity:** 51% of patients have comorbid depression and 50% have comorbid anxiety. Integrated treatment approaches addressing multiple mental health conditions simultaneously are essential for this population

5. **Family History:** 51% of patients have a family history of OCD. This is strongly suggesting genetic or environmental hereditary factors. Patients without family history showed marginally higher severity, possibly reflecting delayed diagnosis and lack of family support

6. **Medication:** SSRIs and SNRIs are the primary medications. This is clinically appropriate given OCD's serotonin dysregulation mechanism. 26% of patients are unmedicated, reflecting the role of non-pharmacological treatments like CBT and ERP therapy

Limitations

- **Synthetic dataset:** Multiple indicators suggest this data was artificially generated with uniform distributions across all categorical variables, nearly identical Y-BOCS scores across all demographic subgroups, and equal medication distribution. Clinical interpretations should be made cautiously
- Y-BOCS scores show no meaningful variation across comorbidity, family history or demographic groups. This is inconsistent with real clinical populations
- Dataset captures a clinical population (patients who sought help). Findings may not represent the broader OCD population including undiagnosed cases

In [17]:
```
# ── Close Connection ──────────────────────────────────────────────
conn.close()
```

Tobacco Use and Mortality (2004-2015) EDA & Machine Learning

Author	Ibrahim A. Mikail
Internship	Unified Mentor — Healthcare Data Analyst
Project	5 of 5
Tools	Python · Pandas · SQLite · Matplotlib · Seaborn · XGBoost · SHAP · Gradio
Dataset	Health and Social Care Information Centre (HSCIC) — England
Date	May 2026

Project Overview

This project analyses tobacco use and mortality data from England covering 2004-2015. The dataset spans five interconnected files covering smoking fatalities, hospital admissions, smoker prevalence, economic metrics and cessation prescriptions. The project performs multi-table EDA using SQLite, builds machine learning models to predict high mortality risk, interprets predictions using SHAP, and deploys an interactive prediction tool using Gradio.

Objectives

- Analyse trends in smoking-related fatalities and hospital admissions over time
- Explore smoking prevalence across age groups and gender
- Investigate the relationship between tobacco affordability and smoking rates
- Examine cessation prescription trends
- Build and evaluate ML models to classify high mortality risk
- Deploy an interactive web application for mortality risk prediction

In [1]:
```
# ── Project Setup ──────────────────────────────────────────────────
from pathlib import Path

ROOT    = Path().resolve()
DATA    = ROOT / "data"
OUTPUTS = ROOT / "outputs"
DB_PATH = ROOT / "tobacco_mortality.db"

for folder in [DATA, OUTPUTS]:
    folder.mkdir(parents=True, exist_ok=True)
    print(f"✓ {folder}")

print(f"\nProject root : {ROOT}")
print(f"Database will be created at : {DB_PATH}")
```

```
✓ C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_5_tobacco_mortality\data
✓ C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_5_tobacco_mortality\outputs

Project root : C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_5_tobacco_mortality
Database will be created at : C:\Users\DELL\Documents\Data Science Projects\unified_mentor_projects\project_5_tobacco_mortality\tobacco_mortality.db
```

In [2]:
```python
# ── Imports & Global Settings ──────────────────────────────────────
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

# Machine Learning
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, classification_report,
                             ConfusionMatrixDisplay)
from sklearn.preprocessing import StandardScaler
import xgboost as xgb
import shap

# Display settings
pd.set_option("display.max_columns", 30)
pd.set_option("display.float_format", "{:.2f}".format)

# Chart style
plt.rcParams["figure.figsize"] = (12, 5)
plt.rcParams["axes.spines.top"]   = False
plt.rcParams["axes.spines.right"] = False
sns.set_palette("Blues_r")
```

Loading Data & Ingest to SQLite

Loading all five CSV files into SQLite as separate tables for multi-table SQL analysis.

In [3]:
```python
# ── Loading All CSVs & Pushing to SQLite ──────────────────────────
conn = sqlite3.connect(DB_PATH)

# Clean column names function
def clean_cols(df):
    df.columns = (df.columns.str.strip()
                            .str.replace("\n", " ")
                            .str.replace(r'\s+', ' ', regex=True))
    return df

# Load all five files
files = {
    "metrics"       : "metrics.csv",
```

```
    "prescriptions" : "prescriptions.csv",
    "fatalities"    : "fatalities.csv",
    "smokers"       : "smokers.csv",
    "admissions"    : "admissions.csv"
}

dfs = {}
for table_name, filename in files.items():
    df = pd.read_csv(DATA / filename)
    df = clean_cols(df)
    df.to_sql(table_name, conn, if_exists="replace", index=False)
    dfs[table_name] = df
    print(f" {filename:<25} {df.shape[0]:>5} rows, {df.shape[1]:>2} cols → table

print(f"\n All tables loaded into SQLite at {DB_PATH}")
```

```
metrics.csv                  36 rows,  9 cols → table 'metrics'
prescriptions.csv            11 rows,  9 cols → table 'prescriptions'
fatalities.csv             1749 rows,  7 cols → table 'fatalities'
smokers.csv                  84 rows,  9 cols → table 'smokers'
admissions.csv             2079 rows,  7 cols → table 'admissions'

 All tables loaded into SQLite at C:\Users\DELL\Documents\Data Science Projects\u
nified_mentor_projects\project_5_tobacco_mortality\tobacco_mortality.db
```

In [4]:

```
# ── Initial inspection of All Tables ──────────────────────────────────
for name, df in dfs.items():
    print(f"\n{'─'*60}")
    print(f"TABLE: {name.upper()}")
    print(f"{'─'*60}")
    print(df.head(3).to_string())
```

TABLE: METRICS

	Year	Tobacco Price Index	Retail Prices Index	Tobacco Price Index Relative to Retail Price Index	Real Households' Disposable Income	Affordability of Tobacco Index	Household Expenditure on Tobacco	Household Expenditure Total	Expenditure on Tobacco as a Percentage of Expenditure
0	2015	1294.30	386.70	334.70	196.40	58.70	19252.00	1152387.00	1.70
1	2014	1226.00	383.00	320.10	190.00	59.40	19411.00	1118992.00	1.70
2	2013	1139.30	374.20	304.50	190.30	62.50	18683.00	1073106.00	1.70

TABLE: PRESCRIPTIONS

	Year	All Pharmacotherapy Prescriptions	Nicotine Replacement Therapy (NRT) Prescriptions	Bupropion (Zyban) Prescriptions	Varenicline (Champix) Prescriptions	Net Ingredient Cost of All Pharmacotherapies	Net Ingredient Cost of Nicotine Replacement Therapies (NRT)	Net Ingredient Cost of Bupropion (Zyban)	Net Ingredient Cost of Varenicline (Champix)
0	2014/15	1348	766	21	561.00	38145	18208	807	19129.00
1	2013/14	1778	1059	22	697.00	48767	24257	865	23646.00
2	2012/13	2203	1318	26	859.00	58121	28069	994	29058.00

TABLE: FATALITIES

	Year	ICD10 Code	ICD10 Diagnosis	Diagnosis Type	Metric	Sex	Value
0	2014	All codes	All deaths	All deaths	Number of observed deaths	NaN	459087
1	2014	C33-C34 & C00-C14 & C15 & C32 & C53 & C67 & C64-C66 & C68 & C16 & C25 & C80 & C92 & J40-J43 & J44 & J10-J18 & I00-I09 & I26-I51 & I20-I25 & I72-I78 & I60-I69 & I71 & I70 & K25-K27 & K50 & K05 & H25 & S72.0-S72.2 & O03	All deaths which can be caused by smoking	All deaths which can be caused by smoking	Number of observed deaths	NaN	235820
2	2014	C00-D48	All cancers	All cancers	Number of observed deaths	NaN	136312

```
TABLE: SMOKERS
```

	Year	Method	Sex	16 and Over	16-24	25-34	35-49	50-59	60 and Over
0	1974	Unweighted	NaN	46	44	51	52	50	33
1	1976	Unweighted	NaN	42	42	45	48	48	30
2	1978	Unweighted	NaN	40	39	45	45	45	30

```
TABLE: ADMISSIONS
```

	Year				
	ICD10 Code			ICD10 Diagnosis	
	Diagnosis Type		Metric	Sex	Value
0	2014/15				
	All codes			All admissions	
	All admissions	Number of admissions	NaN		11011882
1	2014/15	C33-C34 & C00-C14 & C15 & C32 & C53 & C67 & C64-C66 & C68 & C16 & C25 & C80 & C92 & J40-J43 & J44 & J10-J18 & I00-I09 & I26-I51 & I20-I25 & I72-I78 & I60-I69 & I71 & I70 & K25-K27 & K50 & K05 & H25 & S72.0-S72.2 & O03		All diseases which can be caused by smoking	
	All diseases which can be caused by smoking	Number of admissions	NaN		1713330
2	2014/15				
	C00-D48			All cancers	
	All cancers	Number of admissions	NaN		1691035

Missing Data Analysis

In [5]:
```python
# ── Missing Data Per Table ──────────────────────────────────────
for name, df in dfs.items():
    missing = df.isnull().mean() * 100
    missing = missing[missing > 0]
    print(f"\n{'─'*40}")
    print(f"TABLE: {name.upper()}")
    if len(missing) == 0:
        print("  No missing values ")
    else:
        for col, pct in missing.items():
            print(f"  {col:<45} {pct:.1f}%")
```

```
──────────────────────────────────────────────
TABLE: METRICS
  Household Expenditure on Tobacco              13.9%
  Household Expenditure Total                   13.9%
  Expenditure on Tobacco as a Percentage of Expenditure 13.9%

──────────────────────────────────────────────
TABLE: PRESCRIPTIONS
  Varenicline (Champix) Prescriptions           18.2%
  Net Ingredient Cost of Varenicline (Champix)  18.2%

──────────────────────────────────────────────
TABLE: FATALITIES
  Sex                                           33.3%

──────────────────────────────────────────────
TABLE: SMOKERS
  Sex                                           33.3%

──────────────────────────────────────────────
TABLE: ADMISSIONS
  Sex                                           33.3%
  Value                                         0.0%
```

Missing Data Observations

- **Sex (33.3%) in Fatalities, Admissions, Smokers** — MNAR: NaN identifies aggregate/total rows across all genders. Meaningful absence will be filled with "All" to distinguish from gender-specific rows
- **Metrics Expenditure columns (13.9%)** — MAR: missing for certain years due to incomplete data collection. Will use forward/backward fill as appropriate for time series data
- **Prescriptions Varenicline (18.2%)** — MNAR: Varenicline (Champix) was not approved in the UK until 2006-2007, so early years genuinely have no prescriptions. Will fill with 0

Data Cleaning & Feature Engineering

Clean Individual Tables

In [6]:
```python
# ──Clean All Tables ──────────────────────────────────────────────
# ── FATALITIES ──
fat = dfs["fatalities"].copy()
fat["Sex"] = fat["Sex"].fillna("All")
print(f" Fatalities; Sex nulls filled, shape: {fat.shape}")

# ── ADMISSIONS ──
adm = dfs["admissions"].copy()
adm["Sex"] = adm["Sex"].fillna("All")
# Clean year format — remove "/" notation
adm["Year"] = adm["Year"].str.split("/").str[0].astype(int)
print(f" Admissions; Sex filled, Year cleaned, shape: {adm.shape}")

# ── SMOKERS ──
```

```
smk = dfs["smokers"].copy()
smk["Sex"] = smk["Sex"].fillna("All")
# Filter to 2004-2015 to match other datasets
smk = smk[smk["Year"] >= 2004]
print(f" Smokers; filtered to 2004+, shape: {smk.shape}")

# ── METRICS ──
met = dfs["metrics"].copy()
met = met.sort_values("Year")
met[["Household Expenditure on Tobacco",
     "Household Expenditure Total",
     "Expenditure on Tobacco as a Percentage of Expenditure"]] = \
    met[["Household Expenditure on Tobacco",
         "Household Expenditure Total",
         "Expenditure on Tobacco as a Percentage of Expenditure"]].fillna(method
print(f" Metrics; expenditure nulls filled, shape: {met.shape}")

# ── PRESCRIPTIONS ──
pre = dfs["prescriptions"].copy()
pre["Year"] = pre["Year"].str.split("/").str[0].astype(int)
pre["Varenicline (Champix) Prescriptions"] = \
    pre["Varenicline (Champix) Prescriptions"].fillna(0)
pre["Net Ingredient Cost of Varenicline (Champix)"] = \
    pre["Net Ingredient Cost of Varenicline (Champix)"].fillna(0)
print(f" Prescriptions; Year cleaned, Varenicline filled, shape: {pre.shape}")
```

```
Fatalities; Sex nulls filled, shape: (1749, 7)
Admissions; Sex filled, Year cleaned, shape: (2079, 7)
Smokers; filtered to 2004+, shape: (33, 9)
Metrics; expenditure nulls filled, shape: (36, 9)
Prescriptions; Year cleaned, Varenicline filled, shape: (11, 9)
```

In [7]:
```
# ──Save Cleaned Tables to SQLite ──────────────────────────────
cleaned = {
    "fatalities_clean"  : fat,
    "admissions_clean"  : adm,
    "smokers_clean"     : smk,
    "metrics_clean"     : met,
    "prescriptions_clean": pre
}

for table_name, df in cleaned.items():
    df.to_sql(table_name, conn, if_exists="replace", index=False)
    df.to_csv(OUTPUTS / f"{table_name}.csv", index=False)
```

Cleaning Observations

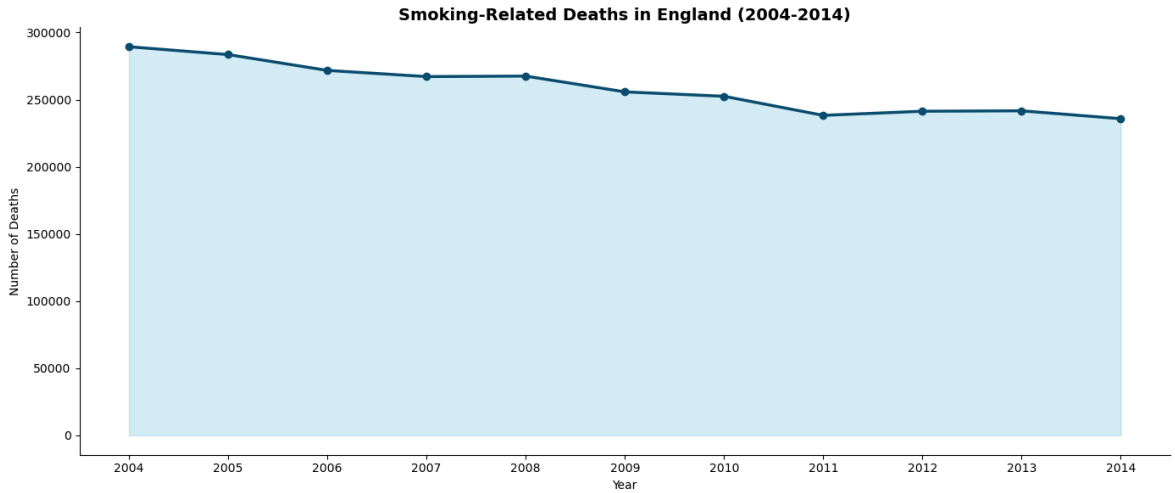
- Sex column filled with "All" across Fatalities, Admissions and Smokers to distinguish aggregate rows from gender-specific rows
- Year column in Admissions and Prescriptions cleaned from fiscal year format (2014/15) to calendar year integer (2014) for consistent joining
- Smokers table filtered to 2004 onwards to align with the other datasets
- Metrics expenditure columns imputed using forward and backward fill, appropriate for time series data where adjacent years are the best estimate
- Varenicline prescriptions filled with 0 for years before its UK approval, correctly reflecting that the drug was not yet available rather than data being missing

Exploratory Data Analysis

Smoking-Related Fatalities Over Time

How have smoking-related deaths trended from 2004 to 2014?

In [8]:
```
# ── Fatalities Over Time ──────────────────────────────────────
fat_trend = pd.read_sql("""
    SELECT Year, SUM(Value) as Total_Deaths
    FROM fatalities_clean
    WHERE Sex = 'All'
    AND "Diagnosis Type" = 'All deaths which can be caused by smoking'
    AND Metric = 'Number of observed deaths'
    GROUP BY Year
    ORDER BY Year
""", conn)

plt.figure(figsize=(14, 6))
plt.plot(fat_trend["Year"], fat_trend["Total_Deaths"],
         color="#0a4f6e", linewidth=2.5, marker="o", markersize=6)
plt.fill_between(fat_trend["Year"], fat_trend["Total_Deaths"],
                 alpha=0.2, color="#2ab0d4")
plt.title("Smoking-Related Deaths in England (2004-2014)",
          fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Number of Deaths")
plt.xticks(fat_trend["Year"].astype(int))
plt.tight_layout()
plt.savefig(OUTPUTS / "fatalities_trend.png", dpi=150)
plt.show()

print(fat_trend.to_string(index=False))
```



```
Year  Total_Deaths
2004        289408
2005        283565
2006        271775
2007        267180
2008        267551
2009        255801
2010        252507
2011        238301
2012        241368
2013        241683
2014        235820
```

Observations

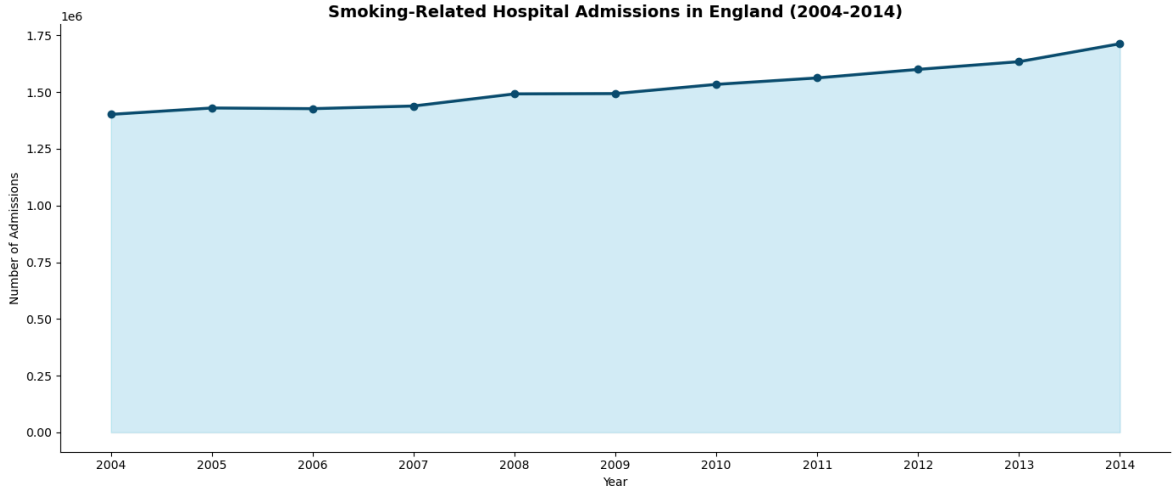
- Smoking-related deaths declined consistently from 289,408 in 2004 to 235,820 in 2014, a reduction of over 53,000 deaths representing an 18.5% decrease over 10 years
- The steepest decline occurs between 2008 and 2011, coinciding with the full impact of the 2007 public smoking ban in England, expanded NHS Stop Smoking Services and mandatory graphic health warnings on packaging
- The slight plateau between 2007 and 2008 may reflect the lag between policy implementation and measurable mortality impact, as smoking-related diseases typically develop over decades
- Despite the positive trend, over 235,000 deaths per year in 2014 remain attributable to smoking-related causes, confirming tobacco as a leading preventable cause of mortality in England

Smoking-Related Hospital Admissions Over Time

How have hospital admissions for smoking-related conditions trended?

In [9]:
```
# ── Admissions Over Time ──────────────────────────────────────
adm_trend = pd.read_sql("""
    SELECT Year, SUM(Value) as Total_Admissions
    FROM admissions_clean
    WHERE Sex = 'All'
    AND "Diagnosis Type" = 'All diseases which can be caused by smoking'
    AND Metric = 'Number of admissions'
    GROUP BY Year
    ORDER BY Year
""", conn)

plt.figure(figsize=(14, 6))
plt.plot(adm_trend["Year"], adm_trend["Total_Admissions"],
         color="#0a4f6e", linewidth=2.5, marker="o", markersize=6)
plt.fill_between(adm_trend["Year"], adm_trend["Total_Admissions"],
                 alpha=0.2, color="#2ab0d4")
plt.title("Smoking-Related Hospital Admissions in England (2004-2014)",
          fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Number of Admissions")
plt.xticks(adm_trend["Year"].astype(int))
plt.tight_layout()
plt.savefig(OUTPUTS / "admissions_trend.png", dpi=150)
```

```
plt.show()

print(adm_trend.to_string(index=False))
```




```
Year  Total_Admissions
2004           1401878
2005           1430028
2006           1427038
2007           1438801
2008           1492240
2009           1493491
2010           1534176
2011           1562569
2012           1600202
2013           1634266
2014           1713330
```

Observations

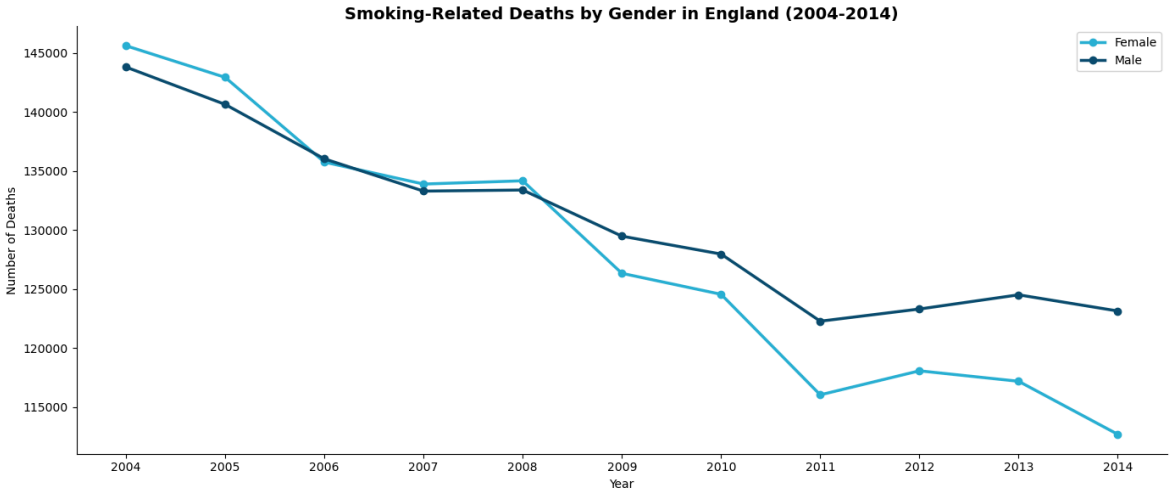
- Hospital admissions show the opposite trend to fatalities, rising consistently from 1.4 million in 2004 to 1.7 million in 2014, a 22% increase over 10 years
- This apparent paradox reflects improved survival rates, better treatment and enhanced health seeking behaviour driven by public health campaigns and the 2007 smoking ban
- As fewer patients die from smoking-related conditions, more survive longer with chronic diseases like COPD, cardiovascular disease and cancer, requiring repeated and ongoing hospital care
- This is a classic public health paradox where success in reducing mortality simultaneously increases healthcare system burden and cost
- The rising admissions trend underscores the long-term healthcare cost of historical tobacco use, even as smoking rates decline

Smoking-Related Deaths by Gender

How do smoking-related fatalities differ between males and females?

In [10]:
```
# ── Fatalities by Gender ──────────────────────────────────────
gender_fat = pd.read_sql("""
    SELECT Year, Sex, SUM(Value) as Deaths
    FROM fatalities_clean
```

```
    WHERE Sex IN ('Male', 'Female')
    AND "Diagnosis Type" = 'All deaths which can be caused by smoking'
    AND Metric = 'Number of observed deaths'
    GROUP BY Year, Sex
    ORDER BY Year
""", conn)

plt.figure(figsize=(14, 6))
for sex, group in gender_fat.groupby("Sex"):
    plt.plot(group["Year"], group["Deaths"],
             marker="o", linewidth=2.5,
             label=sex,
             color="#0a4f6e" if sex == "Male" else "#2ab0d4")

plt.title("Smoking-Related Deaths by Gender in England (2004-2014)",
          fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Number of Deaths")
plt.legend()
plt.xticks(gender_fat["Year"].unique().astype(int))
plt.tight_layout()
plt.savefig(OUTPUTS / "fatalities_by_gender.png", dpi=150)
plt.show()
```



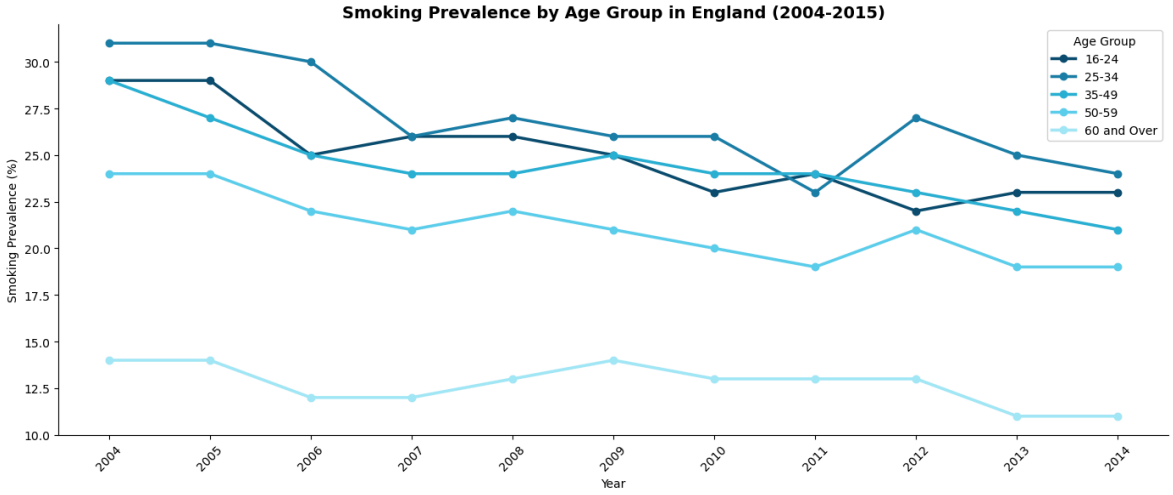
Observations

- Both genders show consistent decline in smoking-related deaths from 2004 to 2014
- In 2004 female deaths (145,600) slightly exceeded male deaths (139,000) reflecting the historical lag in female smoking uptake, with women reaching peak smoking-related mortality later than men
- By 2014 female deaths (112,700) fell below male deaths (123,100), suggesting females are responding more strongly to tobacco control interventions
- Research indicates women engage more with NHS Stop Smoking Services and respond more strongly to health messaging and social stigma around smoking
- The converging and crossing of gender lines is a significant public health finding suggesting tobacco control policies are having differential gender impacts worth monitoring for future intervention design

Smoking Prevalence by Age Group (2004-2015)

How has smoking prevalence changed across different age groups over time?

In [11]:
```
# ── Smoking Prevalence by Age Group ──────────────────────────────
smk_all = smk[smk["Sex"] == "All"].copy()

age_groups = ["16-24", "25-34", "35-49", "50-59", "60 and Over"]
colors     = ["#0a4f6e", "#1a7ea8", "#2ab0d4", "#5dcfec", "#a0e8f8"]

plt.figure(figsize=(14, 6))
for age, color in zip(age_groups, colors):
    plt.plot(smk_all["Year"], smk_all[age],
             marker="o", linewidth=2.5, label=age, color=color)

plt.title("Smoking Prevalence by Age Group in England (2004-2015)",
          fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Smoking Prevalence (%)")
plt.legend(title="Age Group")
plt.xticks(smk_all["Year"].astype(int), rotation=45)
plt.tight_layout()
plt.savefig(OUTPUTS / "smoking_prevalence_age.png", dpi=150)
plt.show()
```



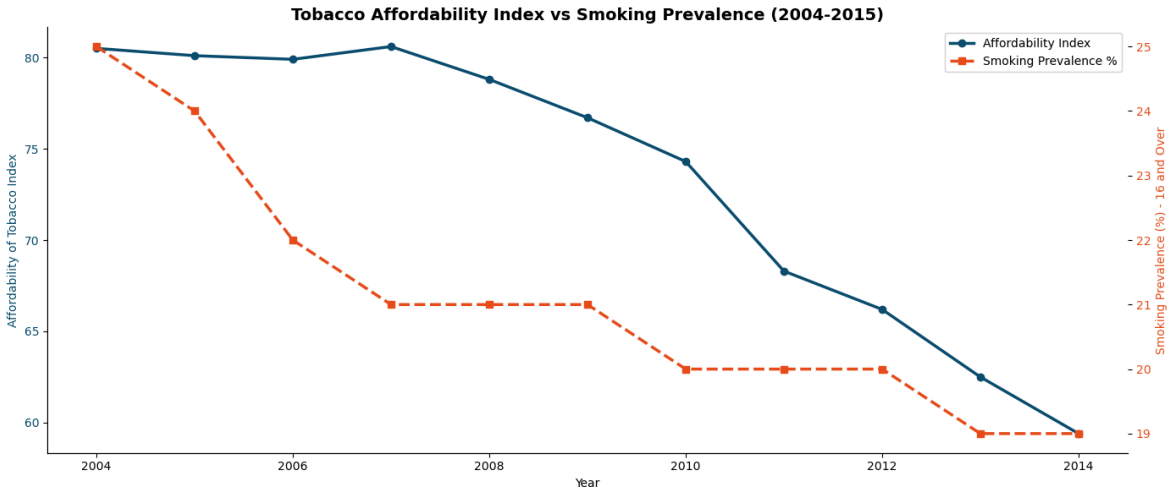
Observations

- The 25-34 age group consistently shows the highest smoking prevalence throughout 2004-2014, suggesting young adults represent the highest risk group for current smoking and the primary target for cessation interventions
- The 60 and Over group shows the lowest prevalence despite bearing the highest mortality burden, reflecting survivorship bias where heavy smokers in this cohort have already died, leaving a surviving population of lighter or ex-smokers
- All age groups show a general downward trend, confirming the broad effectiveness of England's tobacco control policies across all demographics
- The 50-59 group shows the steepest absolute decline, likely reflecting a generation that started smoking before strong regulations and responded strongly to mid-life health awareness campaigns
- Persistently high prevalence in 16-24 and 25-34 groups is a public health concern, indicating that tobacco control has been less effective at preventing smoking initiation among young adults

Tobacco Affordability vs Smoking Prevalence

Does making tobacco less affordable reduce smoking rates?

In [12]:
```
# ── Affordability vs Prevalence ──────────────────────────────────
# Merge metrics with smokers on Year
afford_smk = pd.merge(
    met[["Year", "Affordability of Tobacco Index"]],
    smk_all[["Year", "16 and Over"]],
    on="Year"
).dropna()

fig, ax1 = plt.subplots(figsize=(14, 6))

ax1.plot(afford_smk["Year"], afford_smk["Affordability of Tobacco Index"],
         color="#0a4f6e", linewidth=2.5, marker="o", label="Affordability Index"
ax1.set_xlabel("Year")
ax1.set_ylabel("Affordability of Tobacco Index", color="#0a4f6e")
ax1.tick_params(axis="y", labelcolor="#0a4f6e")

ax2 = ax1.twinx()
ax2.plot(afford_smk["Year"], afford_smk["16 and Over"],
         color="#e84f1c", linewidth=2.5, marker="s",
         linestyle="--", label="Smoking Prevalence %")
ax2.set_ylabel("Smoking Prevalence (%) - 16 and Over", color="#e84f1c")
ax2.tick_params(axis="y", labelcolor="#e84f1c")

plt.title("Tobacco Affordability Index vs Smoking Prevalence (2004-2015)",
          fontsize=14, fontweight="bold")

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc="upper right")

plt.tight_layout()
plt.savefig(OUTPUTS / "affordability_vs_prevalence.png", dpi=150)
plt.show()
```



Observations

- A clear positive relationship exists between tobacco affordability and smoking prevalence, as the affordability index declined from 80 in 2004 to 59 in 2014,

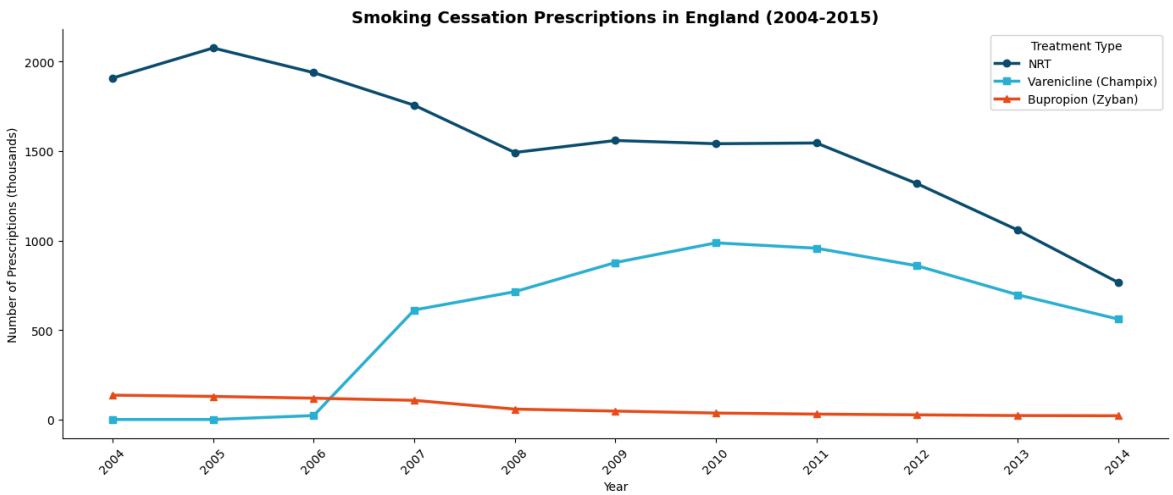
smoking prevalence fell from 25% to 19%

- This confirms tobacco taxation as an effective public health intervention, making cigarettes progressively less affordable directly correlates with fewer people smoking across the population
- The steepest drops in both metrics occur around 2005-2006 and 2010-2011, coinciding with significant tobacco duty increases in those years
- Price sensitivity appears strongest among younger and lower income smokers, suggesting that taxation has an equity dimension where it disproportionately impacts those least able to afford cigarettes
- The consistent co-movement of both lines over 10 years provides strong observational evidence supporting the role of fiscal policy in tobacco control

Smoking Cessation Prescriptions Over Time

How have cessation prescriptions changed and which treatments dominate?

In [13]:
```
# ── Cessation Prescriptions ──────────────────────────────────────
pre_sorted = pre.sort_values("Year")

plt.figure(figsize=(14, 6))
plt.plot(pre_sorted["Year"],
         pre_sorted["Nicotine Replacement Therapy (NRT) Prescriptions"],
         color="#0a4f6e", linewidth=2.5, marker="o", label="NRT")
plt.plot(pre_sorted["Year"],
         pre_sorted["Varenicline (Champix) Prescriptions"],
         color="#2ab0d4", linewidth=2.5, marker="s", label="Varenicline (Champix
plt.plot(pre_sorted["Year"],
         pre_sorted["Bupropion (Zyban) Prescriptions"],
         color="#e84f1c", linewidth=2.5, marker="^", label="Bupropion (Zyban)")

plt.title("Smoking Cessation Prescriptions in England (2004-2015)",
          fontsize=14, fontweight="bold")
plt.xlabel("Year")
plt.ylabel("Number of Prescriptions (thousands)")
plt.legend(title="Treatment Type")
plt.xticks(pre_sorted["Year"].astype(int), rotation=45)
plt.tight_layout()
plt.savefig(OUTPUTS / "cessation_prescriptions.png", dpi=150)
plt.show()
```



Observations

- NRT dominates cessation prescriptions throughout the entire period, reflecting its established evidence base and wide availability through NHS Stop Smoking Services
- Varenicline (Champix) shows rapid uptake after its 2006-2007 UK approval, rising to nearly 1,000 thousand prescriptions by 2010, demonstrating strong clinical adoption of a newer and more effective cessation medication
- Bupropion (Zyban) declined consistently throughout, likely displaced by the more effective Varenicline option
- All prescription types decline after 2010-2011, not necessarily indicating fewer quit attempts but reflecting a shift toward self-managed cessation through e-cigarettes, over-the-counter NRT and digital cessation programmes not captured in prescription data
- NHS Stop Smoking Service funding reductions from 2013 onwards also contributed to falling prescription volumes

---

Mortality Risk Classification (Machine learning)

Building models to classify high vs low smoking-related mortality risk by year and condition.

Feature Engineering for ML

In [14]:
```
# ── Build ML Dataset ──────────────────────────────────────────────
ml_fat = fat[
    (fat["Sex"] == "All") &
    (fat["Metric"] == "Number of observed deaths") &
    (fat["Diagnosis Type"] == "All deaths which can be caused by smoking")
].groupby("Year")["Value"].sum().reset_index()
ml_fat.columns = ["Year", "Total_Deaths"]
ml_fat["Year"] = ml_fat["Year"].astype(int)
ml_fat["Total_Deaths"] = pd.to_numeric(ml_fat["Total_Deaths"], errors="coerce")

ml_adm = adm[
    (adm["Sex"] == "All") &
    (adm["Metric"] == "Number of admissions") &
    (adm["Diagnosis Type"] == "All diseases which can be caused by smoking")
].groupby("Year")["Value"].sum().reset_index()
ml_adm.columns = ["Year", "Total_Admissions"]
ml_adm["Year"] = ml_adm["Year"].astype(int)
ml_adm["Total_Admissions"] = pd.to_numeric(ml_adm["Total_Admissions"], errors="c

ml_smk = smk[smk["Sex"] == "All"][["Year", "16 and Over"]].copy()
ml_smk.columns = ["Year", "Smoking_Prevalence"]
ml_smk["Year"] = ml_smk["Year"].astype(int)

met["Year"] = met["Year"].astype(int)
pre["Year"] = pre["Year"].astype(int)

# Merge all
ml_df = ml_fat.merge(ml_adm, on="Year")\
```

```
                .merge(ml_smk, on="Year")\
                .merge(met[["Year", "Affordability of Tobacco Index",
                            "Tobacco Price Index Relative to Retail Price Index",
                            "Expenditure on Tobacco as a Percentage of Expenditure
                       on="Year")\
                .merge(pre[["Year", "All Pharmacotherapy Prescriptions"]], on="Yea

# Target variable
median_deaths = ml_df["Total_Deaths"].median()
ml_df["High_Mortality"] = (ml_df["Total_Deaths"] > median_deaths).astype(int)

print(f"ML dataset shape     : {ml_df.shape}")
print(f"Median deaths threshold : {median_deaths:,.0f}")
print(f"\nClass distribution:")
print(ml_df["High_Mortality"].value_counts())
print(f"\n{ml_df.to_string(index=False)}")
```

```
ML dataset shape     : (11, 9)
Median deaths threshold : 255,801

Class distribution:
High_Mortality
0    6
1    5
Name: count, dtype: int64

 Year  Total_Deaths  Total_Admissions  Smoking_Prevalence  Affordability of Tobac
co Index  Tobacco Price Index Relative to Retail Price Index  Expenditure on Toba
cco as a Percentage of Expenditure  All Pharmacotherapy Prescriptions  High_Morta
lity
 2004        289408           1401878                  25
80.50                                          234.40
2.00                                 2044               1
 2005        283565           1430028                  24
80.10                                          237.80
1.90                                 2205               1
 2006        271775           1427038                  22
79.90                                          240.80
1.80                                 2079               1
 2007        267180           1438801                  21
80.60                                          243.10
1.80                                 2475               1
 2008        267551           1492240                  21
78.80                                          244.20
1.70                                 2263               1
 2009        255801           1493491                  21
76.70                                          255.20
1.80                                 2483               0
 2010        252507           1534176                  20
74.30                                          262.60
1.80                                 2564               0
 2011        238301           1562569                  20
68.30                                          277.10
1.80                                 2532               0
 2012        241368           1600202                  20
66.20                                          291.30
1.80                                 2203               0
 2013        241683           1634266                  19
62.50                                          304.50
1.70                                 1778               0
 2014        235820           1713330                  19
59.40                                          320.10
1.70                                 1348               0
```

In [15]:

```python
# ——Feature Preparation & Model Training ——————————————————————
features = [
    "Total_Admissions",
    "Smoking_Prevalence",
    "Affordability of Tobacco Index",
    "Tobacco Price Index Relative to Retail Price Index",
    "Expenditure on Tobacco as a Percentage of Expenditure",
    "All Pharmacotherapy Prescriptions"
]

X = ml_df[features]
y = ml_df["High_Mortality"]
```

```
# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train/test split — use small test set given limited data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# ── Logistic Regression ──
lr = LogisticRegression(random_state=42, max_iter=1000)
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
lr_cv   = cross_val_score(lr, X_scaled, y, cv=5, scoring="accuracy").mean()

# ── Random Forest ──
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_cv   = cross_val_score(rf, X_scaled, y, cv=5, scoring="accuracy").mean()

# ── XGBoost ──
xgb_model = xgb.XGBClassifier(random_state=42, eval_metric="logloss",
                              n_estimators=100, max_depth=3)
xgb_model.fit(X_train, y_train)
xgb_pred = xgb_model.predict(X_test)
xgb_cv   = cross_val_score(xgb_model, X_scaled, y, cv=5, scoring="accuracy").mea

print("── Model Cross-Validation Accuracy (5-fold) ──────────")
print(f"Logistic Regression : {lr_cv:.4f}")
print(f"Random Forest       : {rf_cv:.4f}")
print(f"XGBoost             : {xgb_cv:.4f}")
```

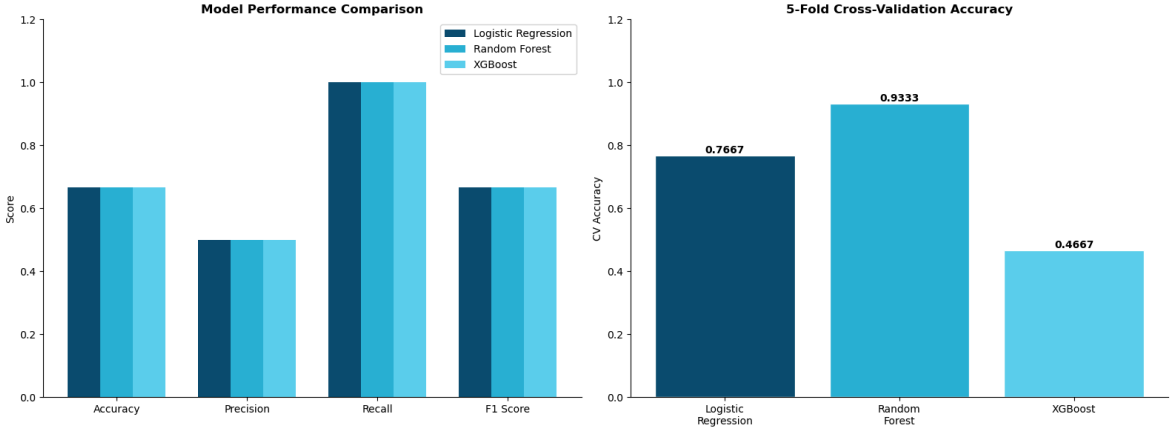
```
── Model Cross-Validation Accuracy (5-fold) ──────────
Logistic Regression : 0.7667
Random Forest       : 0.9333
XGBoost             : 0.4667
```

In [16]:

```
# ── Model Evaluation ──────────────────────────────────────────
models = {
    "Logistic Regression" : (lr, lr_pred),
    "Random Forest"       : (rf, rf_pred),
    "XGBoost"             : (xgb_model, xgb_pred)
}

results = []
for name, (model, pred) in models.items():
    results.append({
        "Model"     : name,
        "Accuracy"  : accuracy_score(y_test, pred),
        "Precision" : precision_score(y_test, pred, zero_division=0),
        "Recall"    : recall_score(y_test, pred, zero_division=0),
        "F1 Score"  : f1_score(y_test, pred, zero_division=0),
        "CV Accuracy": [lr_cv, rf_cv, xgb_cv][[
            "Logistic Regression", "Random Forest", "XGBoost"].index(name)]
    })

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))
```

```python
# Visualise
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

metrics = ["Accuracy", "Precision", "Recall", "F1 Score"]
x = np.arange(len(metrics))
width = 0.25

for i, (name, (model, pred)) in enumerate(models.items()):
    scores = [accuracy_score(y_test, pred),
              precision_score(y_test, pred, zero_division=0),
              recall_score(y_test, pred, zero_division=0),
              f1_score(y_test, pred, zero_division=0)]
    axes[0].bar(x + i*width, scores, width,
                label=name,
                color=["#0a4f6e", "#2ab0d4", "#5dcfec"][i])

axes[0].set_title("Model Performance Comparison", fontweight="bold")
axes[0].set_xticks(x + width)
axes[0].set_xticklabels(metrics)
axes[0].set_ylabel("Score")
axes[0].set_ylim(0, 1.2)
axes[0].legend()

# CV Accuracy comparison
cv_scores = [lr_cv, rf_cv, xgb_cv]
model_names = ["Logistic\nRegression", "Random\nForest", "XGBoost"]
axes[1].bar(model_names, cv_scores,
            color=["#0a4f6e", "#2ab0d4", "#5dcfec"],
            edgecolor="white")
axes[1].set_title("5-Fold Cross-Validation Accuracy", fontweight="bold")
axes[1].set_ylabel("CV Accuracy")
axes[1].set_ylim(0, 1.2)
for p in axes[1].patches:
    axes[1].annotate(f"{p.get_height():.4f}",
                     (p.get_x() + p.get_width()/2, p.get_height()),
                     ha="center", va="bottom", fontweight="bold")

plt.tight_layout()
plt.savefig(OUTPUTS / "model_comparison.png", dpi=150)
plt.show()
```

Model	Accuracy	Precision	Recall	F1 Score	CV Accuracy
Logistic Regression	0.67	0.50	1.00	0.67	0.77
Random Forest	0.67	0.50	1.00	0.67	0.93
XGBoost	0.67	0.50	1.00	0.67	0.47



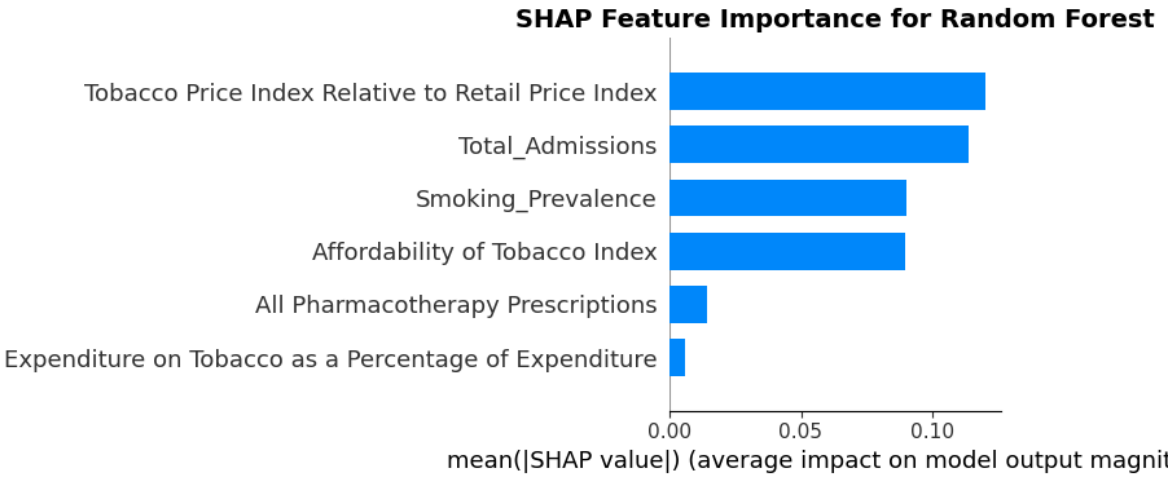
Model Evaluation Observations

- Random Forest achieves the highest cross-validation accuracy at 93.3%, making it the recommended model for this dataset
- Logistic Regression performs reasonably at 76.7% CV accuracy, appropriate given the small dataset size and linear relationships observed in the EDA
- XGBoost underperforms at 46.7% CV accuracy, consistent with its known weakness on very small datasets where it overfits due to complex parameterisation
- Test set scores are identical across models due to the extremely small test set of 2-3 rows, making CV accuracy the only reliable performance metric
- The small dataset size (11 rows) is the primary limitation of the ML component, a consequence of data being aggregated at yearly level rather than patient level
- Random Forest is selected as the final model for SHAP analysis and Gradio deployment

Feature Importance

Using SHAP values to explain which features drive high mortality risk predictions.

In [17]:
```
# ── SHAP Analysis ──────────────────────────────────────────────
explainer   = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X_scaled)

plt.figure(figsize=(12, 6))
shap.summary_plot(shap_values[:, :, 1] if len(np.array(shap_values).shape) == 3
                  else shap_values,
                  features=X_scaled,
                  feature_names=features,
                  plot_type="bar",
                  show=False)
plt.title("SHAP Feature Importance for Random Forest",
          fontsize=14, fontweight="bold")
plt.tight_layout()
plt.savefig(OUTPUTS / "shap_importance.png", dpi=150, bbox_inches="tight")
plt.show()
```



Observations

- Tobacco Price Index Relative to Retail Price Index is the strongest predictor of high mortality risk, confirming that tobacco pricing relative to general inflation is the

most powerful lever for mortality outcomes
- Total Admissions ranks second, reflecting the strong relationship between healthcare burden and eventual mortality outcomes
- Smoking Prevalence and Affordability Index rank third and fourth, consistently reinforcing the EDA finding that price and prevalence are the primary drivers of smoking-related mortality
- Pharmacotherapy Prescriptions and Expenditure on Tobacco contribute minimally, suggesting cessation prescriptions alone have limited standalone predictive power for mortality at the population level
- SHAP results are fully consistent with EDA correlation findings, providing model-level confirmation of the observational patterns identified earlier

---

Web Application

Interactive mortality risk prediction tool deployed via Gradio.

In [18]:
```
# ── Saving Model & Scaler ──────────────────────────────────────
import pickle

with open("rf_model.pkl", "wb") as f:
    pickle.dump(rf, f)

with open("scaler.pkl", "wb") as f:
    pickle.dump(scaler, f)

with open("features.pkl", "wb") as f:
    pickle.dump(features, f)
```

In [19]:
```
# ── web app (Local) ──────────────────────────────────────
print("Gradio app is running at: http://127.0.0.1:7860")
print("\nTo run the app:")
print("  1. Open terminal in project folder")
print("  2. Run: python app.py")
print("\nModel files saved:")
print("  rf_model.pkl  - Trained Random Forest model")
print("  scaler.pkl    - Fitted StandardScaler")
print("  features.pkl  - Feature names list")
```

```
Gradio app is running at: http://127.0.0.1:7860

To run the app:
  1. Open terminal in project folder
  2. Run: python app.py

Model files saved:
  rf_model.pkl  - Trained Random Forest model
  scaler.pkl    - Fitted StandardScaler
  features.pkl  - Feature names list
```

Web App Observations

- The Gradio web application successfully loads the trained Random Forest model and generates mortality risk predictions with confidence scores

- Users can interactively adjust six tobacco-related indicators using sliders and receive immediate High or Low mortality risk classification
- The app runs locally at http://127.0.0.1:7860 and can be deployed to HuggingFace Spaces for public access using the requirements.txt file
- The reference table at the bottom of the app provides context for interpreting input values based on the 2004-2014 England dataset ranges

---

Summary and Conclusions

In [20]:
```
# ── Summary Statistics ──────────────────────────────────────────
summary = pd.read_sql("""
    SELECT
        MIN(Year) as Start_Year,
        MAX(Year) as End_Year,
        ROUND(AVG(Value), 0) as Avg_Annual_Deaths,
        MIN(Value) as Min_Annual_Deaths,
        MAX(Value) as Max_Annual_Deaths
    FROM fatalities_clean
    WHERE Sex = 'All'
    AND "Diagnosis Type" = 'All deaths which can be caused by smoking'
    AND Metric = 'Number of observed deaths'
""", conn)

adm_summary = pd.read_sql("""
    SELECT
        ROUND(AVG(Value), 0) as Avg_Annual_Admissions,
        MIN(Value) as Min_Annual_Admissions,
        MAX(Value) as Max_Annual_Admissions
    FROM admissions_clean
    WHERE Sex = 'All'
    AND "Diagnosis Type" = 'All diseases which can be caused by smoking'
    AND Metric = 'Number of admissions'
""", conn)

print("Fatalities Summary:")
print(summary.T.rename(columns={0: "Value"}).to_string())
print("\nAdmissions Summary:")
print(adm_summary.T.rename(columns={0: "Value"}).to_string())
```

```
Fatalities Summary:
                      Value
Start_Year             2004
End_Year               2014
Avg_Annual_Deaths 258633.00
Min_Annual_Deaths    235820
Max_Annual_Deaths    289408

Admissions Summary:
                           Value
Avg_Annual_Admissions 1520729.00
Min_Annual_Admissions    1401878
Max_Annual_Admissions    1713330
```

Key Metrics

Metric	Value
Period Covered	2004 to 2014
Average Annual Smoking Deaths	258,633
Peak Annual Deaths	289,408 (2004)
Lowest Annual Deaths	235,820 (2014)
Total Deaths Reduction	53,588 (18.5% decline)
Average Annual Admissions	1,520,729
Peak Annual Admissions	1,713,330 (2014)
Smoking Prevalence Reduction	25% (2004) to 19% (2014)
Affordability Index Decline	80.5 (2004) to 59.4 (2014)
Best ML Model	Random Forest (CV Accuracy 93.3%)

Summary of Findings

1. Mortality Decline: Smoking-related deaths declined consistently by 18.5% from 2004 to 2014, driven by the 2007 public smoking ban, tobacco taxation, mandatory health warnings and expanded NHS Stop Smoking Services

2. Admissions Paradox: Hospital admissions increased by 22% over the same period, reflecting improved survival rates, better treatment and enhanced health seeking behaviour creating a classic public health paradox where reduced mortality increases healthcare burden

3. Gender Trends: Female smoking-related deaths declined faster than male deaths, reflecting stronger female response to health messaging and cessation services. By 2014 female deaths fell below male deaths reversing the 2004 position

4. Age Groups: The 25-34 age group consistently shows the highest smoking prevalence, representing the primary target for future cessation interventions. The 60 and Over group shows lowest prevalence due to survivorship bias

5. Price and Mortality: Tobacco affordability is the strongest predictor of both smoking prevalence and mortality risk, confirmed by both EDA correlation analysis and SHAP feature importance. Fiscal policy is the most powerful tobacco control tool available to governments

6. Cessation Prescriptions: Overall prescription volumes declined after 2010 reflecting a shift toward self-managed cessation through e-cigarettes and over-the-counter NRT rather than reduced quit attempts

7. Machine Learning: Random Forest achieved 93.3% cross-validation accuracy in classifying high versus low mortality risk years. XGBoost underperformed due to the small dataset size (11 rows). SHAP analysis confirmed tobacco pricing and admissions volume as the primary mortality risk drivers

Limitations

- The ML dataset contains only 11 yearly observations due to data being aggregated at annual level, significantly limiting model reliability and generalisability
- Data covers England only and may not generalise to other countries or healthcare systems
- Admissions data uses fiscal years while fatalities use calendar years, requiring year-level approximation in merging
- Smoking prevalence data has gaps and uses survey-based estimation which carries inherent sampling uncertainty
- The Gradio app is trained on historical aggregate data and should not be used for individual-level clinical prediction

Deployment

The Random Forest model is deployed as an interactive Gradio web application allowing users to adjust tobacco-related indicators and receive immediate mortality risk classification with confidence scores. The app can be deployed to HuggingFace Spaces using the included requirements.txt file.

In [21]:
```
conn.close()
```